ADVANCES IN MARGINAL INFERENCE FOR CHAIN-GRAPH LOGICAL CREDAL NETWORKS

Anonymous authors
Paper under double-blind review

ABSTRACT

Logical Credal Networks (LCNs) are an expressive probabilistic logic framework that combines propositional formulas with imprecise probability bounds to represent sets of probability distributions. LCNs allow cyclic dependencies and impose few syntactic restrictions, but inference is hard: the only exact method optimizes a nonconvex program over all 2^n joint states, and the only approximate method is the ARIEL message-passing algorithm. We introduce Credal Junction Tree (CJT), an exact inference algorithm for LCNs whose structure is a chain graph. CJT replaces the joint distribution by local marginals on the clusters of a junction tree of the chain graph, imposes each probabilistic sentence and independence assertion on a cluster that contains it, and solves a nonconvex program with $O(n\,2^{w+1})$ variables, where $w+1$ is the size of the largest cluster (an upper bound on the treewidth of the moralized chain graph). We prove that, under conditions that can be checked mechanically, its global optimum gives the exact lower and upper probabilities. CJT uses only the structure of the chain graph and requires no compilation into a credal network. As a complementary approach, we compile chain-graph LCNs into credal networks, characterize the family-realizable LCNs on which this compilation preserves the exact bounds, and adapt three credal-network algorithms — variable elimination, interval belief propagation and coordinate descent — with their exactness and bound guarantees. We evaluate all methods on synthetic and realistic LCN instances.

1 INTRODUCTION

Combining logical and probabilistic reasoning is a long-standing goal of Artificial Intelligence. Frameworks such as Markov Logic Networks (Richardson & Domingos, 2006), ProbLog (De Raedt et al., 2007), and Probabilistic Soft Logic (Bach et al., 2017) encode domain knowledge as logical rules augmented with uncertainty, but most of them require *precise* probability values or impose strong structural assumptions such as acyclicity. In many real applications, however, domain expertise yields only *ranges* of plausible probabilities and the rules of interest are neither directed nor acyclic.

Logical Credal Networks (LCNs) (Marinescu et al., 2022) were introduced to address this gap: they allow *imprecise* specifications of the form $\alpha \leq P(\phi \,|\, \varphi) \leq \beta$ over arbitrary propositional formulas, with no acyclicity restriction. An LCN defines a set of probability distributions consistent with its constraints and a local Markov condition, and inference amounts to computing tight lower and upper bounds on the probabilities of query formulas. This flexibility makes LCNs well suited to causal reasoning with partial knowledge, robust decision making, and trustworthy neuro-symbolic systems, but it also makes inference considerably harder than in Bayesian or classical credal networks (Pearl, 1988; Cozman, 2000). The only exact method optimizes a nonconvex program over the full joint distribution, with one variable for each of the 2^n truth assignments (Marinescu et al., 2022), which confines it to a handful of propositions; the only approximate method is the ARIEL message-passing scheme (Marinescu et al., 2023).

Contribution. We show that exact LCN inference need not pay the 2^n cost. Our main contribution is *Credal Junction Tree* (CJT), an exact algorithm for LCNs whose structure is a chain graph. Building on the chain-graph factorization of LCNs established by Cozman (2023), CJT moralizes and triangulates the chain graph, builds a junction tree, and replaces the global joint distribution by one

local marginal per cluster, tied together by separator-consistency constraints. The LCN's probabilistic sentences are imposed on the clusters that contain them, and its independence assertions are either entailed by the tree's topology, reduced to equivalent smaller assertions that fit in a cluster, or implied by one constraint per node of a topologically ordered tree. The resulting nonlinear program has $O(n\,2^{w+1})$ variables instead of 2^n, where $w+1$ is the largest cluster size of the tree it builds, and we prove that, under conditions that the algorithm checks for each instance, its certified global optimum is the exact LCN bound. CJT needs only the chain graph's *structure*: unlike the approaches below, it never compiles the LCN into a credal network, never solves per-family local problems and never enumerates extreme points, so it is not affected by the relaxation those steps introduce.

Our second contribution is a systematic *credal-network compilation* of chain-graph LCNs, which makes the well-developed machinery of credal networks (Cozman, 2000; Mauá & Cozman, 2020; Ide & Cozman, 2008; Antonucci et al., 2015) available for LCN inference. The compilation replaces the LCN's distribution set by the strong extension of a credal network over the chain components, which contains it when the LCN factorizes (as every DAG-structured LCN does); we identify conditions, centred on *family-realizable* sentences, under which the two have the same bounds. We adapt three credal-network algorithms: credal variable elimination (CVE), exact for the strong extension with or without evidence, with its fully polynomial-time approximation scheme CVE$_\varepsilon$; interval belief propagation (IVBP), a credal sum-product algorithm, exact when the compiled credal network is a polytree; and a coordinate-descent method (CDVE) that computes inner bounds for the strong extension. We evaluate all methods on synthetic and realistic LCN benchmarks against CJT as the exact reference.

The Appendix contains the proofs, detailed descriptions of all algorithms with worked examples, and the full experimental results.

2 Background

2.1 Credal Networks

A *credal network* (CN) (Cozman, 2000) is a tuple $\langle \mathbf{V}, \mathbf{\Omega}, \mathbf{K}, D\rangle$, where $\mathbf{V} = \{V_1, \ldots, V_m\}$ is a set of discrete (not necessarily binary) variables with finite domains $\mathbf{\Omega} = \{\Omega_1, \ldots, \Omega_m\}$, D is a directed acyclic graph (DAG) with node set $\mathbf{V}$, and $\mathbf{K} = \{K_{v,\pi}\}$ contains a separately specified *local credal set* $K_{v,\pi}$, a set of distributions over the states of v (Levi, 1980), for each $v \in \mathbf{V}$ and each configuration π of its parents $\mathrm{Pa}(v)$ in D. We consider non-empty local credal sets that are polytopes (closed and convex with finitely many vertices, e.g. probability intervals). Let $\mathcal{P}_{\mathrm{SE}}$ denote the set of joint distributions that factorize along D with every row chosen, separately for each v and π, from its local credal set:

$$\mathcal{P}_{\mathrm{SE}} = \Big\{ \textstyle\prod_{v\in\mathbf{V}} \theta_v(v \mid \mathrm{Pa}(v)) \;:\; \theta_{v,\pi} \in K_{v,\pi}\ \forall v, \pi \Big\}, \tag{1}$$

where $\theta_{v,\pi} = \theta_v(\cdot \mid \mathrm{Pa}(v){=}\pi)$ is the row of the table θ_v at π. The *strong extension* of the credal network is its convex hull (Cozman, 2000; Mauá & Cozman, 2020),

$$\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}\big(\mathcal{P}_{\mathrm{SE}}\big), \tag{2}$$

a polytope whose extreme points are *among* the products of vertices of the local credal sets. $\mathcal{D}_{\mathrm{SE}}$ may contain mixtures violating the independences of D, but both sets have the same lower and upper probabilities, for marginals and for conditionals as defined in Section 2.2 (Lemma 3). Set-level results below concern $\mathcal{P}_{\mathrm{SE}}$.

2.2 Logical Credal Networks

Credal networks attach credal sets to the families of a fixed DAG. A *Logical Credal Network* (LCN) (Marinescu et al., 2022) instead attaches probability bounds to arbitrary propositional formulas, and derives its independences from the formulas. An LCN consists of binary variables (propositions, or atoms) $\mathbf{X} = \{X_1, \ldots, X_n\}$ and a set $\mathcal{S}$ of probabilistic sentences. A sentence $s \in \mathcal{S}$ has the Type-2 form $\ell_s \le P(\varphi_s \mid \psi_s) \le u_s$ over propositional formulas, or the Type-1 form $\ell_s \le P(\varphi_s) \le u_s$ (i.e. $\psi_s = \top$); $\mathrm{atoms}(s)$ is the set of atoms of φ_s and ψ_s, and a flag $\tau_s \in \{\mathrm{True}, \mathrm{False}\}$ declares whether the atoms of φ_s are mutually dependent (it matters only when φ_s has two or more atoms). A ratio is undefined when $P(\psi_s) = 0$, so we use the *cleared* form (Remark 1),

$$c_s^-(p) = P(\varphi_s \wedge \psi_s) - \ell_s P(\psi_s) \;\ge\; 0, \qquad c_s^+(p) = u_s P(\psi_s) - P(\varphi_s \wedge \psi_s) \;\ge\; 0, \tag{3}$$

$0.1 \leq P(B) \leq 0.2$

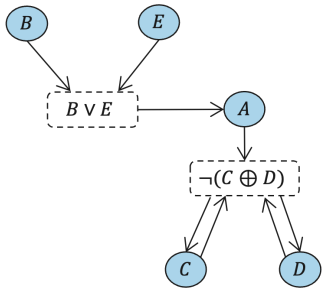
$0.05 \leq P(E) \leq 0.1$

$0.6 \leq P(A|B \vee E) \leq 0.9$

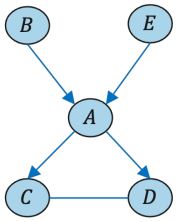
$0.5 \leq P(\neg(C \oplus D)|A) \leq 0.7$

$0.05 \leq P(A) \leq 0.1$

(a) LCN constraints



(b) LCN primal graph



(c) LCN structure (chain graph)

Figure 1: A simple LCN instance with its primal graph and corresponding structure.

which is defined for every distribution p and linear in p.

The *primal graph* G_P of an LCN has a *proposition node* per atom and a *formula node* per formula of a sentence, a single positive literal being identified with its atom's node and self-loops omitted (Fig. 1b). A Type-1 sentence s adds the edges $\varphi_s \to X$, and a Type-2 sentence the edges $X' \to \psi_s$, $\psi_s \to \varphi_s$ and $\varphi_s \to X$, for all $X \in \mathrm{atoms}(\varphi_s)$, $X' \in \mathrm{atoms}(\psi_s)$; if $\tau_s = \mathrm{True}$, the edges between φ_s and its atoms are bi-directed (Marinescu et al., 2022). The *structure* of the LCN is the mixed graph over $\mathbf{X}$ obtained as follows (Cozman, 2023): (i) for each sentence with $\tau_s = \mathrm{True}$, place an undirected edge between any two atoms of φ_s; (ii) for each Type-2 sentence, add a directed edge from each atom of ψ_s to each atom of φ_s; (iii) replace every pair of edges $A \leftrightarrows B$ by an undirected edge; (iv) remove duplicate edges. With $\tau_s = \mathrm{False}$ the atoms of φ_s are thus not linked to each other.

The parents $\mathrm{pa}_P(X)$ of an atom X in G_P are the atoms that reach X via a directed path whose intermediate nodes (if any) are formula nodes. The descendants $\mathrm{de}_P(X)$ are the atoms $Y \neq X$ reachable from X via a directed path none of whose intermediate nodes belongs to $\mathrm{pa}_P(X)$. An LCN is endowed with a Local Markov Condition (LMC(LCN)).

Definition 1 (LMC(LCN))**.** *Given an LCN $\mathcal{L}$ with primal graph G_P, every atom X is independent of its non-parent non-descendants $\mathbf{N}_X = \mathbf{X} \setminus (\{X\} \cup \mathrm{pa}_P(X) \cup \mathrm{de}_P(X))$ given $\mathrm{pa}_P(X)$: the assertion $(X \perp\!\!\!\perp \mathbf{N}_X \mid \mathrm{pa}_P(X))$, for each X with $\mathbf{N}_X \neq \varnothing$.*

An assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ holds in p if $p(x, \mathbf{y}, \mathbf{z})\, p(\mathbf{z}) = p(x, \mathbf{z})\, p(\mathbf{y}, \mathbf{z})$ for *all* configurations $(x, \mathbf{y}, \mathbf{z})$ of $\{X\} \cup \mathbf{Y} \cup \mathbf{Z}$ (Definition 6); configurations with $p(\mathbf{z}) = 0$ satisfy it vacuously. The *LCN distribution set*, a compact subset of the simplex $\Delta(2^{\mathbf{X}})$ of joint distributions of the 2^n truth assignments to $\mathbf{X}$, is

$$\mathcal{D}_{\mathrm{LCN}} = \left\{ p \in \Delta(2^{\mathbf{X}}) \;:\; c_s^-(p) \geq 0,\; c_s^+(p) \geq 0 \;\forall s \in \mathcal{S},\; \text{every LMC assertion holds in } p \right\}. \quad (4)$$

For a query atom a and evidence $\mathbf{e}$, an assignment to a set of atoms $\mathbf{E}$, the inference task asks for $\underline{P}(a \mid \mathbf{e}) = \inf\{p(a \mid \mathbf{e}) : p \in \mathcal{D}_{\mathrm{LCN}},\; p(\mathbf{e}) > 0\}$ and $\overline{P}(a \mid \mathbf{e})$, the corresponding supremum, with the vacuous $[0, 1]$ if no member gives $\mathbf{e}$ positive probability (Definition 7). Marginals are the case $\mathbf{E} = \varnothing$; the bounds are attained when $\min_{p \in \mathcal{D}_{\mathrm{LCN}}} p(\mathbf{e}) > 0$. The same definition applies to any set of distributions, e.g. $\mathcal{D}_{\mathrm{SE}}$. The current state of the art consists of two approaches: the exact nonlinear optimization method of (Marinescu et al., 2022) and the approximate message-passing algorithm called ARIEL (Marinescu et al., 2023), respectively.

Example 1. *Figure 1 shows a simple LCN with 5 propositions and 5 constraints. The corresponding primal graph and structure are given in Figures 1b and 1c, respectively. The LMC implies that: B and E are marginally independent, C is conditionally independent of B and E given its boundary $\{A, D\}$ (its parents in G_P), and D is conditionally independent of B and E given $\{A, C\}$, respectively.*

2.3 Chain Graphs and Factorization

Chain graphs (Cowell et al., 1999) generalize both DAGs and undirected graphs. Let $G = (\mathbf{X}, E)$ be a mixed graph whose nodes are the discrete random variables $\mathbf{X}$, with a configuration of $\mathbf{X}$ denoted by $\mathbf{x}$.

Definition 2 (chain graph)**.** *Let $G = (\mathbf{X}, E)$ be a mixed graph with $E = E_{\rightarrow} \cup E_{-}$, where $E_{\rightarrow}$ is a set of directed edges and E_{-} is a set of undirected edges. The graph G is a* chain graph *if there exists*

a partition $\mathbf{X} = \mathcal{T}_1 \dot\cup \mathcal{T}_2 \dot\cup \cdots \dot\cup \mathcal{T}_m$ *such that: (i) for each* i*, the subgraph induced by* $\mathcal{T}_i$ *contains only undirected edges; (ii) for* $i \neq j$*, every edge between* $\mathcal{T}_i$ *and* $\mathcal{T}_j$ *is directed, and all such edges have the same orientation; (iii) the directed graph obtained by contracting each chain component* $\mathcal{T}_i$ *into a single node is acyclic. The sets* $\mathcal{T}_1, \ldots, \mathcal{T}_m$ *are called the* chain components *of* G*.*

A DAG is a chain graph whose components are all singletons, and an undirected graph is a chain graph with a single component. If $A \to B$ in G then A is a *parent* of B, and $\mathrm{pa}_G(B)$ denotes the parents of B. If $A - B$ then A and B are *neighbors*, and $\mathrm{ne}_G(A)$ denotes the neighbors of A. The *boundary* of a node A is $\mathrm{bd}_G(A) = \mathrm{pa}_G(A) \cup \mathrm{ne}_G(A)$, and that of a set $\mathcal{B}$ of nodes is $\mathrm{bd}_G(\mathcal{B}) = \bigcup_{A \in \mathcal{B}} \mathrm{bd}_G(A) \setminus \mathcal{B}$. A node B is a *descendant* of A, $B \in \mathrm{de}_G(A)$, if there is a path from A to B that follows every directed edge in its direction, uses undirected edges in either direction, and contains at least one directed edge. The Local Markov Condition (LMC(C)) for chain graphs is defined as follows:

Definition 3 (LMC(C))**.** *Let* G *be a chain graph. A node* A *of* G *is independent, given its boundary* $\mathrm{bd}_G(A)$*, of all nodes that are not* A *itself nor in* $\mathrm{de}_G(A)$ *nor in* $\mathrm{bd}_G(A)$*.*

For strictly positive distributions the local and global Markov conditions of a chain graph are equivalent (Cowell et al., 1999), and a strictly positive joint distribution p satisfying LMC(C) factorizes according to G as

$$p(\mathbf{x}) = \prod_{i=1}^{m} p(\mathbf{x}_{\mathcal{T}_i} \mid \mathbf{x}_{\mathrm{bd}_G(\mathcal{T}_i)}), \tag{5}$$

where $\mathrm{bd}_G(\mathcal{T}_i)$ is the set of parents of $\mathcal{T}_i$'s nodes outside $\mathcal{T}_i$, and $\mathbf{x}_{\mathcal{T}_i}$, $\mathbf{x}_{\mathrm{bd}_G(\mathcal{T}_i)}$ are the projections of $\mathbf{x}$ onto these variables. When G is a DAG positivity is not needed (Lemma 4). For components with several variables it is, since deriving the independence of a whole component from the node-wise conditions requires the intersection property (Cowell et al., 1999).

Contracting each chain component into a single node turns G into the *component DAG* D, whose node set $\mathbf{V}$ ($m = |\mathbf{V}|$) is in bijection with the components. For a node $v \in \mathbf{V}$, $\mathrm{ch}_v \subseteq \mathbf{X}$ is the set of atoms of its component, $\mathrm{Pa}(v)$ the set of its parent nodes in D, and $\mathrm{pa}_v = \bigcup_{u \in \mathrm{Pa}(v)} \mathrm{ch}_u$ the set of atoms of its parent components; π denotes an assignment of pa_v. Each v is a variable whose values are the joint states of ch_v, *compound* (and multi-valued) when $|\mathrm{ch}_v| \geq 2$. A distribution p *factorizes along* D if $p(\mathbf{x}) = \prod_v \theta_v(\mathbf{x}_{\mathrm{ch}_v} \mid \mathbf{x}_{\mathrm{pa}_v})$ for *family tables* θ_v whose rows $\theta_{v,\pi}$ are distributions over the states of ch_v; since $\mathrm{bd}_G(\mathcal{T}_i) \subseteq \mathrm{pa}_v$, equation 5 implies this. The pairs $(\mathrm{ch}_v, \mathrm{pa}_v)$ are the *families*, $\mathrm{ch}_v \cup \mathrm{pa}_v$ the *family scopes*, and a sentence s is *attached* to family v if $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$.

Chain-graph LCNs. Cozman (2023) show that when the structure of an LCN is a chain graph over its atoms, LMC(LCN) (Definition 1) coincides with LMC(C), so every strictly positive member of $\mathcal{D}_{\mathrm{LCN}}$ factorizes along D. We call such an LCN a *chain-graph LCN* and consider only these; its components, D and families are those of its structure. The implementation rejects a structure with a directed edge between two atoms of one undirected component that are not joined by an undirected edge (contraction would silently drop it), so every structure it accepts is a genuine chain graph. An LCN is *factorizing* if every $p \in \mathcal{D}_{\mathrm{LCN}}$ factorizes along D. A DAG-structured LCN (every component a single atom) is factorizing, and every product of its family tables satisfies its LMC (Lemma 4). With compound components only the strictly positive members of $\mathcal{D}_{\mathrm{LCN}}$ are guaranteed to factorize (Remark 2 states what the compilation then guarantees).

3 Exact LCN Inference at Treewidth Cost

Exact inference in an LCN computes $\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P_p(a)$ and $\overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P_p(a)$ (attained, $\mathcal{D}_{\mathrm{LCN}}$ being compact) and, under evidence $\mathbf{e}$ on atoms $\mathbf{E}$, the $\inf$ and $\sup$ of $P_p(a \mid \mathbf{e})$ over $\{p \in \mathcal{D}_{\mathrm{LCN}} : p(\mathbf{e}) > 0\}$ (Definition 7). Optimizing over $\mathcal{D}_{\mathrm{LCN}}$ directly requires a nonlinear program with one variable per joint state, 2^n in total (Marinescu et al., 2022). We introduce *Credal Junction Tree* (CJT), which solves the same problem exactly with a program whose size is exponential only in the width of a junction tree of the chain graph.

CJT uses the chain graph only through its family scopes (Section 2). Bucket elimination over these scopes (and over the atoms of every sentence that fits in no family scope) moralizes and triangulates the chain graph and yields a junction tree T whose clusters $C \in \mathcal{C}$ are sets of atoms; $G^*(T)$ is the

graph on the atoms in which two atoms are adjacent iff they share a cluster of T. Instead of one variable per joint state, CJT introduces a local marginal $q_C \in \Delta(2^{|C|})$ for each cluster and solves (Appendices B–C):

$$\begin{aligned} \min / \max \quad & \rho_a(q) \\ \text{s.t.} \quad & \textstyle\sum_i q_{C,i} = 1,\ q_{C,i} \geq 0 && \forall C \in \mathcal{C}, \\ & q_C \restriction S = q_{C'} \restriction S && \forall (C, C') \in T,\ S = C \cap C', \\ & \ell\langle A_\psi, q_h\rangle \leq \langle A_{\varphi\wedge\psi}, q_h\rangle \leq u\langle A_\psi, q_h\rangle && \forall \text{ sentence}, \\ & \langle A_{x\wedge\mathbf{y}\wedge\mathbf{z}}, q_h\rangle\langle A_{\mathbf{z}}, q_h\rangle = \langle A_{x\wedge\mathbf{z}}, q_h\rangle\langle A_{\mathbf{y}\wedge\mathbf{z}}, q_h\rangle && \forall (X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z}) \in \Lambda_T,\ \forall \mathbf{y}, \mathbf{z}, \end{aligned}$$

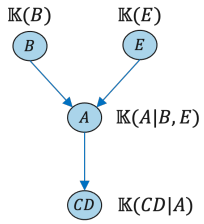
where q_h is the marginal of the cluster C_h hosting the row (a constraint is *hosted* on any cluster containing its atoms), A_φ is the $0/1$ indicator vector of the formula φ over the host cluster's states, $\langle\cdot,\cdot\rangle$ is the dot product, x is the literal $X{=}1$ (the $X{=}0$ rows follow), Λ_T is the set of independence rows CJT imposes (below), and $\rho_a(q) = \langle A_a, q_C\rangle$ reads $P(a)$ off a cluster $C \ni a$. A conditional $P(a \mid \mathbf{e})$ is solved in Charnes–Cooper form ($y_C = r\, q_C$, one scale variable r, one row $\langle A_{\mathbf{e}}, y_C\rangle = 1$, linear objective $\rho_a^{\mathrm{cc}}(y) = \langle A_{a\wedge\mathbf{e}}, y_C\rangle$ on a cluster $C \supseteq \{a\} \cup \mathbf{E}$). The rows for the LMC assertions Λ need care, since an assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ may involve more atoms than any cluster holds. CJT discharges each assertion in one of three ways (Appendix B.2). If $\mathbf{Z}$ separates X from $\mathbf{Y}$ in $G^*(T)$ the assertion is *separator-entailed*: every joint reconstructed from consistent cluster marginals satisfies it, so no row is needed. Otherwise $\mathbf{Y}$ can be replaced by its part $\mathbf{Y}_{\mathrm{in}}$ still connected to X once $\mathbf{Z}$ is removed; the resulting assertion is equivalent to the original for every such joint, and is usually small enough to be hosted, if necessary after widening the tree. Alternatively, eliminating the component DAG D in reverse topological order gives a tree on which the single row $(v \perp\!\!\!\perp N_v \setminus \mathrm{Pa}(v) \mid \mathrm{Pa}(v))$ per node v, with $\{v\} \cup N_v$ its elimination clique, implies every assertion, provided each row follows from one LCN assertion and every LCN assertion is a d-separation of D between unions of whole nodes (Proposition 4; both conditions are checked).

Example 2. *For the LCN of Figure 1, T has the clusters $\{B, E, A\}$, $\{A, C, D\}$, $\{E, A\}$ and $\{E\}$. The assertions $(C \perp\!\!\!\perp B, E \mid A, D)$ and $(D \perp\!\!\!\perp B, E \mid A, C)$ are separator-entailed, because every path from C or D to B or E passes through A; the assertion $(B \perp\!\!\!\perp E)$ is not, since B and E are adjacent in $G^*(T)$, and is imposed on $\{B, E, A\}$ as two bilinear rows. The program has 22 variables and 24 constraints, against $2^5 = 32$ variables for the joint (Appendix C.1).*

The hypotheses H1–H4 below are checked for each instance: H1 by testing that the clusters containing each atom are connected in T, H2 and H4 by searching for hosts; H3 holds by construction unless an assertion had to be relaxed, "implied by Λ" being certified syntactically (each row follows from one LCN assertion with the same conditioning set by decomposition and symmetry). When H4 fails for a query, CJT builds for it a tree augmented by the nodes of the query and the evidence. Each endpoint is solved by spatial branch-and-bound (SCIP) and, if uncertified, reported by its dual bound, so every SCIP interval is outer, and exact (up to tolerance) when H1–H4 hold and both endpoints are certified (Appendix B.3).

Theorem 1 (exactness)**.** *Let $\mathcal{L}$ be a chain-graph LCN with LMC assertions Λ and $\mathcal{D}_{\mathrm{LCN}} \neq \varnothing$, let T be a cluster tree (or forest) over its atoms, let $G^*(T)$ be the graph on the atoms in which two atoms are adjacent iff they share a cluster of T, and let Λ_T be the independence rows imposed on T. Assume* (H1) *for every atom, the clusters of T containing it form a connected subtree of T;* (H2) *every sentence is hosted by a cluster of T;* (H3) *every row of Λ_T is implied by Λ (it holds for every distribution satisfying Λ), and every distribution that factorizes over the cliques of $G^*(T)$ and satisfies Λ_T satisfies Λ;* (H4) *the query atom a and the evidence atoms $\mathbf{E}$ lie in a common cluster. Then the optima of the CJT program are $\min_{p\in\mathcal{D}_{\mathrm{LCN}}} P_p(a)$ and $\max_{p\in\mathcal{D}_{\mathrm{LCN}}} P_p(a)$, and the infimum and supremum of its Charnes–Cooper form are $\inf$ and $\sup$ of $P_p(a \mid \mathbf{e})$ over $\{p \in \mathcal{D}_{\mathrm{LCN}} : p(\mathbf{e}) > 0\}$, the program being infeasible iff that set is empty. Without H1, H2 and the second half of H3, these values are still outer bounds.*

The proof (Appendix C.2) shows two inclusions. Projecting any $p \in \mathcal{D}_{\mathrm{LCN}}$ onto the clusters yields a feasible point with the same objective value (first half of H3), which gives the outer bound. Conversely, by H1 any feasible point is the set of cluster marginals of a joint that factorizes over the clusters (Lemma 6); that joint satisfies every sentence by H2 and every LMC assertion by H3, so it belongs to $\mathcal{D}_{\mathrm{LCN}}$. The reconstruction handles zero separator marginals, where the junction-tree formula has undefined ratios, and uses no positivity assumption.



(a) Credal network

$$\underline{P}(A=1|B=0,E=1)$$
$$\min \frac{P(a \wedge \neg b \wedge e)}{P(\neg b \wedge e)}$$
$$0.1 \le P(b) \le 0.2$$
$$0.05 \le P(e) \le 0.1$$
$$0.6 \le P(a|b \vee e) \le 0.9$$
$$0.05 \le P(a) \le 0.1$$

$$\overline{P}(A=1|B=0,E=1)$$
$$\max \frac{P(a \wedge \neg b \wedge e)}{P(\neg b \wedge e)},$$
$$0.1 \le P(b) \le 0.2$$
$$0.05 \le P(e) \le 0.1$$
$$0.6 \le P(a|b \vee e) \le 0.9$$
$$0.05 \le P(a) \le 0.1$$

(b) Local fractional LPs

B	E	A	$\mathbb{K}(A\|B,E)$
0	0	0	[0.96,1.0]
0	0	1	[0.0,0.04]
0	1	0	[0.0,1.0]
0	1	1	[0.0, 1.0]
…	…	…	…

(c) Local credal set

Figure 2: The credal network for the chain-graph factorization of the LCN from Figure 1.

Theorem 2 (size)**.** *Let $w+1$ be the number of atoms in the largest cluster of T and $m \le n$ the number of nodes of D. The CJT program has $\sum_C 2^{|C|} \le m\, 2^{w+1}$ variables, at most $(m-1)\, 2^w$ separator equalities, two rows per sentence, and $2^{|\mathbf{Y}|+|\mathbf{Z}|} \le 2^w$ rows per imposed assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$, of which $2^{|\mathbf{Z}|}(2^{|\mathbf{Y}|}-1)$ are independent. When H3 holds at most n assertions are imposed, hence $O(n\, 2^w)$ independence rows. Evidence adds one variable and one row.*

Here w is the width of the tree actually built (widening can increase it). The bound concerns the formulation, not the solving time: the program is nonconvex, and its global optimization is not polynomial in its size; but it is exponential in w rather than in n, so exact inference becomes possible where the 2^n formulation cannot even be written down. CJT never compiles a credal network: it uses no local credal sets, and its target is $\mathcal{D}_{\mathrm{LCN}}$ itself. Section 4 shows what is gained, and what is lost, by compiling instead.

4 Approximate Inference via Credal Network Compilation

An alternative to solving the LCN program, exactly as CJT does or approximately, is to compile the LCN into a credal network (Section 2.1) and use the algorithms developed for those. We describe the compilation, characterize when it is exact, and adapt three credal-network algorithms to it.

4.1 The Chain Graph Approximation for LCNs

Given a chain-graph LCN $\mathcal{L}$ over propositions $\mathbf{X} = \{X_1, \ldots, X_n\}$, we approximate $\mathcal{D}_{\mathrm{LCN}}$ by the strong extension $\mathcal{D}_{\mathrm{SE}}$ of an induced credal network over the component DAG D (Section 2): node v is a variable over the joint states of ch_v (multi-valued when compound) with parents $\mathrm{Pa}(v)$. For each assignment π of pa_v, the local credal set $K_{v,\pi}$ (Definition 11) is the set of rows $\rho(\mathrm{ch}_v \mid \pi)$ of the joints ρ on the family scope $\mathrm{ch}_v \cup \mathrm{pa}_v$ that satisfy the sentences *attached* to v ($\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$) and have $\rho(\pi) > 0$ (Marinescu et al., 2022); no LMC equality is imposed. By the Charnes–Cooper transformation (Charnes & Cooper, 1962) it is the projection of a linear program's feasible set, a polytope: an interval for a single-atom child; for a compound child, whose per-state box can be strictly larger, we compute its vertices with an LP oracle. An infeasible program gives, by convention, the simplex. Let $K_v = \prod_\pi K_{v,\pi}$; tables θ_v have rows $\theta_{v,\pi}$.

Example 3. *Figure 2a shows the credal network induced by the chain-graph factorization of the LCN from Figure 1. Any joint distribution in $\mathcal{P}_{\mathrm{SE}}$ factorizes as $P(B) \cdot P(E) \cdot P(A \mid B, E) \cdot P(C, D \mid A)$. In Figure 2b we outline the fractional LPs for computing lower and upper bounds on $P(A = 1 \mid B = 0, E = 1)$, respectively. A partial local credal set $K(A \mid B, E)$ is given in Figure 2c.*

Given a reference credal set $\mathcal{R}$ with true query bounds $[\underline{q}^\star, \overline{q}^\star]$, an estimated interval is *exact* when it equals the true bounds, an *outer bound* when it encloses them, and an *inner bound* when it is enclosed by them. Intervals that satisfy neither property are termed *unreliable*. We can show that:

Proposition 1 (inclusion)**.** *For any factorizing chain-graph LCN, $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{SE}}$. Consequently, any inference algorithm that is* exact *or* outer *for $\mathcal{D}_{\mathrm{SE}}$ computes valid* outer bounds *for $\mathcal{D}_{\mathrm{LCN}}$, for every marginal and every conditional query with $\overline{P}_{\mathcal{D}_{\mathrm{LCN}}}(\mathbf{e}) > 0$ (Definition 7).*

Rows of positive weight are locally feasible, the others free (Appendix E); without factorization, see Remark 2.

4.2 FAMILY-REALIZABLE SENTENCES

The proposed approximation introduces two looseness gaps. First, each $K_{v,\pi}$ is computed per family in isolation, so it may admit rows that no global $p \in \mathcal{D}_{\mathrm{LCN}}$ realizes: it ignores the sentences not attached to v and all LMC equalities, including those inside the family scope. So the rows $K^{\mathrm{true}}_{v,\pi}$ of members of $\mathcal{D}_{\mathrm{LCN}}$ satisfy $K^{\mathrm{true}}_{v,\pi} \subseteq K_{v,\pi}$ (Appendix E), sometimes strictly. Second, even with $K_{v,\pi} = K^{\mathrm{true}}_{v,\pi}$, the free product of equation 1 lets each family pick its rows independently. This can violate a sentence whose scope spans several families, and LMC assertions that constrain a family's table, which a free row does not enforce. We therefore introduce a class of LCN sentences, called *family-realizable sentences*, and identify conditions under which $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, so that $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same bounds.

Definition 4 (family-realizable sentence)**.** *Let $\mathcal{L}$ be a chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$, and let s be a sentence with functional g_s. Say s is* family-realizable in family v *if it is attached to v*[1] *and the value $g_s(p)$ is determined by the conditional table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$* alone*: for any two joints p, p' whose rows agree on every π with $p(\pi), p'(\pi) > 0$, $g_s(p) = g_s(p')$ (for a Type-2 sentence, whenever $P(\psi) > 0$ and $P'(\psi) > 0$), whatever the parents' marginals $P(\mathrm{pa}_v)$ and $P'(\mathrm{pa}_v)$ may be. In particular, s is family-realizable in v if it has one of the two forms:*

- (R1) Type-1 on a root block: *s is $\ell \le P(\varphi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$ and $\mathrm{pa}_v = \varnothing$. Then $P(\varphi)$ is a function of $P(\mathrm{ch}_v)$, which* is *the family's table.*
- (R2) Type-2 pinning one full parent configuration: *s is $\ell \le P(\varphi \mid \psi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$, $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, and ψ satisfied by* exactly one *assignment $\pi^\star$ of pa_v. Then $P(\varphi \mid \psi) = P(\varphi \mid \mathrm{pa}_v{=}\pi^\star)$ is a single row of the table.*

A sentence is non-realizable *if it is family-realizable in no family. The LCN is* fully realizable *if every sentence is family-realizable in some family.*

R1 and R2, the forms our generator emits, are sufficient but not necessary (e.g. $P(C \mid D)$ on a root block $\{C, D\}$). Lemma 8 in the Appendix gives the exact characterization.

Proposition 2 (sufficiency)**.** *Let $\mathcal{L}$ be a consistent chain-graph LCN with exact local sets (no box fallback) that is (i)* locally feasible *(every local program is feasible), (ii)* factorizing *(every $p \in \mathcal{D}_{\mathrm{LCN}}$ factorizes along the component DAG) and (iii)* product-Markov *(every product of family tables along D satisfies the LMC);* DAG-structured *LCNs satisfy (ii) and (iii) (Lemma 4). If every sentence of $\mathcal{L}$ is family-realizable in some family (Definition 4), then $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$. Hence $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same lower and upper probabilities for every marginal and every conditional query $P(a \mid \mathbf{e})$ (Definition 7).*

Under these hypotheses every row of a local set enforces the sentences realizable at it (Appendix, Lemma 10); since a product q of tables has family marginal $q(\pi)\,\theta_{v,\pi}$ (Lemma 9), every sentence holds at q, and q satisfies the LMC by (iii). (iii) fails when the LMC constrains a family's table, inside a row (a component $C - D - E$ with $(C \perp\!\!\!\perp E \mid D)$) or across rows (e.g. $(D \perp\!\!\!\perp Y \mid C)$ for $Y \to \{C, D\}$, or a child depending on part of a compound parent); CJT is then the exact method. The hypotheses and exact polytopes are needed (Appendix E.4).

We next present three algorithms for $\mathcal{D}_{\mathrm{SE}}$: CVE is exact, IVBP is exact on polytree credal networks (a heuristic on loopy ones), and CDVE gives inner bounds. By Proposition 1, the exact ones give outer bounds for $\mathcal{D}_{\mathrm{LCN}}$ when $\mathcal{L}$ is factorizing (exact ones under Proposition 2); inner bounds for $\mathcal{D}_{\mathrm{SE}}$ transfer to $\mathcal{D}_{\mathrm{LCN}}$ only when the two have the same bounds.

4.3 CREDAL VARIABLE ELIMINATION

Credal Variable Elimination (CVE) extends Variable Elimination (VE) for Bayesian networks (Dechter, 2003) to credal networks by propagating finite sets of *pairs* of non-negative functions instead of single functions (Mauá et al., 2012; Mauá & Cozman, 2020).

Definition 5 (pair potential)**.** *A* potential $\phi(\mathbf{Y})$ *over nodes $\mathbf{Y}$ is a finite set of pairs (p, q) of non-negative functions on the joint domain of $\mathbf{Y}$. The* product *of two potentials is $\phi \cdot \psi = \{(pr, qs) :$*

[1] A sentence can be attached to several families while being realizable in only one of them.

$(p, q) \in \phi,\ (r, s) \in \psi\}$ *over* $\mathbf{Y} \cup \mathbf{Z}$*, and the* sum-marginal $\sum_X \phi = \{(\sum_x p, \sum_x q) : (p, q) \in \phi\}$ *sums both components.*

For a query atom a of node Z, let t be the $0/1$ indicator of a over the states of Z. Every family table enters as $\{(f, f)\}$, one f per choice of a vertex of every row's local set $K_{v,\pi}$, and the evidence $\mathbf{e}$ as $0/1$ indicators multiplied into both components. The query potential $\{(1-t,\ t)\}$ splits the mass, so that after all nodes are eliminated each pair carries $p = p_\theta(\neg a, \mathbf{e})$ and $q = p_\theta(a, \mathbf{e})$ for one product p_θ of vertices. The bounds are the minimum and maximum of $q/(p+q)$ over the final pairs with $p + q > 0$, and $[0, 1]$ if there is none. CVE eliminates Z last, so every pair has $p = q$ until the query enters; one elimination per node serves all of Z's atoms, and compound nodes need no projection. Every query-free potential, including every intermediate product of a bucket, is reduced to the pairs whose rays $f/\sum f$ are vertices of the convex hull of its rays: $q/(p+q)$ is linear-fractional in each such function, so its extrema are attained at vertices (Lemma 3, Appendix F).

FPTAS. Since the intermediate potentials can grow exponentially, we use the fully polynomial-time approximation scheme of Mauá et al. (2012). For $\alpha > 1$, two pairs, each scaled to total mass 1, are α*-equivalent* when every coordinate of both components is zero in both, or positive in both with the same $\lfloor \log_\alpha \cdot \rfloor$. The α*-quotient* keeps at most two real pairs per class and is applied after every pairwise product, every summation and the query step. We denote this variant CVE_ε.

Theorem 3 (complexity)**.** *For one query node, CVE runs in time $O(m \cdot C^2 \cdot d^{w^*+1})$ plus the cost of the hull reductions, and space $O((C+m)\, C\, d^{w^*+1})$, where m is the number of nodes, d bounds their domain sizes, w^* is the induced width of the elimination order, and C is the maximum number of pairs of an input or reduced potential (which can be exponential); all marginals cost a factor m more. For CVE_ε, C is polynomial in the size of the compiled network and in $1/\varepsilon$ when w^* and d are bounded (Mauá et al., 2012).*

Theorem 4 (exactness)**.** *Let $\mathcal{L}$ be a chain-graph LCN, a an atom and $\mathbf{e}$ evidence with $\overline{P}(\mathbf{e}) > 0$. CVE computes the infimum and supremum of $p(a \mid \mathbf{e})$ over the $p \in \mathcal{D}_{\text{SE}}$ with $p(\mathbf{e}) > 0$ (Definition 7), exactly for the compiled local sets (Definition 11; outer for $\mathcal{D}_{\text{SE}}$ on the exact sets when a compound row keeps the box fallback) up to the floating-point precision of the enumerated vertices. If $\overline{P}(\mathbf{e}) = 0$ it returns $[0, 1]$. If $\mathcal{L}$ also satisfies the hypotheses of Proposition 2 and no row keeps the box, these are the bounds with respect to $\mathcal{D}_{\text{LCN}}$.*

Proposition 3 (FPTAS guarantee)**.** *Under the assumptions of Theorem 4, let N be the number of α-quotients in the elimination for a's node, and let $\alpha = (1+\varepsilon)^{1/(2N)}$. Then CVE_ε returns* inner *bounds within a factor $1+\varepsilon$ of CVE's $[\underline{P}, \overline{P}]$: $\overline{P}/(1+\varepsilon) \le \overline{P}_\varepsilon \le \overline{P}$ and $\underline{P} \le \underline{P}_\varepsilon \le (1+\varepsilon)\,\underline{P}$.*

Each reduction keeps, for every way the rest of the elimination can weight a pair, a pair at least as good for each bound, exactly for hull pruning and up to α^2 for a quotient; the factors multiply along the elimination (Lemma 12).

4.4 Interval Belief Propagation

Our second approach, Interval Belief Propagation (IVBP), runs the sum-product algorithm (Kschischang et al., 2001) on the *factor graph* of the credal network. There is one variable node per credal-network node V, one family factor $f_V(V, \text{Pa}(V))$ holding the local sets $K_{V,\pi}$, and one evidence factor holding the $0/1$ indicator of the observed atoms of V. For a fixed choice of the rows, sum-product sends

$$\mu_{v\to f}(x) = \prod_{g \in \mathcal{N}(v)\setminus f} \mu_{g\to v}(x), \qquad \mu_{f\to v}(x) = \sum_{\mathbf{x}_{\mathcal{N}(f)\setminus v}} \phi_f(x, \mathbf{x}_{\mathcal{N}(f)\setminus v}) \prod_{w \in \mathcal{N}(f)\setminus v} \mu_{w\to f}(x_w), \tag{6}$$

and the belief $b_v \propto \prod_{g\in\mathcal{N}(v)} \mu_{g\to v}$ equals $p(V \mid \mathbf{e})$ on a tree. As the rows range over their local sets, each message ranges over a set. Messages only matter up to scale, so IVBP propagates a *credal message*: the finite set of vertices of the convex hull of the normalized message vectors. For a binary node this is an interval, the messages to a child and to a parent play the roles of the π- and λ-messages of 2U (Fagiuoli & Zaffalon, 1998), and on binary polytrees the output is that of 2U.

Every update is a projective map (linear, then normalized) of each incoming message and of each row separately, which maps the convex hull of a set into the convex hull of its image, so the vertices

are enough. The variable-to-factor update multiplies the vertices of the incoming sets. The update towards a child is linear in the rows, so for each combination of parent vertices it is a Minkowski sum of the rows' vertex sets. The update towards a parent U is a ratio in which each coordinate u depends only on the rows with $\pi_U = u$, so for each combination of incoming vertices it ranges over a box, whose normalized corners are the candidates. Vectors of zero mass are dropped; an empty belief means $\overline{P}(\mathbf{e}) = 0$, and IVBP then reports $[0, 1]$ for every atom, like CVE and CDVE. Messages are swept in topological and reverse topological order until they no longer change; on a loopy graph, until the change falls below $\tau = 10^{-6}$ or for at most $T = 100$ sweeps, as in loopy 2U (Ide & Cozman, 2008).

Theorem 5 (complexity)**.** *One sweep of IVBP costs $O\big(d \sum_V \big(C^{\pi}_V M_V e_V + \sum_{U \in \mathrm{Pa}(V)} C^{\lambda}_{V,U}(E_V + 2^d)\big)\big)$ arithmetic operations plus the products and hull reductions, where C^{π}_V ($C^{\lambda}_{V,U}$) counts the combinations of incoming message vertices for the message of f_V to V (to U), e_V and E_V are the maximum and total numbers of row vertices of V, M_V is the total size of the partial Minkowski-sum hulls, and d the maximum domain size; these counts can be exponential. On a polytree at most $D \leq m$ sweeps reach the fixed point and one more confirms it, D the largest number of family factors on a path.*

Theorem 6 (exactness)**.** *Let $\mathcal{L}$ be a chain-graph LCN whose compiled credal network is a polytree, a an atom and $\mathbf{e}$ evidence with $\overline{P}(\mathbf{e}) > 0$. IVBP computes the infimum and supremum of $p(a \mid \mathbf{e})$ over the $p \in \mathcal{D}_{\mathrm{SE}}$ with $p(\mathbf{e}) > 0$, for binary and compound nodes alike, with the qualifications of Theorem 4 (compiled local sets, floating-point precision); if $\overline{P}(\mathbf{e}) = 0$ it returns $[0, 1]$. Under the hypotheses of Proposition 2, with no box fallback, these are the bounds with respect to $\mathcal{D}_{\mathrm{LCN}}$.*

On a polytree the inputs of every update depend on disjoint sets of rows, so they vary independently, and every update is linear in each input before normalization; the hull of each message set is therefore propagated without loss, and every computed vertex is attained (Lemma 15). On loopy structures the inputs share parameters, and IVBP is a heuristic with no guarantee in either direction. With multi-valued nodes the vertex sets can grow exponentially, consistently with the NP-hardness of exact polytree inference for ternary variables (Mauá et al., 2014). For binary nodes every message has at most two vertices.

4.5 Coordinate Descent with Variable Elimination

Our third approach, Coordinate Descent with Variable Elimination (CDVE), follows ApproxLP (Antonucci et al., 2015). It optimizes the multilinear objective by coordinate descent over the local credal sets, one family at a time, and solves each local problem exactly. For an atom a of a node Z and evidence $\mathbf{e}$, the objective is $p(a \mid \mathbf{e}) = \sum_{s \models a} p(Z{=}s, \mathbf{e}) / \sum_s p(Z{=}s, \mathbf{e})$. With every family but V fixed, both sums are linear in the rows of V: $p(\cdot, \mathbf{e}) = \sum_{x,\pi} \theta_{V,\pi}(x)\, T_V(x, \pi, Z)$, where T_V is the product of all other factors and of the evidence indicators, summed by variable elimination onto $\{V\} \cup \mathrm{Pa}(V) \cup \{Z\}$. The local problem is therefore the linear-fractional program $\max_r \sum_\pi \langle n_\pi, r_\pi \rangle / \sum_\pi \langle d_\pi, r_\pi \rangle$ over the product of the rows' credal sets. Dinkelbach's method solves it exactly: for a given ratio t, every row independently takes the vertex that maximizes $\langle n_\pi - t\, d_\pi, v \rangle$, and t is updated until it no longer increases. A sweep applies this step to every family that is an ancestor of Z or of the evidence; the other families are barren and do not affect $p(Z, \mathbf{e})$. Sweeps repeat until none improves, at most $T = 50$ times, for the senses $\min$ and $\max$ and for every non-evidence atom, atoms of compound nodes included.

Descent starts at the *vertex average* of every row (the mean of its vertices), a symmetric point that is often a saddle, so from there a family moves to a best vertex choice even without improvement. CDVE also restarts from $R = 8$ random vertex choices and keeps the best result. Starts ending with $p(\mathbf{e}) = 0$ are discarded; all are discarded only when $\overline{P}(\mathbf{e}) = 0$, and CDVE then returns $[0, 1]$. Every point kept is a member of $\mathcal{P}_{\mathrm{SE}}$ with $p(\mathbf{e}) > 0$, so the result is an *inner* bound for $\mathcal{D}_{\mathrm{SE}}$. It is an inner bound for $\mathcal{D}_{\mathrm{LCN}}$ when $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same bounds, e.g. under the hypotheses of Proposition 2; otherwise its relation to the LCN interval is not determined.

Theorem 7 (complexity)**.** *CDVE runs in time $O\big(2A\,(R{+}1)\big(T \sum_V (m\, d^{w^*+1} + E_V\,(d + I)) + m\, d^{w^*+1}\big)\big)$ and space $O(m\, d^{w^*+1})$, where A is the number of atoms, m the number of nodes, d the maximum domain size, w^* the induced width of the elimination orders, E_V the total number of row vertices of V, and I the number of Dinkelbach iterations per step, which is finite.*

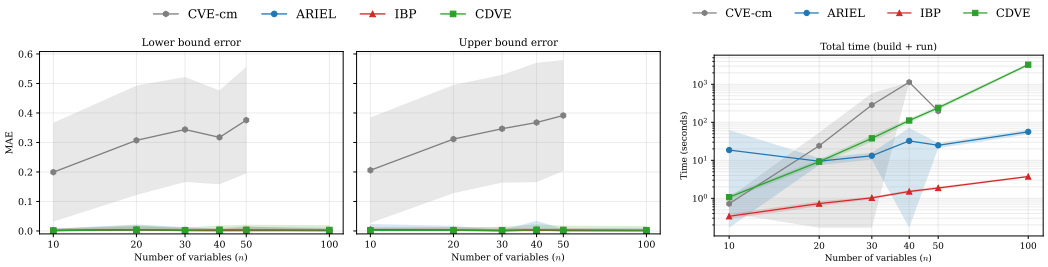


Figure 3: Large random `dag` LCNs: (left) MAEs against the CJT reference, with bands giving one standard deviation over 10 instances per size – curves stopping where the budget was exceeded; (right) total time in seconds.

Instance	Algorithm	MAE_l	MAE_u	Time (s)
Alarm (n=10), CJT ref 0.72s	ARIEL	0.000	0.000	3.92
	IVBP	0.101	0.015	0.13
	CDVE	0.101	0.015	0.69
	CVE_{cm}	0.087	0.067	0.24
Credit (n=11), CJT ref 1.15s	ARIEL	0.050	0.000	3.18
	IVBP	0.257	0.000	0.24
	CDVE	0.257	0.000	0.69
	CVE_{cm}	0.108	0.165	1.73
Hepatitis (n=10), CJT ref 0.56s	ARIEL	0.008	0.008	3.91
	IVBP	0.069	0.080	0.75
	CDVE	0.070	0.085	1.15
	CVE_{cm}	0.114	0.184	16.53
MScaling (n=12), CJT ref 0.66s	ARIEL	0.028	0.000	8.13
	IVBP	0.129	0.493	0.35
	CDVE	0.129	0.501	1.23
	CVE_{cm}	0.079	0.227	0.57
BScaling (n=32), CJT ref 2.42s	ARIEL	0.000	0.000	19.80
	IVBP	0.115	0.225	0.69
	CDVE	0.115	0.225	20.55
	CVE_{cm}	0.064	0.072	9.18
PScaling (n=40), CJT ref 3.27s	ARIEL	0.000	0.000	34.72
	IVBP	0.129	0.263	0.91
	CDVE	0.129	0.263	51.25
	CVE_{cm}	0.070	0.057	658.84

Table 1: Realistic LCNs: three instances derived from real Bayesian networks (top) and three Supreme Court chain-graph instances (bottom), all against the CJT reference. Time is total time including the credal-network build; the reference's own time on each instance is given with the instance name.

5 Experiments

In this section, we evaluate the proposed marginal inference schemes on several classes of LCNs, comparing them with the exact method of Marinescu et al. (2022) and the approximate ARIEL algorithm (Marinescu et al., 2023). All algorithms are implemented in Python 3.12 and use `ipopt` 3.14 (Wächter & Biegler, 2006) for the nonlinear programs. Every run has a limit of 3600 seconds and 30 GB of memory; a dash in a table means the algorithm did not finish inside that budget.

Benchmarks. We consider two types of LCN instances. *Random LCNs* come in four topologies, each generated so that every sentence is *family-realizable*: `tree`, `polytree` (skeleton a forest, at most two parents per atom), `ktree` (k=3) and `dag` (moralized treewidth ≤ 3, at most two parents per atom). Each topology is generated at two scales, *small* ($n \in \{5, \ldots, 10\}$) and *large* ($n \in \{10, 20, 30, 40, 50, 100\}$), with 10 instances per size, giving $8 \times 60 = 480$ instances. Sentence bounds are drawn as $[\max(0.01, v-\varepsilon), \min(1, v+\varepsilon)]$ with $v \sim U(0,1)$ and $\varepsilon = 0.3$. Each atom gets one sentence, a prior if it is a root and otherwise a conditional on the full conjunction of signed literals over its parents (form R2), so a sentence spans at most $1+\max|\text{pa}|$ atoms (3, and 4 for `ktree`); two extra interval bounds on root marginals are added (both on the single root of a `tree`);

the seed is 42. Every instance is consistent: the fully independent product that sets each atom to the midpoint of its interval satisfies every sentence, and trivially the LMC. Every instance passes the structural part of the CJT exactness gate; whether each endpoint is certified is an output of the reference run (Appendix I.1). *Almost-realizable LCNs* add $m \in \{0, 1, 2, 4, 8\}$ non-realizable sentences to `tree` and `dag` instances (Appendix I.2). *Realistic LCNs* are of two kinds: ten instances derived from real Bayesian networks (Constantinou et al., 2020) ($n = 4$–11, 6–24 sentences), and sixteen instances from the causal chain graphs of the US Supreme Court analysis of Ogburn et al. (2020). The latter stress two different axes: the `scaling` instances are large and sparse ($n = 6$–40, 11–79 sentences), whereas the `supreme_court` instances are small but extremely dense (n=10 with 345–1009 sentences).

Algorithms, reference and metrics. We compare four approximate algorithms: ARIEL (Marinescu et al., 2023), which operates directly on the LCN factor graph (10 iterations) and needs no credal-network build; IVBP, interval belief propagation on the credal network (100 iterations); CDVE, coordinate descent over extreme points (50 iterations), which by construction returns *inner* bounds; and CVE$_{cm}$, Credal Variable Elimination with its intermediate potentials clustered to at most 10 functions using the mean representative (an earlier heuristic, replaced by CVE$_{\varepsilon}$ in the rerun).

Plain CVE (exact over the strong extension, no clustering) is included where it terminates. All credal-network methods share the same build: the chain-graph factorization, no scope merging, and per-family local credal sets solved as linear programs. As the reference we use CJT, the junction-tree NLP formulation (using the `ipopt` backend): it is the tightest engine tractable at every size here, whereas exact inference of Marinescu et al. (2022) over the 2^n joint solves only $15/60$ of `tree_small` and none of the Supreme Court instances. We report the mean absolute error of the lower and upper marginal bounds against the reference, $\mathrm{MAE}_l = \frac{1}{n}\sum_i |\underline{P}^*(x_i) - \underline{P}(x_i)|$ and MAE_u analogously, and wall-clock time in seconds including the credal-network build (CJT needs no vertex enumeration). All queries are marginals without evidence. The random instances satisfy Proposition 2, so CVE is exact on all of them, and IVBP where the compiled network is singly connected: every `tree` instance, but only $42/60$ `polytree_small` and $38/60$ `polytree_large` ones.

Random LCNs. Figure 3 shows the loopy `dag` LCNs at the large scale, with shaded bands giving one standard deviation over the ten instances per size (full results on all LCNs are included in the Appendix). Three findings hold across all four topologies. First, IVBP dominates: its error against the reference is at or below 4×10^{-3} on every scale (small, large) and size — indistinguishable from CJT on `tree` and on the singly connected `polytree` instances, as Theorem 6 predicts — while being the fastest method everywhere, by one to three orders of magnitude at n=100. Second, CDVE matches that accuracy but not that cost: its mean error stays in the 10^{-3} range, yet its runtime grows from under a second on the small instances to 273–614 seconds on the large ones, making it slower than ARIEL at n=100. ARIEL is likewise accurate (exactly zero error on `tree` and `polytree`) but pays 25–35 seconds at scale. Third, approximating the potentials by clustering buys speed at a real cost in quality: CVE$_{cm}$ runs in a fraction of a second on the small instances but its error grows with n, from 0.08–0.11 on the small instances to 0.31–0.35 on the large ones. Plain CVE is the scalability bottleneck: it is essentially exact where it runs but costs 226–1141 seconds, times out on all of `ktree_small` and `polytree_large`, and is intractable on the large loopy LCNs.

Realistic LCNs. Table 1 reports six representative instances. These instances are chain-graph LCNs, but they are not family-realizable, so the hypotheses of Proposition 2 do not hold. The ordering is different here: ARIEL is clearly the tightest method, at or near zero error on four of the six instances, because it reasons on the original LCN rather than on its credal-network relaxation. IVBP and CDVE agree closely and produce widths 1.4–7.7$\times$ the reference, with no outer guarantee here (CDVE is inner for $\mathcal{D}_{\mathrm{SE}}$; compound LCNs need not factorize, Appendix E.4). On running time the picture is again favourable to IVBP, which is the fastest method on every instance and the only one that stays under a second at n=40, where CDVE needs 51 seconds and CVE$_{cm}$ 659.

6 Related Work

Exact inference in LCNs has so far been limited to constrained nonlinear optimization over the full joint distribution (Marinescu et al., 2022), and approximate inference to the ARIEL message-passing algorithm (Marinescu et al., 2023); related work has addressed abductive reasoning (Marinescu et al., 2024) and the chain-graph factorization of LCNs (Cozman, 2023). CJT turns that factorization

into an exact algorithm. It is related to junction-tree methods for Bayesian networks (Lauritzen & Spiegelhalter, 1988; Jensen et al., 1990), which compute marginals by propagating messages over a cluster tree; CJT uses the same running-intersection structure, but, since an LCN defines a nonconvex set of distributions rather than a single one, it imposes calibration as constraints of one optimization problem rather than computing it by message passing, and it must additionally discharge the LCN's independence assertions on the tree. Credal networks (CNs) (Cozman, 2000) generalize Bayesian networks by replacing precise conditional probabilities with credal sets; despite the hardness of exact inference (Mauá & Cozman, 2020; Mauá et al., 2014), the area has produced a rich algorithmic toolbox, including 2U/L2U (Ide & Cozman, 2008), GL2U (Antonucci et al., 2010) and ApproxLP (Antonucci et al., 2015). Our compilation makes this toolbox applicable to LCNs, and the family-realizability condition identifies when doing so loses no precision. LCNs also relate to probabilistic logics such as ProbLog, DeepProbLog (Manhaeve et al., 2021), MLNs and PSL, but allow imprecise assessments.

7 Conclusion

We introduced Credal Junction Tree (CJT), an exact inference algorithm for chain-graph Logical Credal Networks that replaces the 2^n-variable joint formulation by a nonlinear program over junction-tree cluster marginals, with size exponential in the treewidth rather than in the number of propositions. CJT needs only the chain graph's structure; it discharges the LCN's independence assertions on the tree by separation, exact reduction or an ordered construction, and we proved that under conditions it checks mechanically its certified optimum gives the exact lower and upper probabilities. As a complementary route, we compiled chain-graph LCNs into credal networks, characterized the family-realizable LCNs on which this compilation preserves the exact bounds, and adapted variable elimination (with an FPTAS), interval belief propagation and coordinate descent to it, with exactness guarantees for marginal and conditional queries under regular conditioning. CJT also provides an exact reference against which the approximate methods can be measured. Next steps include compiled representations for compound families whose tables the LMC constrains.

AI use statement

We used generative AI tools to assist with language editing, grammar correction, and code development. In particular, AI-assisted coding tools were used during the implementation of the proposed algorithms. All AI-generated content was reviewed, verified, and, where applicable, extensively tested by the authors. For example, AI-generated code was manually inspected for correctness and validated through unit and integration testing. The authors take full responsibility for the accuracy, validity, and integrity of all content presented in this paper.

Ethics statement

This work did not involve human subjects.

Reproducibility statement

To ensure reproducibility, we have included the Python code and benchmark problem instances in the supplementary material. We will open-source the code and benchmarks if the paper is accepted.

References

A. Antonucci, C. P. de Campos, D. Huber, and M. Zaffalon. Approximate credal network updating by linear programming with applications to decision making. *International Journal of Approximate Reasoning*, 58:25–38, 2015.

Alessandro Antonucci, Yi Sun, Cassio De Campos, and Marco Zaffalon. Generalized loopy 2U: A new algorithm for approximate inference in credal networks. *International Journal of Approximate Reasoning*, 51(5):474–484, 2010.

Stephen Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. Hinge-loss markov random fields and probabilistic soft logic. *Journal of Machine Learning Research (JMLR)*, 18(1):1–67, 2017.

A. Charnes and W. W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3-4):181–186, 1962.

Anthony Constantinou, Yang Liu, Kiattikun Chobtham, Zhigao Guo, and Neville Kitson. The bayesys data and bayesian network repository. Technical report, Bayesian Artificial Intelligence research lab, Queen Mary University of London, London, UK, 2020.

Robert Cowell, Philip Dawid, Steffen Lauritzen, and David Spiegelhalter. *Probabilistic Networks and Expert Systems*. Springer-Verlag, New York, 1999.

Fabio Cozman. Credal networks. *Artificial Intelligence*, 120(2):199–233, 2000.

Fabio Gagliardi Cozman. Markov conditions and factorization in logical credal networks. In Enrique Miranda, Ignacio Montes, Erik Quaeghebeur, and Barbara Vantaggi (eds.), *Proceedings of the Thirteenth International Symposium on Imprecise Probability: Theories and Applications*, volume 215 of *Proceedings of Machine Learning Research*, pp. 130–140. PMLR, 11–14 Jul 2023. URL https://proceedings.mlr.press/v215/cozman23a.html.

Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. Problog: A probabilistic prolog and its application in link discovery. In *Proceedings of the International Joint Conference on Artificial Intelligence*, volume 7, pp. 2462–2467, 2007.

R. Dechter. Bucket elimination: A unifying framework for probabilistic inference algorithms. In *Uncertainty in Artificial Intelligence (UAI'96)*, pp. 211–219, 1996.

R. Dechter. *Constraint Processing*. Morgan Kaufmann Publishers, 2003.

Enrico Fagiuoli and Marco Zaffalon. 2U: An exact interval propagation algorithm for polytrees with binary variables. *Artificial Intelligence*, 106(1):77–107, 1998.

Jaime Ide and Fabio Cozman. Approximate algorithms for credal networks with binary variables. *International Journal of Approximate Reasoning*, 48(1):275–296, 2008.

F.V. Jensen, S.L Lauritzen, and K.G. Olesen. Bayesian updating in causal probabilistic networks by local computation. *Computational Statistics Quarterly*, 4:269–282, 1990.

Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

S.L. Lauritzen and D.J. Spiegelhalter. Local computation with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society, Series B*, 50(2): 157–224, 1988.

Steffen L. Lauritzen. *Graphical Models*. Oxford University Press, 1996.

I. Levi. *The Enterprise of Knowledge*. MIT Press, 1980.

Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Neural probabilistic logic programming in deepproblog. *Artificial Intelligence*, 298:103504, 2021. ISSN 0004-3702. doi: https://doi.org/10.1016/j.artint.2021.103504. URL https://www.sciencedirect.com/science/article/pii/S0004370221000552.

Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, Ryan Riegel, and Pravinda Sahu. Logical credal networks. In *36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

Radu Marinescu, Haifeng Qian, Alexander Gray, Debarun Bhattacharjya, Francisco Barahona, Tian Gao, and Ryan Riegel. Approximate inference in logical credal networks. In *32nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.

Radu Marinescu, Fabio Cozman, Debarun Bhattacharjya, Junkyu Lee, and Alexander Gray. Abductive reasoning in logical credal networks. In *38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Denis Mauá and Fabio Cozman. Thirty years of credal networks: Specifications, algorithms and complexity. *International Journal of Approximate Reasoning*, 1(126):133–137, 2020.

Denis D. Mauá, Cassio P. de Campos, and Marco Zaffalon. Updating credal networks is approximable in polynomial time. *International Journal of Approximate Reasoning*, 53(8):1183–1199, 2012.

Denis D. Mauá, Cassio P. de Campos, Alessio Benavoli, and Alessandro Antonucci. Probabilistic inference in credal networks: New complexity results. *Journal of Artificial Intelligence Research*, 50, 2014.

Elizabeth L. Ogburn, Ilya Shpitser, and Youjin Lee. Causal inference, social networks, and chain graphs, 2020. URL `https://arxiv.org/abs/1812.04990`.

Judea Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1-2): 107–136, 2006.

Andreas Wächter and Lorenz Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.

A PRELIMINARIES

A Logical Credal Network specifies imprecise probabilistic knowledge by bounding the probabilities of propositional formulas: a *sentence* is either $\ell \le P(\varphi) \le u$ (Type-1) or $\ell \le P(\varphi \mid \psi) \le u$ (Type-2), over propositional formulas φ, ψ on binary atoms. Together with the independence assertions that the Local Markov Condition derives from the program's structure, a set of sentences determines a compact set $\mathcal{D}_{\mathrm{LCN}}$ of joint distributions, and the inference task is to compute, for each atom a and evidence $\mathbf{e}$, the lower and upper probabilities $\underline{P}(a \mid \mathbf{e})$ and $\overline{P}(a \mid \mathbf{e})$ of Definition 7 (an infimum and a supremum over the members of $\mathcal{D}_{\mathrm{LCN}}$ that give $\mathbf{e}$ positive probability).

This paper relates three ways of computing these bounds. The *direct* exact method of Marinescu et al. (2022) writes one variable per joint state and optimizes over $\mathcal{D}_{\mathrm{LCN}}$ itself; it is exact when its solver certifies global optimality, but its 2^n formulation confines it to small instances. CJT, the main contribution (Section 3), also optimizes over $\mathcal{D}_{\mathrm{LCN}}$, but through cluster marginals of a junction tree, at cost $\sum_C 2^{|C|} = O(n\, 2^{w+1})$; it consumes only the chain graph's *structure* and the LCN's own sentences and independence assertions, and under the hypotheses of Theorem 1 it is exact (Appendices B–C). The *credal-network* methods (CVE, IVBP, CDVE; Section 4) first compile the LCN into a credal network whose local credal sets are intervals (single-atom children) or polytopes (compound children), and then propagate over the strong extension $\mathcal{D}_{\mathrm{SE}}$. $\mathcal{D}_{\mathrm{SE}}$ contains $\mathcal{D}_{\mathrm{LCN}}$ when the LCN is factorizing (Proposition 5; for compound components only the closure of the strictly positive part is guaranteed, Remark 2); these methods scale differently, and are exact for $\mathcal{D}_{\mathrm{LCN}}$ only on family-realizable instances that satisfy the hypotheses of Proposition 6 (e.g. DAG-structured ones; Appendices D–E).

A.1 DEPENDENCY MAP OF THE RESULTS

Table 2 lists, for each formal result, the results its proof uses, so that the chain of arguments can be verified bottom-up. All results use the definitions of this appendix: conditional independence (Definition 6), the cleared sentences equation 8, $\mathcal{D}_{\mathrm{LCN}}$ equation 9, the conditional bounds (Definition 7) and, for the credal-network results, the local credal sets (Definition 11).

A.2 NOTATION

Throughout, boldface $\mathbf{X} = \{X_1, \ldots, X_n\}$ denotes the *binary* variables — the propositions, or atoms — of an LCN, and $\mathbf{V}$ the nodes of the component DAG, one per chain component (a compound node takes values in the joint states of its component's atoms). For an atom set $\mathbf{A} \subseteq \mathbf{X}$, $\mathbf{a}$ denotes a configuration of $\mathbf{A}$, $p(\mathbf{a})$ its probability under the joint p, and $p \restriction \mathbf{A}$ the marginal of p on $\mathbf{A}$. For a propositional formula φ we write $\mathrm{atoms}(\varphi)$ for its atoms, $\omega \models \varphi$ for satisfaction, and $P(\varphi) = \sum_{\omega \models \varphi} p(\omega)$. Table 3 fixes one meaning per symbol.

Table 2: Which result uses which. "F⇒G" is the factorization ⇒ global Markov property of Lemma 1(v).

Result	Uses	Used by
Lemma 1 (semi-graphoid, F⇒G)	Definition 6	Lemmas 4, 7; Proposition 4; Theorem 8
Lemma 2 (locality)	equation 9	Lemma 8; Theorem 8
Lemma 3 (hull)	—	Remark 2; Propositions 5, 6; Lemmas 12, 15; Theorems 4, 6
Lemma 4 (DAG structure)	Lemma 1	Propositions 5, 6, 4
Remark 2 (positive part)	Lemma 3(a); Definition 11; Cozman (2023); Cowell et al. (1999)	Proposition 5
Proposition 5 (inclusion)	Lemmas 3, 4; Remark 2	Theorems 4, 6 (outer for $\mathcal{D}_{\mathrm{LCN}}$)
Lemma 10 (row-wise enforcement)	Lemma 8; Definition 11	Proposition 6
Proposition 6 (sufficiency)	Lemmas 3, 4, 9, 10; Proposition 5	Theorems 4, 6 (exact for $\mathcal{D}_{\mathrm{LCN}}$)
Lemma 5 (Helly)	—	Lemmas 6, 7; Proposition 4
Lemma 6 (RIP reconstruction)	Lemma 5	Theorem 8
Lemma 7 (reduction)	Lemmas 1, 5	Theorem 8
Proposition 4 (ordered tree)	Lemmas 1, 4, 5	Theorem 8
Theorem 8 (CJT exactness)	Lemmas 1, 2, 6, 7; Proposition 4	Theorem 1
Lemma 12 (covering calculus)	Lemma 3	Theorem 4; Proposition 3
Lemma 14 (conditional tables)	—	Theorem 4
Theorem 4 (CVE exactness)	Lemmas 3, 12, 14; Propositions 5, 6	—
Lemma 15 (projective hulls)	Lemma 3	Theorem 6
Theorem 6 (IVBP exactness)	Lemmas 3, 15; Propositions 5, 6	—

Table 3: Notation used throughout the paper.

Symbol	Meaning
$\mathbf{X}$, n	the atoms of the LCN and their number
$\mathcal{S}$, s	the sentences; a sentence $\ell_s \le P(\varphi_s \mid \psi_s) \le u_s$ ($\psi_s = \top$ for Type-1)
$\mathrm{atoms}(s)$, τ_s	$\mathrm{atoms}(\varphi_s) \cup \mathrm{atoms}(\psi_s)$; the file-format flag declaring the atoms of φ_s mutually dependent
c_s^-, c_s^+	cleared rows $P(\varphi_s \wedge \psi_s) - \ell_s P(\psi_s) \ge 0$, $u_s P(\psi_s) - P(\varphi_s \wedge \psi_s) \ge 0$
g_s	the ratio $P(\varphi_s \wedge \psi_s)/P(\psi_s)$, only where $P(\psi_s) > 0$ (realizability only)
G_P; pa_P, de_P	primal graph; its atom parents and descendants (used only by LMC(LCN))
$(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$	conditional independence between disjoint atom sets (Definition 6)
$(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$	an LMC assertion: X a single atom, $\mathbf{Y}$, $\mathbf{Z}$ atom sets
$\Delta(2^{\mathbf{X}})$, p	simplex of joint distributions; a member
$\mathcal{D}_{\mathrm{LCN}}$	the LCN distribution set equation 9
a; $\mathbf{e}$, $\mathbf{E}$	query atom; evidence, an assignment to the atom set $\mathbf{E}$
$\underline{P}(a \mid \mathbf{e})$, $\overline{P}(a \mid \mathbf{e})$	infimum and supremum over the members with $p(\mathbf{e}) > 0$ (Definition 7)
G; $\mathcal{T}_1, \ldots, \mathcal{T}_m$	the structure, a chain graph; its chain components
pa_G, ne_G, bd_G, de_G	parents, neighbours, boundary and descendants in G
D, $\mathbf{V}$, m	the component DAG, its node set, $m = \lvert\mathbf{V}\rvert$
$\mathrm{Pa}(v)$	parent *nodes* of v in D
ch_v; pa_v; π	atoms of component v; atoms of its parent components, $\bigcup_{u \in \mathrm{Pa}(v)} \mathrm{ch}_u$; an assignment of pa_v
θ_v, $\theta_{v,\pi}$	family table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$; its row at π, a distribution over the states of ch_v
$K_{v,\pi}$, K_v	local credal set of row (v, π) (Definition 11); $K_v = \prod_\pi K_{v,\pi}$
$\mathcal{P}_{\mathrm{SE}}$, $\mathcal{D}_{\mathrm{SE}}$	free products of rows from the local sets equation 11; the strong extension $\mathrm{CH}(\mathcal{P}_{\mathrm{SE}})$
ext, CH	extreme points; convex hull
T, G^*	a junction tree; the chordal graph it defines
$C \in \mathcal{C}$, S, β	a cluster of T; a separator; the cluster-atom budget
q_C, A_φ	cluster marginal; $0/1$ indicator vector of formula φ over a cluster's states
ρ_a, ρ_a^{cc}	ratio read-off of $P(a \mid \mathbf{e})$; its linear Charnes–Cooper objective
$\mathcal{M}$	a set of distributions in a lemma (compact or finite where stated)

A.3 CONDITIONAL INDEPENDENCE, THE CREDAL SET $\mathcal{D}_{\mathrm{LCN}}$, AND QUERIES

Definition 6 (Conditional independence)**.** *Let $p \in \Delta(2^{\mathbf{X}})$ and let $\mathbf{A}, \mathbf{B}, \mathbf{Z} \subseteq \mathbf{X}$ be pairwise disjoint, $\mathbf{Z}$ possibly empty. Then* $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$ holds in p *if*

$$p(\mathbf{a}, \mathbf{b}, \mathbf{z})\, p(\mathbf{z}) = p(\mathbf{a}, \mathbf{z})\, p(\mathbf{b}, \mathbf{z}) \qquad \text{for all configurations } (\mathbf{a}, \mathbf{b}, \mathbf{z}) \text{ of } \mathbf{A} \cup \mathbf{B} \cup \mathbf{Z}, \tag{7}$$

with $p(\mathbf{z}) = 1$ when $\mathbf{Z} = \varnothing$. A configuration $\mathbf{z}$ with $p(\mathbf{z}) = 0$ satisfies equation 7 vacuously, since then $p(\mathbf{a}, \mathbf{b}, \mathbf{z})$, $p(\mathbf{a}, \mathbf{z})$ and $p(\mathbf{b}, \mathbf{z})$ are all 0; for $p(\mathbf{z}) > 0$, equation 7 is the usual $p(\mathbf{a}, \mathbf{b} \mid \mathbf{z}) = p(\mathbf{a} \mid \mathbf{z})\, p(\mathbf{b} \mid \mathbf{z})$. An LMC assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ is the case $\mathbf{A} = \{X\}$, $\mathbf{B} = \mathbf{Y}$.

No positivity is assumed anywhere in this paper; the following properties hold for every p.

Lemma 1 (Semi-graphoid properties and F⇒G)**.** *Let $p \in \Delta(2^{\mathbf{X}})$ and let $\mathbf{A}, \mathbf{B}, \mathbf{W}, \mathbf{Z}$ be pairwise disjoint atom sets.*

(i) Symmetry: $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z}) \Rightarrow (\mathbf{B} \perp\!\!\!\perp \mathbf{A} \mid \mathbf{Z})$.

(ii) Decomposition: $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \cup \mathbf{W} \mid \mathbf{Z}) \Rightarrow (\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$.

(iii) Weak union: $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \cup \mathbf{W} \mid \mathbf{Z}) \Rightarrow (\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z} \cup \mathbf{W})$.

(iv) Contraction: $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$ *and* $(\mathbf{A} \perp\!\!\!\perp \mathbf{W} \mid \mathbf{Z} \cup \mathbf{B}) \Rightarrow (\mathbf{A} \perp\!\!\!\perp \mathbf{B} \cup \mathbf{W} \mid \mathbf{Z})$.

(v) F⇒G: *if $p(\mathbf{x}) = \prod_C \phi_C(\mathbf{x}_C)$ for non-negative functions ϕ_C, each on a clique C of an undirected graph H over $\mathbf{X}$, and $\mathbf{Z}$ separates $\mathbf{A}$ from $\mathbf{B}$ in H, then $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$ holds in p (Lauritzen, 1996, Prop. 3.8).*

(vi) Redundant rows: *if $\mathbf{A} = \{X\}$ is a single binary atom, the equalities equation 7 with $x = 1$ imply those with $x = 0$; and for each $(x, \mathbf{z})$ the equalities for all but one configuration $\mathbf{b}$ imply the remaining one. Hence $2^{|\mathbf{Z}|}(2^{|\mathbf{B}|} - 1)$ equalities suffice.*

The intersection property, $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z} \cup \mathbf{W}) \wedge (\mathbf{A} \perp\!\!\!\perp \mathbf{W} \mid \mathbf{Z} \cup \mathbf{B}) \Rightarrow (\mathbf{A} \perp\!\!\!\perp \mathbf{B} \cup \mathbf{W} \mid \mathbf{Z})$, is not *among them: it requires positivity (Cowell et al., 1999).*

Proof. Write $\mathbf{z}$, $\mathbf{w}$, $\mathbf{b}$, $\mathbf{a}$ for configurations of the corresponding sets.

(i) equation 7 is symmetric in $(\mathbf{a}, \mathbf{A})$ and $(\mathbf{b}, \mathbf{B})$.

(ii) Sum the hypothesis $p(\mathbf{a}, \mathbf{b}, \mathbf{w}, \mathbf{z})p(\mathbf{z}) = p(\mathbf{a}, \mathbf{z})p(\mathbf{b}, \mathbf{w}, \mathbf{z})$ over $\mathbf{w}$.

(iii) We need $p(\mathbf{a}, \mathbf{b}, \mathbf{w}, \mathbf{z})\, p(\mathbf{w}, \mathbf{z}) = p(\mathbf{a}, \mathbf{w}, \mathbf{z})\, p(\mathbf{b}, \mathbf{w}, \mathbf{z})$. If $p(\mathbf{w}, \mathbf{z}) = 0$ both sides are 0. Otherwise $p(\mathbf{z}) \geq p(\mathbf{w}, \mathbf{z}) > 0$, and the hypothesis gives $p(\mathbf{a}, \mathbf{b}, \mathbf{w}, \mathbf{z}) = p(\mathbf{a}, \mathbf{z})p(\mathbf{b}, \mathbf{w}, \mathbf{z})/p(\mathbf{z})$, while its decomposition (ii) onto $\mathbf{W}$ gives $p(\mathbf{a}, \mathbf{w}, \mathbf{z}) = p(\mathbf{a}, \mathbf{z})p(\mathbf{w}, \mathbf{z})/p(\mathbf{z})$. Multiplying the first by $p(\mathbf{w}, \mathbf{z})$ and substituting the second gives the claim.

(iv) We need $p(\mathbf{a}, \mathbf{b}, \mathbf{w}, \mathbf{z})\, p(\mathbf{z}) = p(\mathbf{a}, \mathbf{z})\, p(\mathbf{b}, \mathbf{w}, \mathbf{z})$. If $p(\mathbf{b}, \mathbf{z}) = 0$ both sides are 0. Otherwise $p(\mathbf{z}) > 0$; the second hypothesis gives $p(\mathbf{a}, \mathbf{b}, \mathbf{w}, \mathbf{z}) = p(\mathbf{a}, \mathbf{b}, \mathbf{z})\, p(\mathbf{b}, \mathbf{w}, \mathbf{z})/p(\mathbf{b}, \mathbf{z})$ and the first $p(\mathbf{a}, \mathbf{b}, \mathbf{z}) = p(\mathbf{a}, \mathbf{z})\, p(\mathbf{b}, \mathbf{z})/p(\mathbf{z})$; substituting the second into the first gives the claim.

(v) Let $\mathbf{R}$ be the set of atoms connected to $\mathbf{A}$ in H after removing $\mathbf{Z}$, and $\mathbf{R}' = \mathbf{X} \setminus (\mathbf{R} \cup \mathbf{Z})$, so $\mathbf{A} \subseteq \mathbf{R}$ and, by separation, $\mathbf{B} \subseteq \mathbf{R}'$. A clique meeting both $\mathbf{R}$ and $\mathbf{R}'$ would contain an edge between them, which does not exist; so each C lies in $\mathbf{R} \cup \mathbf{Z}$ or in $\mathbf{R}' \cup \mathbf{Z}$, and $p(\mathbf{x}) = f(\mathbf{r}, \mathbf{z})\, g(\mathbf{r}', \mathbf{z})$ with $f, g \geq 0$. Put $\bar{f}(\mathbf{z}) = \sum_{\mathbf{r}} f(\mathbf{r}, \mathbf{z})$ and $\bar{g}(\mathbf{z}) = \sum_{\mathbf{r}'} g(\mathbf{r}', \mathbf{z})$. Then $p(\mathbf{r}, \mathbf{z}) = f\, \bar{g}$, $p(\mathbf{r}', \mathbf{z}) = \bar{f}\, g$ and $p(\mathbf{z}) = \bar{f}\, \bar{g}$, so $p(\mathbf{r}, \mathbf{r}', \mathbf{z})\, p(\mathbf{z}) = f g\, \bar{f}\bar{g} = p(\mathbf{r}, \mathbf{z})\, p(\mathbf{r}', \mathbf{z})$: $(\mathbf{R} \perp\!\!\!\perp \mathbf{R}' \mid \mathbf{Z})$ holds. Decomposition (ii) and symmetry (i) reduce it to $(\mathbf{A} \perp\!\!\!\perp \mathbf{B} \mid \mathbf{Z})$.

(vi) For $x = 0$: $p(0, \mathbf{b}, \mathbf{z})p(\mathbf{z}) = \big(p(\mathbf{b}, \mathbf{z}) - p(1, \mathbf{b}, \mathbf{z})\big)p(\mathbf{z})$, which by the $x = 1$ equality equals $p(\mathbf{b}, \mathbf{z})p(\mathbf{z}) - p(1, \mathbf{z})p(\mathbf{b}, \mathbf{z}) = p(0, \mathbf{z})\, p(\mathbf{b}, \mathbf{z})$. For fixed $(x, \mathbf{z})$, both sides of equation 7 summed over $\mathbf{b}$ equal $p(x, \mathbf{z})p(\mathbf{z})$, so the sum of all $\mathbf{b}$-equalities is an identity and any one of them follows from the others. □

Sentences. A Type-1 sentence $\ell \le P(\varphi) \le u$ contributes the two linear rows $\ell \le \sum_{\omega \models \varphi} p(\omega) \le u$. A Type-2 sentence $\ell \le P(\varphi \mid \psi) \le u$ is read in *cleared* form,

$$c_s^-(p) = P(\varphi \wedge \psi) - \ell\, P(\psi) \;\ge\; 0, \qquad c_s^+(p) = u\, P(\psi) - P(\varphi \wedge \psi) \;\ge\; 0, \tag{8}$$

which avoids a division and is the form the implementation imposes; a Type-1 sentence is the case $\psi = \top$, $P(\psi) = 1$. Both rows are linear in p. Where $P(\psi) > 0$ we write $g_s(p) = P(\varphi \wedge \psi)/P(\psi)$ for the sentence's *ratio functional* ($g_s(p) = P(\varphi)$ for Type-1); there, $c_s^{\pm}(p) \ge 0$ iff $\ell \le g_s(p) \le u$. The ratio g_s is used only in the definition of family-realizability (Definition 10).

Remark 1 (The cleared reading is the one in force)**.** *The cleared rows equation 8 are vacuously satisfied on $\{p : P(\psi) = 0\}$, so they admit joints that a ratio reading requiring $P(\psi) > 0$ excludes. The two readings therefore define different credal sets. Every statement in this paper is made for the cleared reading, matching our implementation.*

The flag τ_s and the primal graph. Each sentence carries a flag τ_s, written as a trailing `; True` or `; False` in the `.lcn` format (absent means False). It only affects the graphs (Section 2.2): with $\tau_s = \mathrm{True}$ the edges between the formula node φ_s and its atoms are bi-directed in G_P, and the atoms of φ_s are pairwise joined by undirected structure edges; with $\tau_s = \mathrm{False}$ the edges point from φ_s to its atoms only, and the atoms of φ_s are not joined. When φ_s has at most one atom the flag changes neither pa_P, de_P nor the structure.

The set $\mathcal{D}_{\mathrm{LCN}}$. The Local Markov Condition (Definition 1) attaches to the program a finite set Λ of assertions $\iota = (X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ with X a single atom and $\mathbf{Y}, \mathbf{Z}$ disjoint atom sets; each is imposed as the equalities equation 7 of Definition 6. Then

$$\mathcal{D}_{\mathrm{LCN}} = \big\{\, p \in \Delta(2^{\mathbf{X}}) \;:\; c_s^-(p) \ge 0,\; c_s^+(p) \ge 0 \;\forall s \in \mathcal{S}, \qquad \iota \text{ holds in } p \;\forall \iota \in \Lambda \,\big\}. \tag{9}$$

$\mathcal{D}_{\mathrm{LCN}}$ is compact: it is the intersection of the compact simplex with the closed sets $\{c_s^{\pm} \ge 0\}$ and $\{\text{equation 7}\}$, all defined by polynomials in p. It may be empty (an *inconsistent* LCN), and it need not be convex (the equalities equation 7 are bilinear).

Definition 7 (Conditional bounds)**.** *Let $\mathcal{M} \subseteq \Delta(2^{\mathbf{X}})$ be a set of distributions, a an atom, and $\mathbf{e}$ an assignment to an atom set $\mathbf{E} \not\ni a$, and put $\mathcal{M}_{\mathbf{e}} = \{p \in \mathcal{M} :\; p(\mathbf{e}) > 0\}$. If $\mathcal{M}_{\mathbf{e}} \neq \varnothing$,*

$$\underline{P}_{\mathcal{M}}(a \mid \mathbf{e}) = \inf_{p \in \mathcal{M}_{\mathbf{e}}} \frac{p(a, \mathbf{e})}{p(\mathbf{e})}, \qquad \overline{P}_{\mathcal{M}}(a \mid \mathbf{e}) = \sup_{p \in \mathcal{M}_{\mathbf{e}}} \frac{p(a, \mathbf{e})}{p(\mathbf{e})};$$

if $\mathcal{M}_{\mathbf{e}} = \varnothing$ (in particular if $\mathcal{M} = \varnothing$), the interval is the vacuous $[0, 1]$ by convention. Marginal bounds are the case $\mathbf{E} = \varnothing$ ($p(\mathbf{e}) = 1$, $\mathcal{M}_{\mathbf{e}} = \mathcal{M}$). Without a subscript, $\underline{P}$ and $\overline{P}$ refer to $\mathcal{M} = \mathcal{D}_{\mathrm{LCN}}$.

This is *regular conditioning*: members that give the evidence probability 0 are discarded, not assigned a conditional. Every engine implements it, and every engine reports the vacuous interval when no member is left. The bounds are *attained* (are a minimum and a maximum) whenever $\mathcal{M}$ is compact and $\min_{p \in \mathcal{M}} p(\mathbf{e}) > 0$, since the ratio is then continuous on the compact set $\mathcal{M}_{\mathbf{e}} = \mathcal{M}$; in particular marginal bounds over a non-empty $\mathcal{D}_{\mathrm{LCN}}$ are attained. They are also attained when $\mathcal{M}$ is the convex hull of a finite set, such as $\mathcal{D}_{\mathrm{SE}}$ (Lemma 3(c)). Otherwise an infimum need not be attained, which is why the bounds are stated as inf/sup.

The following elementary lemma is the workhorse of our theoretical contribution: it is what licenses imposing a constraint on a single cluster.

Lemma 2 (Locality)**.** *Every constraint in equation 9 depends on p only through its marginal on a fixed atom set, its* scope*: the rows $c_s^{\pm}$ of a sentence s through $p \restriction \mathrm{atoms}(s)$, and an LMC assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ through $p \restriction (\{X\} \cup \mathbf{Y} \cup \mathbf{Z})$; likewise the ratio $p(a, \mathbf{e})/p(\mathbf{e})$ depends on p only through $p \restriction (\{a\} \cup \mathbf{E})$. Consequently, if p and p' satisfy $p \restriction \mathbf{A} = p' \restriction \mathbf{A}$ for the scope $\mathbf{A}$ of a constraint c, then c holds for p iff it holds for p'.*

Proof. Let c be a constraint with scope $\mathbf{A}$.

1. *Probability of a formula.* Let φ be a formula with $\mathrm{atoms}(\varphi) \subseteq \mathbf{A}$. The truth value of φ at an assignment ω of $\mathbf{X}$ is determined by $\omega \restriction \mathbf{A}$; grouping the sum $P(\varphi) = \sum_{\omega \models \varphi} p(\omega)$ by

the value of $\omega \restriction \mathbf{A}$ gives $P(\varphi) = \sum_{\alpha \models \varphi} (p \restriction \mathbf{A})(\alpha)$, where α ranges over assignments of $\mathbf{A}$. This is a function of $p \restriction \mathbf{A}$ alone.

2. *Sentences.* $c_s^{\pm}$ are linear combinations of $P(\varphi_s \wedge \psi_s)$ and $P(\psi_s)$, whose atoms lie in $\mathrm{atoms}(s)$; by step 1 they are functions of $p \restriction \mathrm{atoms}(s)$.

3. *LMC assertions.* Each factor in equation 7 is the probability of a conjunction of literals over $\{X\} \cup \mathbf{Y} \cup \mathbf{Z}$, hence by step 1 a function of $p \restriction (\{X\} \cup \mathbf{Y} \cup \mathbf{Z})$; so is each product of two such factors.

4. *Query ratio.* $p(a, \mathbf{e})$ and $p(\mathbf{e})$ are probabilities of conjunctions of literals over $\{a\} \cup \mathbf{E}$; step 1 applies.

In each case the constraint's truth value is determined by $p \restriction \mathbf{A}$, which is precisely the stated consequence. □

A.4 Chain graph, strong extension, and the two gaps

The LCN pipeline first checks that the structure is a chain graph (`LCN.is_chain_graph`): it rejects a directed edge between two atoms of one undirected component that are not joined by an undirected edge (`LCN.intra_component_directed_edges`; a directed edge parallel to an undirected one is absorbed by it), and a directed cycle among the contracted components (in particular a self-loop, which a Type-2 sentence whose φ and ψ share an atom would create). It then *contracts* each connected component of the structure's undirected subgraph into a single *compound* node (`LCN.simplify_structure_graph`; rendered with a `-`-joined name, e.g. `C-D`), which yields the component DAG D. A distribution p *factorizes along* D if there are family tables θ_v, with each row $\theta_{v,\pi}$ a distribution over the states of ch_v, such that

$$p(\mathbf{x}) = \prod_{v \in \mathbf{V}} \theta_v(\mathbf{x}_{\mathrm{ch}_v} \mid \mathbf{x}_{\mathrm{pa}_v}) \qquad \text{for all } \mathbf{x}, \tag{10}$$

one factor per node v. The *family scope* at v is $\mathrm{ch}_v \cup \mathrm{pa}_v$; root families have $\mathrm{pa}_v = \varnothing$. A chain-graph LCN is *factorizing* if every $p \in \mathcal{D}_{\mathrm{LCN}}$ factorizes along D, and *product-Markov* if every product equation 10 of family tables satisfies every LMC assertion.

The credal-network based methods replace each table by local credal sets: the compiler assigns each row (v, π) the set $K_{v,\pi}$ of Definition 11, a non-empty polytope of distributions over the states of ch_v. It is an interval for a single-atom child and the exact row polytope for a compound child; where the polytope oracle fails, the per-state interval box is kept as a sound fallback and the row is reported as inexact; and an infeasible local program gives the whole simplex by convention. They then work with the *product set*

$$\mathcal{P}_{\mathrm{SE}} = \left\{ \textstyle\prod_{v \in \mathbf{V}} \theta_v \ : \ \theta_{v,\pi} \in K_{v,\pi} \text{ chosen independently for each } v \text{ and each } \pi \right\} \tag{11}$$

and its convex hull, the *strong extension* $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{SE}})$. Mixtures of products need not factorize, so $\mathcal{D}_{\mathrm{SE}}$ may contain distributions violating the independences of the chain graph; but the two sets have the same bounds.

Lemma 3 (Hull invariance of bounds)**.** *Let Ω be a finite set of states.*

(a) Compactness and vertices. *If every $K_{v,\pi}$ is a non-empty polytope, then $\mathcal{P}_{\mathrm{SE}}$ is compact, and $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{V})$ for the finite set $\mathcal{V} = \{\prod_v \theta_v : \theta_{v,\pi} \in \mathrm{ext}(K_{v,\pi}) \ \forall v, \pi\} \subseteq \mathcal{P}_{\mathrm{SE}}$ of vertex products. Hence $\mathcal{D}_{\mathrm{SE}}$ is a polytope and $\mathrm{ext}(\mathcal{D}_{\mathrm{SE}}) \subseteq \mathcal{V}$.*

(b) Conditional bounds. *Let $\mathcal{M} \subseteq \Delta(\Omega)$ be any set, $b \in \mathbb{R}^\Omega$ with $b \geq 0$, and $h \in \mathbb{R}^\Omega$ with $h_\omega = 0$ whenever $b_\omega = 0$; let $f(p) = \langle h, p \rangle / \langle b, p \rangle$ on $\{p : \langle b, p \rangle > 0\}$, and write $\mathcal{M}^+ = \{p \in \mathcal{M} : \langle b, p \rangle > 0\}$. Then $\mathrm{CH}(\mathcal{M})^+ \neq \varnothing$ iff $\mathcal{M}^+ \neq \varnothing$, and in that case*

$$\inf_{\mathrm{CH}(\mathcal{M})^+} f = \inf_{\mathcal{M}^+} f, \qquad \sup_{\mathrm{CH}(\mathcal{M})^+} f = \sup_{\mathcal{M}^+} f.$$

(c) Attainment. *If moreover $\mathcal{M}$ is finite, both are attained at members of $\mathcal{M}^+$.*

In particular, for a query $P(a \mid \mathbf{e})$, take $h = \mathbf{1}[a \wedge \mathbf{e}]$ and $b = \mathbf{1}[\mathbf{e}]$ (for a marginal, $b = \mathbf{1}$). By (b), $\mathcal{P}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{SE}}$ have the same bounds in the sense of Definition 7, including the vacuous case; by (a) and (b) these equal the bounds over the finite set $\mathcal{V}$, and by (c) they are attained at a vertex product. No hypothesis $\underline{P}(\mathbf{e}) > 0$ is needed, only $\overline{P}(\mathbf{e}) > 0$ for a non-vacuous answer.

Proof. (a) Index the rows by $r = (v, \pi)$; there are finitely many.

1. Let $\Phi : \prod_r K_r \to \mathbb{R}^{2^n}$ map a choice of rows θ to $\Phi(\theta)(\mathbf{x}) = \prod_v \theta_{v,\mathbf{x}_{\mathrm{pa}_v}}(\mathbf{x}_{\mathrm{ch}_v})$. $\Phi(\theta)$ is a distribution: summing out the nodes of D in reverse topological order, each node's factor is the only one containing its atoms (its children's factors having been summed out already) and sums to 1 as a row. Φ is a polynomial, hence continuous, and its domain is a finite product of compact sets, hence compact. So $\mathcal{P}_{\mathrm{SE}} = \Phi(\prod_r K_r)$ is compact.
2. For fixed $\mathbf{x}$, the product $\Phi(\theta)(\mathbf{x})$ contains, for each node v, an entry of the single row $(v, \mathbf{x}_{\mathrm{pa}_v})$ only. So each coordinate of $\Phi(\theta)$ is affine (in fact linear or constant) in each row θ_r separately, the other rows being fixed.
3. A polytope is the convex hull of its finitely many vertices, so $\theta_r = \sum_j \mu_j w_j$ with $w_j \in \mathrm{ext}(K_r)$, $\mu_j \geq 0$, $\sum_j \mu_j = 1$. By step 2, $\Phi(\theta) = \sum_j \mu_j \Phi(\theta[r \mapsto w_j])$, where $\theta[r \mapsto w_j]$ replaces row r by w_j.
4. Applying step 3 to the rows one at a time, $\Phi(\theta)$ is a convex combination of points of $\mathcal{V}$. Hence $\mathcal{P}_{\mathrm{SE}} \subseteq \mathrm{CH}(\mathcal{V})$, and with $\mathcal{V} \subseteq \mathcal{P}_{\mathrm{SE}}$ we get $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{SE}}) = \mathrm{CH}(\mathcal{V})$, the convex hull of a finite set, i.e. a polytope. Every extreme point of the convex hull of a finite set belongs to the set, so $\mathrm{ext}(\mathcal{D}_{\mathrm{SE}}) \subseteq \mathcal{V}$.

(b)

1. Since $\mathcal{M}^+ \subseteq \mathrm{CH}(\mathcal{M})^+$, the infimum over $\mathrm{CH}(\mathcal{M})^+$ is at most, and the supremum at least, the one over $\mathcal{M}^+$; and $\mathcal{M}^+ \neq \varnothing$ implies $\mathrm{CH}(\mathcal{M})^+ \neq \varnothing$.
2. Let $p \in \mathrm{CH}(\mathcal{M})^+$, $p = \sum_{k=1}^{K} \lambda_k p_k$ with $p_k \in \mathcal{M}$, $\lambda_k > 0$, $\sum_k \lambda_k = 1$. Then $\langle b, p\rangle = \sum_k \lambda_k \langle b, p_k\rangle$ with every term ≥ 0 (as $b \geq 0$, $p_k \geq 0$); since $\langle b, p\rangle > 0$, the index set $K^+ = \{k : \langle b, p_k\rangle > 0\}$ is non-empty, and $p_k \in \mathcal{M}^+$ for $k \in K^+$. This proves the converse of step 1's last claim.
3. For $k \notin K^+$, $\langle b, p_k\rangle = 0$ with non-negative terms, so $p_k(\omega) = 0$ wherever $b_\omega > 0$; since $h_\omega = 0$ wherever $b_\omega = 0$, $\langle h, p_k\rangle = 0$.
4. Hence $f(p) = \sum_{k\in K^+} \lambda_k \langle h, p_k\rangle \big/ \sum_{k\in K^+} \lambda_k \langle b, p_k\rangle = \sum_{k\in K^+} \mu_k f(p_k)$ with $\mu_k = \lambda_k \langle b, p_k\rangle / \langle b, p\rangle > 0$ and $\sum_{k\in K^+} \mu_k = 1$.
5. So $\inf_{\mathcal{M}^+} f \leq \min_{k\in K^+} f(p_k) \leq f(p) \leq \max_{k\in K^+} f(p_k) \leq \sup_{\mathcal{M}^+} f$. Taking the infimum and supremum over $p \in \mathrm{CH}(\mathcal{M})^+$ and combining with step 1 gives the equalities.

(c) If $\mathcal{M}$ is finite, so is $\mathcal{M}^+ \neq \varnothing$, and $\inf_{\mathcal{M}^+} f$ and $\sup_{\mathcal{M}^+} f$ are a minimum and a maximum, attained at members of $\mathcal{M}^+ \subseteq \mathrm{CH}(\mathcal{M})^+$; by (b) these are the infimum and supremum over $\mathrm{CH}(\mathcal{M})^+$.

The query. With $h = \mathbf{1}[a \wedge \mathbf{e}]$, $b = \mathbf{1}[\mathbf{e}]$ we have $b \geq 0$ and $h_\omega = 1$ only where $b_\omega = 1$, so (b) applies; $\langle b, p\rangle = p(\mathbf{e})$ and $f(p) = p(a \mid \mathbf{e})$. Applied to $\mathcal{M} = \mathcal{P}_{\mathrm{SE}}$ it gives the equality of the bounds of $\mathcal{P}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{SE}}$, and of the emptiness of $\mathcal{M}_{\mathbf{e}}$ (the vacuous case of Definition 7). Applied to $\mathcal{M} = \mathcal{V}$, with $\mathrm{CH}(\mathcal{V}) = \mathcal{D}_{\mathrm{SE}}$ by (a), it gives the equality with the bounds over $\mathcal{V}$, attained by (c). □

Lemma 4 (DAG-structured LCNs)**.** *Let $\mathcal{L}$ be an LCN whose structure is a DAG D (no undirected edge; every chain component a single atom, so $\mathbf{V} = \mathbf{X}$, $\mathrm{ch}_X = \{X\}$ and $\mathrm{pa}_X = \mathrm{pa}_D(X)$). Write $\mathrm{de}_D(X)$ for the descendants of X in D (X excluded) and $\mathrm{nd}_D(X) = \mathbf{X} \setminus (\{X\} \cup \mathrm{de}_D(X))$. Then:*

(i) $\mathrm{pa}_P(X) = \mathrm{pa}_D(X)$ *and* $\mathrm{de}_P(X) = \mathrm{de}_D(X)$ *for every atom X. Hence the LMC(LCN) assertions are $(X \perp\!\!\!\perp \mathrm{nd}_D(X) \setminus \mathrm{pa}_D(X) \mid \mathrm{pa}_D(X))$, the* directed local Markov property *(DL) of D (Lauritzen, 1996).*

(ii) Every $p \in \Delta(2^{\mathbf{X}})$ satisfying (DL) factorizes along D, with $\theta_{X,\pi} = p(X \mid \pi)$ wherever $p(\pi) > 0$. In particular $\mathcal{L}$ is factorizing.

(iii) Every product $q = \prod_X \theta_X(x \mid \mathbf{x}_{\mathrm{pa}_D(X)})$ of family tables satisfies (DL), hence every LMC assertion: $\mathcal{L}$ is product-Markov.

No positivity is assumed. (Parts (ii) and (iii) are the classical equivalence of the directed factorization and local Markov properties (Lauritzen, 1996, Thm. 3.27); we include proofs to make the absence of positivity explicit.)

Proof. (i) We use the constructions of Section 2.2.

1. *Consequences of DAG structure.* (α) No sentence with $\tau_s = \mathrm{True}$ has two or more atoms in φ_s: structure step (i) would create an undirected edge, which no later step removes. (β) Two atoms are never joined by directed edges in both directions, since step (iii) would make them undirected. Hence (γ) $Y \to X$ is an edge of D iff some Type-2 sentence has $Y \in \mathrm{atoms}(\psi_s)$ and $X \in \mathrm{atoms}(\varphi_s)$, and then $Y \neq X$ (a self-loop is a directed cycle).

2. *Edges of G_P at formula nodes.* By construction, the only edges between two formula nodes are $\psi_s \to \varphi_s$ for a Type-2 sentence with both formulas non-atomic, and a ψ_s formula node has in-edges only from $\mathrm{atoms}(\psi_s)$. The in-edges of a formula node φ_s come from ψ_s (Type-2 only) and, if $\tau_s = \mathrm{True}$, from the atoms of φ_s — by (α), from the single atom of φ_s. An atom-to-atom edge $Y \to X$ exists only for a Type-2 sentence with $\psi_s = Y$ and $\varphi_s = X$ both atomic.

3. *Hops.* Call a directed path of G_P from an atom U to an atom W whose intermediate nodes are all formula nodes a *hop*. We claim: a hop from U to $W \neq U$ exists iff $U \to W$ in D. Consider the last node f before W. If $f = U$ is an atom, step 2 gives a Type-2 sentence with $\psi_s = U$, $\varphi_s = W$, so $U \to W$ by (γ). If f is the formula node φ_s of a Type-1 sentence, its only possible predecessor is the single atom of φ_s, which is W, so $U = W$. If $f = \varphi_s$ of a Type-2 sentence, its predecessors are W itself (if $\tau_s = \mathrm{True}$) or ψ_s, which is either the atom $U \in \mathrm{atoms}(\psi_s)$ or a formula node whose predecessors are atoms $U \in \mathrm{atoms}(\psi_s)$; so $U = W$ or $U \in \mathrm{atoms}(\psi_s)$, $W \in \mathrm{atoms}(\varphi_s)$, i.e. $U \to W$ by (γ). If $f = \psi_s$ with $\varphi_s = W$ atomic, its predecessors are the atoms $U \in \mathrm{atoms}(\psi_s)$, and $U \to W$. Conversely, if $U \to W$ in D, (γ) gives a Type-2 sentence with $U \in \mathrm{atoms}(\psi_s)$, $W \in \mathrm{atoms}(\varphi_s)$, and U, ψ_s, φ_s, W (with an atomic formula identified with its atom) is a hop.

4. *Parents.* By definition $\mathrm{pa}_P(W)$ is the set of atoms $U \neq W$ with a hop to W; by step 3 this is $\mathrm{pa}_D(W)$.

5. *Descendants, $\subseteq$.* A directed path of G_P from X to an atom $Y \neq X$ splits at its atom nodes $X = U_0, U_1, \ldots, U_k = Y$ into hops. Deleting the hops with $U_{j+1} = U_j$ leaves a non-empty sequence of hops between distinct atoms, each an edge of D by step 3, i.e. a directed walk from X to Y in D. So $\mathrm{de}_P(X) \subseteq \mathrm{de}_D(X)$.

6. *Descendants, $\supseteq$.* Let $X = U_0 \to U_1 \to \cdots \to U_k = Y$ be a directed path of D. Replacing each edge by a hop (step 3) gives a directed walk of G_P from X to Y, which contains a directed path whose intermediate nodes are among the walk's. These are formula nodes, which are not atoms, and atoms U_j ($0 < j < k$), which are descendants of X in D and hence not parents of X (a parent that is a descendant closes a directed cycle). By step 4 none is in $\mathrm{pa}_P(X)$, so $Y \in \mathrm{de}_P(X)$.

7. By steps 4–6, $\mathbf{N}_X = \mathbf{X} \setminus (\{X\} \cup \mathrm{pa}_D(X) \cup \mathrm{de}_D(X)) = \mathrm{nd}_D(X) \setminus \mathrm{pa}_D(X)$ in Definition 1.

(ii) Let $X_1, \ldots, X_n$ be a topological order of D, $\mathbf{P}_i = \{X_1, \ldots, X_{i-1}\}$ and $\Pi_i = \mathrm{pa}_D(X_i)$.

1. $\mathbf{P}_i \subseteq \mathrm{nd}_D(X_i)$ (a descendant comes later) and $\Pi_i \subseteq \mathbf{P}_i$. With $\mathbf{W}_i = \mathbf{P}_i \setminus \Pi_i \subseteq \mathrm{nd}_D(X_i) \setminus \Pi_i$, (DL) and decomposition (Lemma 1(ii)) give $(X_i \perp\!\!\!\perp \mathbf{W}_i \mid \Pi_i)$ (trivial if $\mathbf{W}_i = \varnothing$).

2. Define $\theta_{X_i,\pi}(x) = p(x, \pi)/p(\pi)$ if $p(\pi) > 0$, and any distribution (e.g. uniform) otherwise.

3. We show $p(x_1, \ldots, x_i) = \prod_{j \le i} \theta_{X_j}(x_j \mid \mathbf{x}_{\Pi_j})$ by induction on i; $i = 0$ is $1 = 1$. Let $\pi = \mathbf{x}_{\Pi_i}$ and $\mathbf{w} = \mathbf{x}_{\mathbf{W}_i}$. If $p(x_1, \ldots, x_{i-1}) = 0$, the left side is 0 (it is at most that probability) and so is the right side, by the induction hypothesis. Otherwise $p(\pi) \ge p(x_1, \ldots, x_{i-1}) > 0$, and the equality equation 7 of $(X_i \perp\!\!\!\perp \mathbf{W}_i \mid \Pi_i)$ gives $p(x_i, \mathbf{w}, \pi) = \theta_{X_i}(x_i \mid \pi)\, p(\mathbf{w}, \pi)$, i.e. $p(x_1, \ldots, x_i) = \theta_{X_i}(x_i \mid \pi)\, p(x_1, \ldots, x_{i-1})$, and the induction hypothesis concludes.

4. For $i = n$ this is equation 10. Every $p \in \mathcal{D}_{\mathrm{LCN}}$ satisfies the LMC, which is (DL) by (i), so $\mathcal{L}$ is factorizing.

(iii) Fix an atom X with $\Pi = \mathrm{pa}_D(X)$, $\mathbf{N} = \mathrm{nd}_D(X) \setminus \Pi$.

1. *Ancestral marginals.* Call $\mathbf{A} \subseteq \mathbf{X}$ ancestral if it contains the parents of its members. Then $q(\mathbf{x}_\mathbf{A}) = \prod_{U \in \mathbf{A}} \theta_U(x_U \mid \mathbf{x}_{\mathrm{pa}_D(U)})$. Indeed, sum out the atoms outside $\mathbf{A}$ one at a time, always the last one in the topological order among those remaining: no remaining atom is its child (its children outside $\mathbf{A}$ come later and are gone, and none is in $\mathbf{A}$, which is ancestral), so its own factor is the only one containing it, and it sums to 1. (This is the marginalization step of Lemma 9.)

2. $\mathrm{nd}_D(X)$ and $\mathrm{nd}_D(X) \cup \{X\}$ are ancestral: a parent of a non-descendant is a non-descendant, and the parents of X are non-descendants by acyclicity. By step 1, $q(x, \mathbf{x}_{\mathrm{nd}}) = \theta_X(x \mid \pi)\, q(\mathbf{x}_{\mathrm{nd}})$ with $\pi = \mathbf{x}_\Pi$, $\Pi \subseteq \mathrm{nd}_D(X)$.

3. Writing $\mathbf{x}_{\mathrm{nd}} = (\mathbf{n}, \pi)$: $q(x, \mathbf{n}, \pi) = \theta_X(x \mid \pi)\, q(\mathbf{n}, \pi)$. Summing over $\mathbf{n}$, $q(x, \pi) = \theta_X(x \mid \pi)\, q(\pi)$. Therefore $q(x, \mathbf{n}, \pi)\, q(\pi) = \theta_X(x \mid \pi)\, q(\mathbf{n}, \pi)\, q(\pi) = q(x, \pi)\, q(\mathbf{n}, \pi)$ for all $(x, \mathbf{n}, \pi)$, which is equation 7 for $(X \perp\!\!\!\perp \mathbf{N} \mid \Pi)$. By (i) these are exactly the LMC assertions. □

The implementation uses Lemma 4(iii) to short-circuit its product-Markov test on DAG-structured LCNs. For compound components the factorizing property is not tested; what holds unconditionally is the following.

Remark 2 (Compound components: the positive part)**.** *Let $\mathcal{L}$ be a chain-graph LCN and $\mathcal{D}^+_{\mathrm{LCN}} = \{p \in \mathcal{D}_{\mathrm{LCN}} : \; p(\mathbf{x}) > 0 \; \forall \mathbf{x}\}$ its strictly positive part. Then*

(a) every $p \in \mathcal{D}^+_{\mathrm{LCN}}$ factorizes along D, with $\theta_{v,\pi} = p(\mathrm{ch}_v \mid \pi)$: p satisfies LMC(LCN), which equals LMC(C) (Cozman, 2023), so by positivity it factorizes as in equation 5 (Cowell et al., 1999), and $\mathrm{bd}_G(\mathcal{T}_v) \subseteq \mathrm{pa}_v$;

(b) $\mathcal{D}^+_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}}$: every row of p has $p(\pi) > 0$, and the marginal of p on the family scope is a feasible point of the local program of Definition 11, so $\theta_{v,\pi} \in K_{v,\pi}$ (this is the argument of Proposition 5, applied to a factorizing member);

(c) $\overline{\mathcal{D}^+_{\mathrm{LCN}}} \subseteq \mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{SE}}$, since $\mathcal{P}_{\mathrm{SE}}$ is closed (Lemma 3(a)).

Consequently, if $\mathcal{D}^+_{\mathrm{LCN}} \neq \varnothing$, then for every query the bounds over $\overline{\mathcal{D}^+_{\mathrm{LCN}}}$ (Definition 7), which equal those over $\mathcal{D}^+_{\mathrm{LCN}}$ because the ratio is continuous on $\{p(\mathbf{e}) > 0\}$, lie within the bounds over $\mathcal{D}_{\mathrm{SE}}$: an engine that is exact or outer for $\mathcal{D}_{\mathrm{SE}}$ is outer for the closure of the positive part. Nothing is claimed for members of $\mathcal{D}_{\mathrm{LCN}} \setminus \overline{\mathcal{D}^+_{\mathrm{LCN}}}$, and $\mathcal{D}^+_{\mathrm{LCN}}$ may be empty (e.g. when a sentence forces a probability to 0); on a non-factorizing compound LCN the credal-network bounds need not be outer for $\mathcal{D}_{\mathrm{LCN}}$ (`fr_counter_factorizing`, Section E.4).

Every statement below about *sets* is made for $\mathcal{P}_{\mathrm{SE}}$; by Lemma 3 it transfers to the *bounds* of $\mathcal{D}_{\mathrm{SE}}$. The inclusion $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}}$ of a factorizing LCN is generally strict, for two reasons that it is worth naming since they are exactly what CJT avoids:

- **Gap A (local-set widening).** Each $K_{v,\pi}$ is computed from the sentences attached to family v alone and imposes *no* LMC equality — neither the cross-family ones nor those whose scope lies inside the family scope. It may therefore contain rows that no $p \in \mathcal{D}_{\mathrm{LCN}}$ with $p(\pi) > 0$ induces: the *true* local set $K^{\mathrm{true}}_{v,\pi} = \{p(\mathrm{ch}_v \mid \pi) : \; p \in \mathcal{D}_{\mathrm{LCN}},\; p(\pi) > 0\}$ satisfies

$K^{\text{true}}_{v,\pi} \subseteq K_{v,\pi}$, sometimes strictly. (The per-state interval box kept where the polytope oracle fails widens a compound child's set further.)

- **Gap B (strong-extension widening).** Even with exact local sets, the free product in equation 11 lets each family pick its row independently, which can violate a sentence whose scope spans several families and, inside a multi-atom chain component, the LMC assertions that a free row does not enforce.

CJT reads neither the interval bounds nor the extreme points of any $K_{v,\pi}$; it uses equation 10 only for its *scopes*. Neither gap therefore arises, and the engine's target is $\mathcal{D}_{\text{LCN}}$ itself. This is the structural reason it is the only engine studied here, besides the direct 2^n method, whose bound character is stated against $\mathcal{D}_{\text{LCN}}$ rather than $\mathcal{D}_{\text{SE}}$.

Definition 8 (Bound character)**.** *Let $\mathcal{R}$ be a reference set (here $\mathcal{D}_{\text{LCN}}$ or $\mathcal{D}_{\text{SE}}$) with true bounds $[\underline{q}^\star, \overline{q}^\star]$ for the query at hand (Definition 7). An engine's interval $[\underline{q}, \overline{q}]$ is* exact w.r.t. $\mathcal{R}$ *if $[\underline{q}, \overline{q}] = [\underline{q}^\star, \overline{q}^\star]$; a valid* outer *bound if $[\underline{q}, \overline{q}] \supseteq [\underline{q}^\star, \overline{q}^\star]$; an* inner *bound if $[\underline{q}, \overline{q}] \subseteq [\underline{q}^\star, \overline{q}^\star]$; and* unreliable *if neither containment is guaranteed. An outer bound is sound as a guarantee; an inner bound is not.*

B Detailed Description of the CJT Algorithm

CJT proceeds in four steps. It builds a cluster tree over the atoms from the family scopes of the chain graph equation 10; it assigns every sentence to a cluster containing its atoms, and discharges every LMC assertion by separation, by an equivalent reduced assertion hosted on a cluster, or by the rows of an ordered tree; it writes a single constraint system whose variables are one distribution per cluster; and it reads each query off a cluster containing it, minimizing and maximizing over that system. Algorithm 1 states the procedure; Section C writes the constraint system formally.

The only input the algorithm takes from the chain graph is the collection of family scopes $\{\text{ch}_v \cup \text{pa}_v\}_v$, together with the LCN's own sentences and assertions. It never forms the local credal sets of Definition 11, and never enumerates their extreme points. This is the structural reason its target is $\mathcal{D}_{\text{LCN}}$ rather than $\mathcal{D}_{\text{SE}}$: neither Gap A nor Gap B can arise from a construction that does not build the objects in which those gaps live.

Notation. Nodes of the component DAG D are written v, u; $\text{Pa}(v)$ is the set of parent nodes of v in D, ch_v the atoms of v and pa_v the atoms of $\text{Pa}(v)$; for a set N of nodes, $\text{atoms}(N)$ is the union of the atoms of its nodes. A cluster $C \in \mathcal{C}$ is identified with its set of atoms, $|C|$ is its number of atoms, and the separator of a tree edge (C, C') is $S = C \cap C'$ (possibly empty). The width w of T is given by

$$w + 1 = \max_{C \in \mathcal{C}} |C|, \tag{12}$$

an upper bound on the treewidth of the moralized chain graph; cluster sizes are always measured in atoms. Λ is the set of LMC assertions (Definition 1), each written $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ with X a single atom and $X, \mathbf{Y}, \mathbf{Z}$ pairwise disjoint; Λ_T is the set of independence rows imposed on T. Evidence is an assignment $\mathbf{e}$ to an atom set $\mathbf{E}$ (possibly empty). The cluster budget β bounds $|C|$ (`max_cluster_atoms`, $\beta = 16$ by default).

In Algorithm 1, BucketTree is the construction of Section B.1; OrderedRows(T_o) returns the rows equation 24 and a flag ok_o, true iff T_o is within β and the two checks of Section B.2 pass; Triage is the min-fill discharge of Section B.2 (reduce, host, widen, relax), returning the possibly widened tree, its rows, and a flag h_m, true iff no assertion was relaxed; Direct is the 2^n program of Marinescu et al. (2022) solved locally (Remark 5).

B.1 Constructing the cluster tree

The scopes to be covered are the family scopes $\{v\} \cup \text{Pa}(v)$, as node sets, augmented by one clique per *cross-family sentence* — a sentence whose atoms fit inside no single family scope — namely the set of nodes containing its atoms. Augmenting forces such a sentence's atoms to co-occur in a cluster, which is what guarantees it a host in Section B.2.

Algorithm 1 CJT: marginal bounds at treewidth cost

Require: sentences Σ, LMC assertions Λ, component DAG D with family scopes $\mathcal{F}$, evidence $\mathbf{e}$ on $\mathbf{E}$, cluster budget β
Ensure: an interval for $P(a \mid \mathbf{e})$ for every atom $a \notin \mathbf{E}$
$\mathcal{S} \leftarrow \mathcal{F} \cup \{\mathrm{atoms}(s) : s \in \Sigma \text{ cross-family}\}$
$T_o \leftarrow \textsc{BucketTree}(\mathcal{S},\ \text{reverse topological order of } D)$; $(\Lambda_o, ok_o) \leftarrow \textsc{OrderedRows}(T_o)$ ▷ Prop. 4
$T_m \leftarrow \textsc{BucketTree}(\mathcal{S}, \textsc{MinFillOrder}(\mathcal{S}))$; $(T_m, \Lambda_m, h_m) \leftarrow \textsc{Triage}(\Lambda, T_m, \beta)$ ▷ §B.2
$(T, \Lambda_T) \leftarrow (T_o, \Lambda_o)$ if ok_o, unless h_m and T_m is narrower; else (T_m, Λ_m)
check that the clusters containing each atom are connected in T ▷ [H1]
for all $s \in \Sigma$ **do** ▷ every sentence must be hosted [H2]
  host s on a cluster $C \supseteq \mathrm{atoms}(s)$, if one exists
if no candidate tree is within β **or** some $s \in \Sigma$ was not hosted **then**
  return $\textsc{Direct}(a, \mathbf{e})$ for every atom $a \notin \mathbf{E}$ ▷ 2^n, uncertified; Rem. 5
build $\mathcal{G}$: $q_C \in \Delta(2^{|C|})$, $q_C \restriction S = q_{C'} \restriction S$, equation 18–equation 19 on each host, equation 20 for Λ_T; scaled by equation 22 if $\mathbf{E} \neq \varnothing$ ▷ once [H3]
for all atoms $a \notin \mathbf{E}$ **do**
  if $\{a\} \cup \mathbf{E} \subseteq C_a$ for some cluster C_a **then** ▷ objective only [H4]
    $[\underline{P}, \overline{P}] \leftarrow \inf$ and $\sup$ of ρ_a over $\mathcal{G}$ (of ρ_a^{cc} if $\mathbf{E} \neq \varnothing$) ▷ certified or dual bound, Rem. 3
  else
    $[\underline{P}, \overline{P}] \leftarrow$ solve on a min-fill tree augmented by the nodes of $\{a\} \cup \mathbf{E}$ ▷ or DIRECT
  clamp $[\underline{P}, \overline{P}]$ to $[0, 1]$

Independence assertions are *not* used to augment wholesale: most are entailed by the tree's topology (Section B.2), and adding their full scopes as cliques would inflate the width for no gain — a Markov chain would collapse from width 2 to a single cluster containing all n atoms.

Two elimination orders are tried. The *min-fill* order is computed greedily on the interaction graph — the graph in which two nodes are adjacent when they co-occur in some scope — eliminating at each step a vertex whose elimination adds fewest fill-in edges. The *ordered* order eliminates the nodes of the component DAG D in *reverse topological* order; on the resulting tree every LMC assertion can be discharged by one hosted row per node, with no widening (Proposition 4). The choice between the two trees is described at the end of Section B.2. For either order, buckets are formed along the order, each scope placed in the bucket of the first order node it contains; a bucket's cluster is the union of its scopes and of the messages of its child buckets, and its parent is the first later order node appearing in the cluster after the bucket node is removed (the cluster minus that node is the bucket's *message*). A bucket whose message is empty has no parent, so T is a forest with one tree per connected component of the interaction graph; no two components are joined by an edge.

This is standard bucket elimination, and its output satisfies the running-intersection property (Dechter, 2003). Since this property is H1, on which Lemma 6 rests, the implementation does not rely on the construction but *checks* it on the final tree (`rip_holds`): for every atom, the clusters containing it must induce a connected subtree of T (Definition 9); otherwise the result is not reported as exact. (An earlier version excluded the query node from elimination on the augmented tree of Section B.3; the buckets whose message was that node alone became separate roots, so several components shared the query atom with no separator tying its copies together. The check catches exactly this.)

Each cluster's atom set is the union of the atoms of its nodes; a compound node contributes all atoms of its chain component at once, so a cluster may be wider in atoms than in nodes. Finally, any *leaf* whose atom set is contained in its parent's is removed, repeatedly: the parent already carries that joint, and removal is safe because a leaf has no children to reattach. It preserves the running-intersection property, since an atom of the removed leaf also lies in its parent, to which the leaf was the only connection. Only subsumed leaves are removed, so the tree is not claimed to be minimal — a non-maximal cluster may survive in the interior.

B.2 Hosting and the assertion triage

The *scope* of a constraint (Lemma 2) is $\mathrm{atoms}(s)$ for a sentence s and $X \cup \mathbf{Y} \cup \mathbf{Z}$ for an assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$. Say a constraint c is *hosted* on a cluster C when $\mathrm{scope}(c) \subseteq C$; by Lemma 2 its

value is then a function of q_C alone. Hosting each constraint on a single cluster suffices: separator consistency propagates its effect to the rest of the tree, as the reconstruction argument of Theorem 8 makes precise. The implementation uses the first hosting cluster in a fixed order.

Every sentence is hosted if it can be. A sentence with no host cannot be represented exactly on this tree, and the algorithm falls back to the direct formulation (Section B.3). A sentence without atoms (a constant) is rejected with an error: it is either vacuous or makes the LCN inconsistent, and CJT does not decide which.

Independence assertions are discharged in one of two ways, depending on the tree.

Min-fill tree: reduce, host, widen, relax. For an assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z}) \in \Lambda$ let K be the set of atoms reachable from X in $G^*(T)$ by paths avoiding $\mathbf{Z}$ (X included), and $\mathbf{Y}_{\text{in}} = \mathbf{Y} \cap K$. Symmetric duplicates ($x \perp\!\!\!\perp \{y\} \mid \mathbf{Z}$ and $y \perp\!\!\!\perp \{x\} \mid \mathbf{Z}$) are imposed once.

1. **Reduce.** Replace the assertion by $(X \perp\!\!\!\perp \mathbf{Y}_{\text{in}} \mid \mathbf{Z})$. By Lemma 7 the two are *equivalent* for every joint the cluster program can reconstruct, so nothing is lost. If $\mathbf{Y}_{\text{in}} = \varnothing$ the assertion is *separator-entailed* — $\mathbf{Z}$ separates X from $\mathbf{Y}$ in $G^*(T)$ — and nothing is imposed; this is what keeps a chain at width 2.
2. **Host.** If $X \cup \mathbf{Y}_{\text{in}} \cup \mathbf{Z} \subseteq C$ for some cluster C, impose the reduced assertion there as the rows equation 20.
3. **Widen.** Otherwise collect, for all assertions still unhostable, the sets of *nodes* containing the atoms of $X \cup \mathbf{Y}_{\text{in}} \cup \mathbf{Z}$, add them all at once as augmenting cliques, rebuild the min-fill tree, and recompute every reduction against the new $G^*(T)$ (widening changes $G^*(T)$). Rounds are repeated until every reduced assertion is hosted; a round whose rebuilt tree would exceed β is discarded and widening stops there. At most $|\Lambda| + 1$ rounds are run. Reductions, hosts, and Lemma 7 all refer to the *final* tree.
4. **Relax.** An assertion still unhostable is replaced by those of its consequences $(X \perp\!\!\!\perp y \mid \mathbf{Z})$, $y \in \mathbf{Y}_{\text{in}}$, that are hostable; the others are dropped (all of them, if none is hostable). Each imposed consequence follows from the original by decomposition, so this is sound — every member of $\mathcal{D}_{\text{LCN}}$ still satisfies every imposed row — but the second half of H3 fails and the bound is a valid outer bound that may be loose.

Every imposed row is thus a reduction or a single-atom consequence of one assertion of Λ with the same conditioning set, hence implied by it (decomposition): the first half of H3 holds by construction.

Ordered tree: one row per node. For every node v whose bucket cluster has node set $\{v\} \cup N_v$, impose the single row $(v \perp\!\!\!\perp N_v \setminus \mathrm{Pa}(v) \mid \mathrm{Pa}(v))$, i.e. at atom level equation 24, on a cluster containing its atoms (the bucket cluster of v, or the cluster that subsumed it); nothing is imposed when $N_v = \mathrm{Pa}(v)$. Its scope lies in one cluster, so it is always hosted. The tree is accepted only if two checks pass: (i) *soundness* — each row follows from a single LCN assertion with the same conditioning set by symmetry and decomposition, i.e. ch_v lies on one side and the atoms of $N_v \setminus \mathrm{Pa}(v)$ on the other (`_lmc_entails`); and (ii) *completeness* — every LCN assertion is, at node level, a d-separation of D between pairwise disjoint sets of nodes, each of X, $\mathbf{Y}$, $\mathbf{Z}$ being the union of the atoms of whole nodes. By Proposition 4, (i) gives the first half of H3 and (ii) the second. Check (ii) fails whenever an atom of a compound node carries an LMC assertion, since its siblings are among its LCN parents and hence in its conditioning set, which splits the node (as in the worked example of Section C.1); in practice the ordered tree is therefore accepted only for DAG-structured LCNs. (For completeness, a compound $\mathrm{ch}_v = (x_1, \ldots, x_k)$ would be imposed through the chain $x_i \perp\!\!\!\perp \mathbf{R} \mid \mathrm{pa}_v, x_{<i}$, $i = 1, \ldots, k$, with $\mathbf{R}$ the atoms of $N_v \setminus \mathrm{Pa}(v)$, which is equivalent to the block row by contraction one way and decomposition and weak union the other; this case does not arise in our instances.)

Choice of tree. The ordered tree is built only for the shared, query-independent tree, and is skipped when it exceeds β. If it passes (i) and (ii) it is used, unless the min-fill tree after triage has nothing relaxed and is strictly narrower; ties go to the ordered tree. Otherwise the min-fill tree is used, relaxed or not. H1 is then checked on the chosen tree. The implementation records the route taken and the numbers of separator-entailed, hosted, relaxed and dropped assertions.

Example 4 (Reduction)**.** *Consider an LCN over $x_0, \dots, x_5$ in which x_2 and x_4 are roots, x_3 has parents x_2, x_4 with $0.096543 \le P(x_3 \mid x_2 \wedge x_4) \le 0.696543$, and x_0, x_1 form a separate component. The LMC gives the assertion $(x_2 \perp\!\!\!\perp x_0, x_1, x_4)$, whose four atoms exceed the largest cluster (3 atoms). In $G^*(T)$ the atoms x_0, x_1 lie in a different connected component from x_2, so the reduction yields the equivalent $(x_2 \perp\!\!\!\perp x_4)$, which is hosted on $\{x_2, x_3, x_4\}$. With $0.673737 \le P(x_2)$ and $0.349412 \le P(x_4)$ the exact bound is $\underline{P}(x_3) = 0.673737 \cdot 0.349412 \cdot 0.096543 = 0.022727$; without the assertion the program could correlate x_2 and x_4 negatively and would return only $0.096543\,(0.673737 + 0.349412 - 1) = 0.0022$.*

B.3 Solving

The constraint system of Step 3 mentions no query, and every atom lies in some cluster because each family scope is covered by one. For marginal queries one system therefore serves all atoms: per atom, only the objective changes, and two optimizations are solved. Under evidence the system is built once in the scaled form equation 22, with its normalization row on the first cluster $C_{\mathbf{e}} \supseteq \mathbf{E}$ (if there is none, every atom violates H4); it serves every atom a for which some cluster $C_a \supseteq \{a\} \cup \mathbf{E}$ exists (H4), the objective being read off C_a. When $C_a \neq C_{\mathbf{e}}$ both clusters give the same $P(\mathbf{e})$, because under H1 both are marginals of the joint reconstructed by Lemma 6 (Step 4 of the proof of Theorem 8). Atoms violating H4 are handled below. This is the sense in which the construction is *query-independent*.

Global solution and certification. The system is nonconvex: the assertion equalities equation 20 are bilinear, while the conditional read-off is linear after the scaling equation 22. Exactness in Theorem 8 refers to the global optimum, so each endpoint is computed by a spatial branch-and-bound solver (SCIP) and recorded with its certificate.

Remark 3 (Uncertified endpoints are reported by their dual bound)**.** *When branch-and-bound stops before closing the gap (e.g. at its time limit), the incumbent of a minimization lies* above *the true minimum, so reporting it would make the interval an unsound inner bound.* CJT *reports the solver's* dual *bound instead, which is $\le$ the true minimum (resp. $\ge$ the true maximum). An endpoint is* certified *when (a) SCIP terminates with status optimal under a relative gap tolerance of 0, in which case the incumbent is reported; or (b) its primal and dual bounds differ by at most 10^{-4}, in which case the dual bound is reported. With a positive gap tolerance an "optimal" status proves only that the relative gap is below it, so it is not treated as (a). In all other cases the dual bound is reported, uncertified. A dual bound outside $[0, 1]$ (such as the $\pm 10^{20}$ sentinel of an infeasible or unbounded run), an infeasible program, or a solver failure gives the vacuous endpoint (0 for a minimum, 1 for a maximum), and all values are clamped to $[0, 1]$. Every interval solved on the cluster program by SCIP is therefore an outer bound, and it is exact (up to 10^{-4}) when H1–H4 hold and both endpoints are certified. The implementation reports an endpoint as exact only in that case, checking H1–H3 on the tree of its own route (shared or augmented); the experiments use a* CJT *endpoint as a reference only if it is certified and structurally exact. Endpoints computed by the direct fallback use a local solver and carry no guarantee.*

Remark 4 (A local solver yields an inner bound)**.** *Solved instead by a local method from a finite set of starting points — keeping the smallest minimum and largest maximum found — the procedure returns, at feasible points, a value no more extreme than the true optimum in each direction, so the interval is contained in the exact one, an* inner *bound in the sense of Definition 8. Points accepted within the solver's feasibility tolerance can violate this slightly, so the ipopt variant carries no formal guarantee. The implementation starts ipopt from uniform cluster marginals and restarts it from random points (at most four times) only when the result is suspicious — a failure, or a value within tolerance of the vacuous endpoint; terminations reported as optimal, locally optimal, feasible or acceptable are accepted. Global solution is what makes Theorem 8 applicable.*

Query and evidence in different clusters. When a query atom a and the evidence share no cluster of the shared tree (H4 fails for a), CJT builds for a a min-fill tree whose scopes are augmented by the node set of $\{a\} \cup \mathbf{E}$, which restores H4; every node, the query's included, is eliminated. It discharges the assertions on that tree with the min-fill triage only (no ordered tree is built for a single query), checks H1–H3 there, and solves the conditional query on it.

Fallback. The direct 2^n formulation is used for every atom when no candidate shared tree is within the budget β (the min-fill tree before widening exceeds β and the ordered tree is unavailable) or a

sentence has no host; and for a single atom when its augmented tree exceeds β or leaves a sentence unhosted. Widening never exceeds β, so it never causes a fallback.

Remark 5 (The fallback is exact in formulation, not in certificate)**.** *The fallback runs the direct method of Marinescu et al. (2022) (`ExactInference`) with ipopt, backed for marginal queries by its SLSQP fallback. It optimizes the true 2^n program over $\mathcal{D}_{\mathrm{LCN}}$, but by a local method, so its output carries no global guarantee and can be over-tight. Conversely, even at n small enough for the joint to be written down, the cluster program is often the easier one to certify: on the worked example of Section C.1 it certifies all endpoints where the 2^n formulation, solved by SCIP, certifies only seven of ten.*

A failed optimization is reported as the corresponding vacuous endpoint (0 for a minimum, 1 for a maximum): uninformative, never unsound. If every marginal returns vacuous, the instance or the evidence is likely inconsistent.

B.4 Complexity

This section proves Theorem 2. Two facts about bucket elimination are used: every node of the bucket cluster of v other than v is eliminated after v, and the cluster of v's parent bucket contains only nodes eliminated after v (Proposition 4(a), Step 1, whose proof holds for any elimination order).

Variables. One per cluster state, so by equation 12

$$\sum_{C\in\mathcal{C}} 2^{|C|} \;\le\; m\,2^{w+1} \;=\; O\big(n\,2^{w+1}\big), \tag{13}$$

since bucket elimination creates one cluster per node of the component DAG and leaf pruning only removes clusters, so $|\mathcal{C}| \le m \le n$. Under evidence the scale r is added. Against 2^n for the direct formulation, this is the algorithm's reason for being: the cost is exponential in the width, not in the number of atoms.

Constraints. There are $|\mathcal{C}|$ normalization equalities, and $\sum_{(C,C')\in T,\, S\neq\varnothing} 2^{|S|} \le (m-1)\,2^w$ separator equalities (empty separators impose nothing). Indeed a forest on $|\mathcal{C}| \le m$ clusters has at most $m-1$ edges, and every edge joins the cluster C_v of some bucket v to the cluster of its parent bucket (pruning removes only leaves, so the remaining edges are unchanged), which contains no atom of v by the second fact above; hence $S \subseteq C_v \setminus \mathrm{ch}_v$ and $|S| \le w$. Each hosted sentence gives two rows. A hosted assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ gives the $X{=}1$ row equation 20 for every configuration $(\mathbf{y}, \mathbf{z})$, so the implementation builds

$$2^{|\mathbf{Y}|+|\mathbf{Z}|} \;\le\; 2^w \quad \text{equalities,} \tag{14}$$

because $X \cup \mathbf{Y} \cup \mathbf{Z} \subseteq C$ gives $|\mathbf{Y}|+|\mathbf{Z}| \le w$. For each $\mathbf{z}$ the rows summed over $\mathbf{y}$ vanish identically, $\sum_{\mathbf{y}} \big(P(x,\mathbf{y},\mathbf{z})P(\mathbf{z}) - P(x,\mathbf{z})P(\mathbf{y},\mathbf{z})\big) = P(x,\mathbf{z})P(\mathbf{z}) - P(x,\mathbf{z})P(\mathbf{z}) = 0$, so one row per $\mathbf{z}$ is redundant and $2^{|\mathbf{Z}|}(2^{|\mathbf{Y}|}-1)$ of them are independent. When H3 holds the imposed assertions number at most n: at most $|\Lambda| \le n$ on a min-fill tree (the LMC gives at most one assertion per atom, and reduction replaces an assertion by one), and at most one per atom of each node on an ordered tree. (Relaxation can impose up to $|\mathbf{Y}_{\mathrm{in}}| \le w$ consequences per relaxed assertion.) This gives $O(n\,2^w)$ independence rows. Under evidence the scaled program adds the variable r and the normalization row $\langle A_{\mathbf{e}}, y_{C_{\mathbf{e}}}\rangle = 1$; the implementation also adds the valid inequalities $r \ge 1$ and $y_{C,i} \le r$ of Section C, which are implied by the program and not counted as rows.

Remark 6 (Which rows are nonconvex, and why the width bound survives)**.** *(a) Only the assertion equalities equation 20 are nonconvex; the conditional objective is linear after the scaling equation 22. The cleared Type-2 rows equation 19 are* linear *in q, since ℓ and u are constants. Consequently an instance with no evidence and no hosted assertion yields a* linear *program, even when it is loopy; the example of Section C.1 has two bilinear rows, from its one hosted assertion. (b) The count equation 14 appears unbounded in $|\mathbf{Y}|$, but a* hosted *assertion satisfies $X \cup \mathbf{Y} \cup \mathbf{Z} \subseteq C$, so it contributes at most 2^w rows and equation 13 is unaffected. An assertion with a wide $\mathbf{Y}$ is reduced, by Lemma 7, to the part of $\mathbf{Y}$ connected to X; what remains is hosted, possibly after widening, which increases w itself, or relaxed to hostable consequences.*

Per atom the cost is two optimizations over the shared system, hence $2|\mathbf{X}|$ in total, plus two per augmented tree.

C THE NONLINEAR PROGRAM

Fix a cluster C and index the $2^{|C|}$ assignments of C by i, writing ω_i^C for the i-th. For a formula φ with $\mathrm{atoms}(\varphi) \subseteq C$ let $A_\varphi^{(C)} \in \{0,1\}^{2^{|C|}}$ be its indicator vector, $A_\varphi^{(C)}[i] = \mathbf{1}[\omega_i^C \models \varphi]$ ($A_\top^{(C)} = \mathbf{1}$), and abbreviate $\langle v, q_C\rangle = \sum_i v_i q_{C,i}$; thus $\langle A_\varphi^{(C)}, q_C\rangle = P(\varphi)$ whenever q_C is the marginal of a joint on C. A configuration $\mathbf{y}$ of an atom set $\mathbf{Y}$ is identified with the conjunction of its literals, and x denotes the literal $X{=}1$.

Variables and normalization. One distribution per cluster,

$$q_C \in [0,1]^{2^{|C|}}, \qquad \sum_{i=0}^{2^{|C|}-1} q_{C,i} = 1 \qquad \forall C \in \mathcal{C}. \tag{15}$$

There are no separator variables, no per-family conditional variables and no global joint variable; the tree structure enters only through the equalities below.

Marginalization. Let $\pi_{C\to S}$ map a state of C to its restriction to $S \subseteq C$, and define the $0/1$ aggregation matrix

$$M_{\sigma,j}^{C\to S} = \mathbf{1}\big[\pi_{C\to S}(j) = \sigma\big], \qquad \text{so} \qquad \big(M^{C\to S} q_C\big)[\sigma] = (q_C \restriction S)(\sigma). \tag{16}$$

Separator consistency. For every edge with a non-empty separator, the two endpoints must agree on it:

$$\sum_{j:\,\pi_{C\to S}(j)=\sigma} q_{C,j} \;=\; \sum_{k:\,\pi_{C'\to S}(k)=\sigma} q_{C',k} \qquad \forall (C,C') \in T,\ S = C\cap C',\ \forall \sigma \in \{0,\dots,2^{|S|}-1\}, \tag{17}$$

equivalently $q_C \restriction S = q_{C'} \restriction S$: linear equalities, $2^{|S|}$ per edge. For an empty separator both sides are the constant 1 by normalization, so nothing is imposed. No messages are computed; the consistency conditions are imposed simultaneously with all others, which is what distinguishes this formulation from message passing.

Sentences. A Type-1 sentence hosted on $C_h \supseteq \mathrm{atoms}(\varphi)$ contributes

$$\ell \;\le\; \big\langle A_\varphi^{(C_h)},\, q_{C_h}\big\rangle \;\le\; u, \tag{18}$$

and a Type-2 sentence contributes the cleared pair

$$\ell\,\big\langle A_\psi^{(C_h)}, q_{C_h}\big\rangle \;\le\; \big\langle A_{\varphi\wedge\psi}^{(C_h)}, q_{C_h}\big\rangle \;\le\; u\,\big\langle A_\psi^{(C_h)}, q_{C_h}\big\rangle. \tag{19}$$

No division appears, and consistently with Remark 1 the rows equation 19 are vacuously satisfied when $P(\psi) = 0$: a Type-2 sentence constrains nothing on a null conditioning event.

Independence assertions. For a hosted $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$, one equality per configuration $(\mathbf{y},\mathbf{z})$ of $\mathbf{Y}\cup\mathbf{Z}$ (all indicators over C_h):

$$\langle A_{x\wedge\mathbf{y}\wedge\mathbf{z}}, q_{C_h}\rangle\,\langle A_{\mathbf{z}}, q_{C_h}\rangle - \langle A_{x\wedge\mathbf{z}}, q_{C_h}\rangle\,\langle A_{\mathbf{y}\wedge\mathbf{z}}, q_{C_h}\rangle = 0, \tag{20}$$

which is the $X{=}1$ slice of the equalities of Definition 6; with $\mathbf{Z} = \varnothing$ ($A_{\mathbf{z}} = \mathbf{1}$) it is the marginal form $\langle A_{x\wedge\mathbf{y}}, q\rangle - \langle A_x, q\rangle\langle A_{\mathbf{y}}, q\rangle = 0$ for $P(x,\mathbf{y}) = P(x)P(\mathbf{y})$. These rows are bilinear, hence nonconvex, and are the reason a global solver is required.

Three features of this encoding matter. First, only the $X{=}1$ slice is needed: since $A_{\neg x\wedge\varphi} = A_\varphi - A_{x\wedge\varphi}$, the $X{=}0$ equality at $(\mathbf{y},\mathbf{z})$ is the difference of the identity $\langle A_{\mathbf{y}\wedge\mathbf{z}}, q\rangle\langle A_{\mathbf{z}}, q\rangle = \langle A_{\mathbf{z}}, q\rangle\langle A_{\mathbf{y}\wedge\mathbf{z}}, q\rangle$ and equation 20. Second, a configuration $\mathbf{z}$ with $P(\mathbf{z}) = 0$ is vacuous, as Definition 6 requires: then $P(x,\mathbf{z}) \le P(\mathbf{z}) = 0$ and both products vanish. Third, the configurations $\mathbf{y}$ must range over *whole* assignments of $\mathbf{Y}$. Replacing $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ by the pairwise family $\{(X \perp\!\!\!\perp y \mid \mathbf{Z})\}_{y\in\mathbf{Y}}$ is strictly weaker for $|\mathbf{Y}| \ge 2$ — pairwise independence does not imply joint independence — and under-constrains the program, yielding spuriously loose bounds.

Objective. Let C_a be a cluster containing the query atom a and all evidence atoms (hypothesis H4). Without evidence the read-off is linear,

$$\rho_a(q) = \left\langle A_a^{(C_a)},\, q_{C_a} \right\rangle = P(a{=}1). \tag{21}$$

With evidence $\mathbf{e}$ the read-off $\rho_a(q) = \langle A_{a\wedge\mathbf{e}}, q_{C_a}\rangle / \langle A_{\mathbf{e}}, q_{C_a}\rangle$, defined where the denominator is positive, is linear-fractional, and is made linear by the Charnes–Cooper transformation (Charnes & Cooper, 1962). The variables become $y_C \in \mathbb{R}^{2^{|C|}}_{\geq 0}$ for every cluster and a scale $r \geq 0$ (intended $y_C = r\, q_C$, $r = 1/P(\mathbf{e})$), with the rows

$$\textstyle\sum_i y_{C,i} = r \quad \forall C, \qquad \left\langle A_{\mathbf{e}}^{(C_{\mathbf{e}})}, y_{C_{\mathbf{e}}} \right\rangle = 1, \qquad \rho_a^{\mathrm{cc}}(y) = \left\langle A_{a\wedge\mathbf{e}}^{(C_a)},\, y_{C_a} \right\rangle, \tag{22}$$

where $C_{\mathbf{e}} \supseteq \mathbf{E}$ is the cluster carrying the normalization row (in the implementation, the first such cluster; $C_{\mathbf{e}} = C_a$ is allowed). Every other row is made homogeneous by multiplying its lower-degree terms by powers of r. The separator equalities, the cleared Type-2 rows and the rows equation 20 with $\mathbf{Z} \neq \varnothing$ are already homogeneous and are imposed on y unchanged; the Type-1 rows become $\ell\, r \leq \langle A_\varphi, y\rangle \leq u\, r$, and the marginal assertion rows $\langle A_{x\wedge\mathbf{y}}, q\rangle = \langle A_x, q\rangle\langle A_{\mathbf{y}}, q\rangle$ become $r\,\langle A_{x\wedge\mathbf{y}}, y\rangle = \langle A_x, y\rangle\langle A_{\mathbf{y}}, y\rangle$. The bounds $q \leq 1$ of equation 15 are dropped (they are implied by normalization and $q \geq 0$). The implementation adds the valid inequalities

$$r \;\geq\; 1, \qquad y_{C,i} \;\leq\; r \quad \forall C, i,$$

which every feasible point satisfies — $r = \sum_i y_{C_{\mathbf{e}},i} \geq \langle A_{\mathbf{e}}, y_{C_{\mathbf{e}}}\rangle = 1$ because $y \geq 0$, and $y_{C,i} \leq \sum_j y_{C,j} = r$ — and without which the feasible region is unbounded and spatial branch-and-bound stalls. Step 7 of the proof of Theorem 8 shows that $(y, r) \mapsto y/r$ is a bijection onto the feasible q with $P_q(\mathbf{e}) > 0$ that carries ρ_a^{cc} to ρ_a; so the program is exact for the conditional query without any denominator floor (Remark 9), and it is infeasible exactly when $P_q(\mathbf{e}) = 0$ for every feasible q, in which case the conditional bound is $[0, 1]$ (Definition 7) and is reported as such.

The program. Collecting equation 15–equation 21, for a marginal query,

$$\begin{array}{rlll} \underline{P}(a),\, \overline{P}(a) \;=\; \min / \max & \rho_a(q) & & \\ \text{s.t.} & \sum_i q_{C,i} = 1, \quad q_{C,i} \in [0,1] & \forall C \in \mathcal{C} \\ & q_C \restriction S = q_{C'} \restriction S & \forall (C, C') \in T,\; S = C \cap C' \neq \varnothing \\ & \text{equation 18, equation 19} & \forall \text{ hosted sentence} \\ & \text{equation 20} & \forall \text{ hosted row of } \Lambda_T. \end{array} \tag{23}$$

For a conditional query the same constraints are imposed on y as described above, together with equation 22, and the $\inf$ and $\sup$ of ρ_a^{cc} are computed.

C.1 A WORKED EXAMPLE

We work the LCN of Figure 1 completely. It is small enough that every variable and constraint can be written down, yet it exercises every part of the construction: its structure is loopy after moralization, it has a compound node, two of its three LMC assertions are separator-entailed, and the third must be imposed as bilinear rows. Figure 4 summarizes the instance, its structure, the junction tree and the program.

C.1.1 THE INSTANCE

The sentences are shown in Figure 4(a). The structure (Figure 4(b)) has the directed edges $B \to A$, $E \to A$, $A \to C$, $A \to D$ and the undirected edge $C - D$, so it is a chain graph with components $\{B\}$, $\{E\}$, $\{A\}$ and $\{C, D\}$. The component DAG has the families

$$B \mid \varnothing, \qquad E \mid \varnothing, \qquad A \mid B, E, \qquad CD \mid A,$$

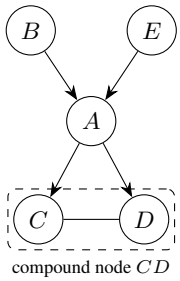
and the LMC yields the three assertions listed in the figure. Moralization connects the co-parents B and E of A, so the moral graph contains the triangle B, E, A and is not singly connected.

(a) LCN

s_1: $0.1 \le P(B) \le 0.2$
s_2: $0.05 \le P(E) \le 0.1$
s_3: $0.6 \le P(A \mid B \vee E) \le 0.9$
s_4: $0.5 \le P(\neg(C \oplus D) \mid A) \le 0.7$
s_5: $0.05 \le P(A) \le 0.1$

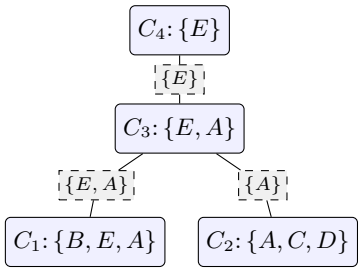
LMC: $(B \perp\!\!\!\perp E)$,
$(C \perp\!\!\!\perp B, E \mid A, D)$,
$(D \perp\!\!\!\perp B, E \mid A, C)$

(b) chain graph



compound node CD

(c) junction tree



s_1, s_2, s_3, s_5
$(B \perp\!\!\!\perp E)$
s_4

(d) nonlinear program for $\underline{P}(A)$ / $\overline{P}(A)$, with $q_1 \in \Delta(8)$ over (B, E, A), $q_2 \in \Delta(8)$ over (A, C, D), $q_3 \in \Delta(4)$ over (E, A), $q_4 \in \Delta(2)$ over (E):

$$\min / \max \; P_{q_1}(A)$$

s.t. $q_k \in \Delta(2^{\lvert C_k \rvert})$, $k = 1, \dots, 4$	(4 simplex)
$q_1 \restriction \{E, A\} = q_3, \quad q_2 \restriction \{A\} = q_3 \restriction \{A\}, \quad q_3 \restriction \{E\} = q_4$	(4+2+2 separator)
$0.1 \le P_{q_1}(B) \le 0.2, \;\; 0.05 \le P_{q_1}(E) \le 0.1, \;\; 0.05 \le P_{q_1}(A) \le 0.1$	(6 sentence, linear)
$0.6\, P_{q_1}(B \vee E) \le P_{q_1}(A \wedge (B \vee E)) \le 0.9\, P_{q_1}(B \vee E)$	(2 sentence, linear)
$0.5\, P_{q_2}(A) \le P_{q_2}(A \wedge \neg(C \oplus D)) \le 0.7\, P_{q_2}(A)$	(2 sentence, linear)
$P_{q_1}(B{=}1, E{=}y) = P_{q_1}(B{=}1)\, P_{q_1}(E{=}y), \quad y \in \{0, 1\}$	(2 assertion, bilinear)

Figure 4: The worked example. (a) The LCN of Figure 1 and the assertions of its LMC. (b) Its structure: C and D form an undirected component, i.e. a compound node. (c) The junction tree, with separators dashed on the edges and the constraints hosted by each cluster below it. (d) The resulting program: 22 variables and 24 constraints, of which only the two rows of $(B \perp\!\!\!\perp E)$ are nonlinear; $P_{q_k}(\varphi)$ abbreviates $\langle A_\varphi^{(C_k)}, q_k \rangle$.

C.1.2 The junction tree

Bucket elimination along a min-fill order produces the four clusters

$$C_1 = \{B, E, A\}, \qquad C_2 = \{A, C, D\}, \qquad C_3 = \{E, A\}, \qquad C_4 = \{E\},$$

with tree edges $C_1 - C_3$, $C_2 - C_3$ and $C_3 - C_4$ and separators $\{E, A\}$, $\{A\}$ and $\{E\}$ (Figure 4(c)). The largest cluster has three atoms, so $w = 2$. The compound node CD contributes both of its atoms to C_2 at once, which is why that cluster has three atoms although it comes from a family with only two nodes. The clusters containing each atom are connected in the tree (B: C_1; E: C_1, C_3, C_4; A: C_1, C_3, C_2; C, D: C_2), so H1 holds. Every sentence fits in a cluster (H2): s_1, s_2, s_3 and s_5 mention only B, E and A and are hosted on C_1, and s_4 is hosted on C_2.

The graph $G^*(T)$ joins two atoms when they share a cluster; its edges are BE, BA, EA, AC, AD and CD. The ordered construction of Proposition 4 does not apply here, because the assertions on C and on D each condition on the other member of the compound node; the min-fill triage of Section B.2 is used instead.

C.1.3 Discharging the assertions

$(C \perp\!\!\!\perp B, E \mid A, D)$ and $(D \perp\!\!\!\perp B, E \mid A, C)$: separator-entailed. Every path in $G^*(T)$ from C or D to B or E passes through A, so deleting the conditioning set $\{A, D\}$ (resp. $\{A, C\}$) leaves C (resp. D) with no neighbour. The reduction of Lemma 7 is therefore empty, and no row is imposed: every joint reconstructed from consistent cluster marginals satisfies these assertions.

$(B \perp\!\!\!\perp E)$: hosted. B and E are adjacent in $G^*(T)$, so the empty set does not separate them and the reduction keeps $\mathbf{Y}_{\text{in}} = \{E\}$. Both atoms lie in C_1, so the assertion is imposed there in the marginal form of equation 20, one row per state of E (only the $B{=}1$ slice is needed):

$$P_{q_1}(B{=}1, E{=}y) = P_{q_1}(B{=}1)\, P_{q_1}(E{=}y), \qquad y \in \{0, 1\}.$$

These two rows, of which one is implied by the other and normalization (Section B.4), are the only nonlinear constraints of the program. Without them the program could correlate B and E, which the LCN forbids, and the bounds on A would widen. With them H3 holds: the imposed rows are those of an LMC assertion, and by Lemma 7 the hosted assertion together with the two separator-entailments gives every assertion of the LMC on every reconstructed joint.

C.1.4 THE PROGRAM

The program is shown in Figure 4(d). Its $8 + 8 + 4 + 2 = 22$ variables are the cells of $q_1, \dots, q_4$. The constraints are 4 simplex equalities; 8 separator equalities, 4 for $\{E, A\}$, 2 for $\{A\}$ and 2 for $\{E\}$, for example

$$q_1[B{=}0, E{=}\epsilon, A{=}\alpha] + q_1[B{=}1, E{=}\epsilon, A{=}\alpha] = q_3[E{=}\epsilon, A{=}\alpha], \qquad \epsilon, \alpha \in \{0, 1\};$$

10 sentence rows, all linear since the cleared Type-2 rows equation 19 are linear in q; and the 2 bilinear assertion rows: 24 constraints in all. The direct formulation over the joint has $2^5 = 32$ variables; the saving grows exponentially with n at fixed width (Theorem 2).

C.1.5 RESULTS

Solving the program for each atom with SCIP certifies all ten endpoints, in 3.5 s in total:

engine	$P(A)$	$P(B)$	$P(C)$	$P(D)$	$P(E)$
CJT (SCIP)	[0.0870, 0.1000]	[0.1000, 0.1228]	[0, 1]	[0, 1]	[0.0500, 0.0741]
direct 2^n (SCIP)	[–, 0.1000]	[0.1000, –]	[0, 1]	[0, 1]	[0.0500, –]

The direct global formulation over the 2^5 joint certifies only 7 of its 10 endpoints within a time limit of 120 s per solve (381 s in total), and those 7 agree with CJT; a dash marks an endpoint it did not certify. The three remaining CJT values can be checked by hand; write $\theta_B = P(B)$ and $\theta_E = P(E)$. *Every member is bounded:* by s_3 and the independence of B and E, $P(A) \geq 0.6\,P(B \vee E) = 0.6\,(\theta_B + \theta_E - \theta_B\theta_E)$, which is increasing in both and so at least $0.6 \cdot 0.145 = 0.087$ under s_1, s_2; conversely s_5 forces $0.6\,P(B \vee E) \leq P(A) \leq 0.1$, i.e. $\theta_B + \theta_E - \theta_B\theta_E \leq 1/6$, so $\theta_B \leq (1/6 - \theta_E)/(1 - \theta_E) \leq (1/6 - 0.05)/0.95 = 0.1228$ and likewise $\theta_E \leq (1/6 - 0.1)/0.9 = 0.0741$. *Each bound is attained:* let B and E be independent with marginals (θ_B, θ_E) equal to $(0.1, 0.05)$, $(0.1228, 0.05)$ or $(0.1, 0.0741)$, let $P(A \mid B \vee E) = 0.6$ and $P(A \mid \neg B \wedge \neg E) = 0$, and let (C, D) be uniform given A, independently of B, E. This joint satisfies s_1–s_5 ($P(A) = 0.6\,P(B \vee E) \in [0.087, 0.1]$, $P(\neg(C \oplus D) \mid A) = 0.5$) and the three assertions, and it attains $\underline{P}(A) = 0.087$, $\overline{P}(B) = 0.1228$ and $\overline{P}(E) = 0.0741$ respectively. The atoms C and D are vacuous because s_4 constrains only whether C and D agree.

Table 4 lists the separator marginals at the minimizer of $P(A)$. Both clusters incident to an edge realize the same vector, as the separator equalities require; and the marginal of q_1 on $\{B, E\}$ is the product of its marginals on B and on E, $P(B{=}1) = 0.1$ and $P(E{=}1) = 0.05$, as the hosted assertion requires. Lemma 6 reassembles these cluster marginals into a joint distribution in $\mathcal{D}_{\mathrm{LCN}}$ attaining $\underline{P}(A)$.

Table 4: Separator marginals at the minimizer of $P(A)$, by assignment of the separator (bits in the listed order). Both clusters incident to each edge realize the same vector.

edge	separator	marginal			
$C_1 - C_3$	$\{E, A\}$	00: 0.8898	01: 0.0602	10: 0.0232	11: 0.0268
$C_2 - C_3$	$\{A\}$	0: 0.9130	1: 0.0870		
$C_3 - C_4$	$\{E\}$	0: 0.9500	1: 0.0500		

C.1.6 A SECOND STRUCTURE

Table 5 compares the program size with that of the five-atom `asia` example, a DAG on which the ordered construction of Proposition 4 applies: there two ordered rows, $(X \perp\!\!\!\perp D \mid C)$ and $(C \perp\!\!\!\perp B \mid S)$, imply all four LMC assertions.

Table 5: Program size on the worked example and on `asia`.

instance	tree	clus.	$w{+}1$	edges	vars	constr.	simplex	sep.	sent.	assert.
alarm (Fig. 4)	min-fill	4	3	3	22	24	4	8	10	2
`asia`	ordered	5	3	4	30	37	5	14	10	8

C.2 Exactness

This section states and proves the guarantees. Besides Lemma 2, the development uses four standard facts, each valid for every distribution, without positivity: the conditional-independence semantics of Definition 6 and its semi-graphoid axioms (symmetry, decomposition, weak union, contraction); factorization over an undirected graph implies its global Markov property (Lauritzen, 1996, Prop. 3.8); factorization along a DAG implies its d-separation property (Lauritzen, 1996, Thm. 3.27); and the Helly property of subtrees of a tree (proved in Lemma 5). It needs no notion of realizability, no property of the strong extension, and no result about any other engine. The chain of results is Lemma 5 $\rightarrow$ Lemmas 6, 7 and Proposition 4 $\rightarrow$ Theorem 8.

Definition 9 (Cluster forest, hosting, cluster-feasible set, read-off)**.** *Let T be a forest whose nodes, the* clusters *$C \in \mathcal{C}$, are sets of atoms with $\bigcup_C C = \mathbf{X}$ (in* CJT*, the tree of Section B.1). The separator of an edge (C, C') is $S = C \cap C'$.*

- *$G^*(T)$ is the graph on $\mathbf{X}$ in which two atoms are adjacent iff they share a cluster of T.*
- *T has the* running-intersection property *(RIP) if, for every atom, the clusters containing it induce a connected subgraph of T (a subtree; in particular they all lie in one component of the forest). This is H1.*
- *A distribution p* factorizes over *T if $p(\mathbf{x}) = \prod_{C \in \mathcal{C}} \phi_C(\mathbf{x}_C)$ for non-negative functions ϕ_C, and* over the cliques of *$G^*(T)$ if it is such a product with one factor per clique of $G^*(T)$.*
- *A constraint c is* hosted *if some cluster $C_h \supseteq \mathrm{scope}(c)$ exists (Section B.2); its rows are then imposed on q_{C_h}.*
- *An assertion $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ is* separator-entailed *if $\mathbf{Z}$ separates X from $\mathbf{Y}$ in $G^*(T)$ (every path from X to $\mathbf{Y}$ meets $\mathbf{Z}$).*
- *Λ is the LCN's set of LMC assertions, and the* imposed rows *Λ_T are the independence statements actually hosted on T: reduced LMC assertions or their relaxed consequences (Lemma 7) on a min-fill tree, or the rows equation 24 on an ordered tree.*
- *The* cluster-feasible set *is*

$$\mathcal{G} = \big\{(q_C)_{C\in\mathcal{C}} \ : q_C \in \Delta(2^{|C|}) \text{ for each } C;\\ q_C \restriction S = q_{C'} \restriction S \text{ for every edge } (C, C') \in T,\ S = C \cap C';\\ \text{every hosted sentence and every row of } \Lambda_T \text{ holds on its host}\big\}.$$

  For $S = \varnothing$ the separator condition is vacuous: both sides are the marginal on the empty set, the constant 1*.*

- *For a query atom a and evidence $\mathbf{e}$ on $\mathbf{E}$ with $\{a\} \cup \mathbf{E} \subseteq C_a$, the* read-off *is $\rho_a(q) = \langle A_{a\wedge\mathbf{e}}, q_{C_a}\rangle / \langle A_{\mathbf{e}}, q_{C_a}\rangle$, defined on $\mathcal{G}^+ = \{q \in \mathcal{G} : \langle A_{\mathbf{e}}, q_{C_{\mathbf{e}}}\rangle > 0\}$, where $C_{\mathbf{e}} \supseteq \mathbf{E}$ is the cluster of the normalization row; for $\mathbf{E} = \varnothing$, $A_{\mathbf{e}} = \mathbf{1}$, $\mathcal{G}^+ = \mathcal{G}$ and $\rho_a(q) = \langle A_a, q_{C_a}\rangle$. ρ_a^{cc} is the linear objective equation 22 of the scaled program.*
- *$\pi(p) = (p \restriction C)_{C\in\mathcal{C}}$ is the* projection *of a joint onto the clusters, and $\mathcal{D}^+_{\mathrm{LCN}} = \{p \in \mathcal{D}_{\mathrm{LCN}} : p(\mathbf{e}) > 0\}$.*

Lemma 5 (Helly property)**.** *Let T have the RIP. (a) Every cluster is a clique of $G^*(T)$. (b) Every clique of $G^*(T)$ is contained in some cluster. Hence a distribution factorizes over the cliques of $G^*(T)$ iff it factorizes over T, and the maximal cliques of $G^*(T)$ are among the clusters.*

Proof. (a) Any two atoms of a cluster share it, so they are adjacent in $G^*(T)$.

(b) Let K be a clique ($K = \varnothing$ lies in every cluster). For $X \in K$ let T_X be the set of clusters containing X; by the RIP it induces a subtree. Any two $X, Y \in K$ are adjacent, so they share a cluster: $T_X \cap T_Y \neq \varnothing$. In particular all T_X lie in one component of T, which is a tree. We show by induction on k that k pairwise intersecting subtrees $T_1, \ldots, T_k$ of a tree have a common node. *Step 1* ($k \leq 2$): immediate. *Step 2* ($k = 3$): pick nodes $\alpha \in T_1 \cap T_2$, $\gamma \in T_2 \cap T_3$, $\delta \in T_1 \cap T_3$. In a tree the three paths between α, γ, δ share a node μ (the median of the three). The path α–γ lies in T_2, γ–δ in T_3 and α–δ in T_1, since a subtree contains the tree path between any two of its nodes; so $\mu \in T_1 \cap T_2 \cap T_3$. *Step 3* ($k > 3$): $T_1 \cap T_2$ is a non-empty subtree, and by Step 2 it meets every T_j, $j \geq 3$; the $k - 1$ subtrees $T_1 \cap T_2, T_3, \ldots, T_k$ are pairwise intersecting, and the induction hypothesis gives a common node. A common node of the T_X, $X \in K$, is a cluster containing K.

For the consequence: a product of factors over clusters is one over cliques by (a), and a product over cliques becomes one over clusters by assigning each factor to a cluster containing its clique (b) and multiplying the factors assigned to the same cluster. A maximal clique lies in a cluster (b), which is a clique (a), so it equals that cluster. □

The reconstruction direction rests on the following lemma. We state and prove it for distributions with zeros, where the familiar quotient formula involves $0/0$.

Lemma 6 (RIP reconstruction, with zeros)**.** *Let T be a cluster forest with the RIP, and let $(q_C) \in \prod_C \Delta(2^{|C|})$ be separator-consistent: $q_C \restriction S = q_{C'} \restriction S$ for every edge (C, C') with $S = C \cap C'$. Then there exists a joint distribution $p \in \Delta(2^{\mathbf{X}})$ with $p \restriction C = q_C$ for every $C \in \mathcal{C}$, and p factorizes over T, hence (Lemma 5) over the cliques of $G^*(T)$.*

Proof. Step 0 (a forest reduces to a tree). By the RIP the clusters containing an atom lie in a single component of T, so distinct components share no atom. Joining the components by arbitrary edges therefore gives a tree whose new separators are empty, whose separator conditions are vacuous (Definition 9), and which still has the RIP, since the clusters of each atom and the edges among them are unchanged. Hence we may assume that T is a tree. We prove, by induction on $|\mathcal{C}|$, the statement for a tree whose clusters cover an atom set $\mathbf{X}_T$, the joint being over $\mathbf{X}_T$.

Step 1 (base). If $|\mathcal{C}| = 1$, take $p = q_C$.

Step 2 (removing a leaf). Let $|\mathcal{C}| \geq 2$, pick a leaf cluster C with neighbour C' and separator $S = C \cap C'$, and let $T' = T - C$, with atoms $\mathbf{X}' = \bigcup_{C'' \in T'} C''$. (2a) T' is a tree with the RIP: removing a leaf of T removes a leaf (or the only node) from each atom's subtree, which stays connected. (2b) $C \setminus S$ meets no cluster of T': if an atom of C lies in $C'' \in T'$, its subtree contains C and C'', hence the path between them, which enters C through its only neighbour C'; so the atom lies in C', i.e. in S. Thus $\mathbf{X}_T = \mathbf{X}' \dot\cup (C \setminus S)$ and $S \subseteq \mathbf{X}'$. (2c) The clusters of T' are still separator-consistent, so the induction hypothesis gives a joint p' over $\mathbf{X}'$ with $p' \restriction C'' = q_{C''}$ for all $C'' \in T'$, and $p' = \prod_{C'' \in T'} \phi_{C''}$. (2d) By separator consistency, $p' \restriction S = (p' \restriction C') \restriction S = q_{C'} \restriction S = q_C \restriction S =: q_S$. If $S = \varnothing$, $q_S \equiv 1$ on the single empty assignment.

Step 3 (the kernel). For each assignment $\mathbf{s}$ of S define a distribution on the assignments of $C \setminus S$,

$$\kappa(\cdot \mid \mathbf{s}) = \begin{cases} q_C(\cdot, \mathbf{s})/q_S(\mathbf{s}), & q_S(\mathbf{s}) > 0, \\ \delta_{c_0}, & q_S(\mathbf{s}) = 0, \end{cases}$$

where c_0 is any fixed assignment of $C \setminus S$. When $q_S(\mathbf{s}) > 0$ the right-hand side is non-negative and sums over $C \setminus S$ to $q_S(\mathbf{s})/q_S(\mathbf{s}) = 1$. If $S = \varnothing$, $\kappa = q_C$.

Step 4 (the joint). Set $p(\mathbf{x}) = p'(\mathbf{x}_{\mathbf{X}'})\, \kappa(\mathbf{x}_{C \setminus S} \mid \mathbf{x}_S)$, well defined by (2b). It is non-negative, and summing over $\mathbf{x}_{C \setminus S}$ first gives p', which sums to 1.

Step 5 (the arbitrary choice is irrelevant). On $\{\mathbf{x} : q_S(\mathbf{x}_S) = 0\}$ we have $p'(\mathbf{x}_{\mathbf{X}'}) = 0$, because $p' \restriction S = q_S$ assigns zero mass to that event and $p' \geq 0$. Hence p vanishes there regardless of κ, so p does not depend on c_0. (This is also the precise sense in which the familiar quotient formula $p = \prod_C q_C / \prod_{(C,C')} q_S$ is legitimate: a zero separator cell forces every contributing cluster cell to zero, since $q_S = q_C \restriction S$ is a sum of non-negative q_C entries, so numerator and denominator vanish together and the term is assigned 0.)

Step 6 (cluster marginals). For $C'' \in T'$: summing p over $\mathbf{x}_{C \setminus S}$ gives p', so $p \restriction C'' = p' \restriction C'' = q_{C''}$. For C itself, at an assignment $(c, \mathbf{s})$ of $(C \setminus S, S)$: $p \restriction C(c, \mathbf{s}) = p' \restriction S(\mathbf{s})\, \kappa(c \mid \mathbf{s}) = q_S(\mathbf{s})\, \kappa(c \mid \mathbf{s})$,

which equals $q_C(c, \mathbf{s})$ when $q_S(\mathbf{s}) > 0$, and equals $0 = q_C(c, \mathbf{s})$ when $q_S(\mathbf{s}) = 0$ (a sum of non-negative entries $q_C(\cdot, \mathbf{s})$ that is zero). So $p \restriction C = q_C$.

Step 7 (factorization). $p = \kappa \cdot \prod_{C'' \in T'} \phi_{C''}$, and κ is a non-negative function of $\mathbf{x}_C$, so p factorizes over T. □

Remark 7 (Side remarks on Lemma 6)**.** *(a)* Uniqueness *(not used below): p is the unique joint with these cluster marginals that factorizes over T. If $\tilde{p}$ is another, the global Markov property of $G^*(T)$ (Lauritzen, 1996, Prop. 3.8) and the separation of $C \setminus S$ from $\mathbf{X}' \setminus S$ by S give $\tilde{p} = \tilde{p} \restriction \mathbf{X}' \cdot \tilde{p}(\mathbf{x}_{C \setminus S} \mid \mathbf{x}_S)$, where the conditional equals κ wherever $q_S(\mathbf{x}_S) > 0$ and both sides vanish elsewhere; summing out $C \setminus S$, whose atoms occur in no other cluster, leaves a factorization of $\tilde{p} \restriction \mathbf{X}'$ over T', so $\tilde{p} \restriction \mathbf{X}' = p'$ by induction. Joints that do not factorize may share the marginals. (b)* Why the RIP for forests matters. *If two components of a forest both contain an atom X, no separator ties the copies of X: the cluster family may give X different marginals in the two components, and then no joint has these marginals. This is exactly the defect of the earlier augmented tree described in Section B.1, which the RIP check now rejects.*

The two triages of Section B.2 discharge H3 through the following two results.

Lemma 7 (Exact reduction of an assertion)**.** *Let T be the final tree (after widening), let p factorize over the cliques of $G^*(T)$, and let $(X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ be an assertion with X, $\mathbf{Y}$, $\mathbf{Z}$ pairwise disjoint. Let K be the set of atoms reachable from X in $G^*(T)$ by paths avoiding $\mathbf{Z}$ ($X \in K$), $\mathbf{Y}_{\text{in}} = \mathbf{Y} \cap K$ and $\mathbf{Y}_{\text{out}} = \mathbf{Y} \setminus K$. Then $X \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z}$ holds under p if and only if $X \perp\!\!\!\perp \mathbf{Y}_{\text{in}} \mid \mathbf{Z}$ does. In particular the assertion is separator-entailed, and holds automatically, when $\mathbf{Y}_{\text{in}} = \varnothing$.*

Proof. All independences are in the sense of Definition 6. *Step 1* ($\Rightarrow$): decomposition. *Step 2* ($\mathbf{Z}$ separates K from $\mathbf{Y}_{\text{out}}$ in $G^*(T)$). The sets are disjoint from $\mathbf{Z}$ (by construction of K and by disjointness of $\mathbf{Y}$ and $\mathbf{Z}$). If a path from some $k \in K$ to some $y \in \mathbf{Y}_{\text{out}}$ avoided $\mathbf{Z}$, appending it to a $\mathbf{Z}$-avoiding path from X to k would make y reachable from X, i.e. $y \in K$, a contradiction. *Step 3.* Since p factorizes over the cliques of $G^*(T)$, it satisfies the global Markov property of $G^*(T)$ (Lauritzen, 1996, Prop. 3.8), so $K \perp\!\!\!\perp \mathbf{Y}_{\text{out}} \mid \mathbf{Z}$, and by decomposition $(X \cup \mathbf{Y}_{\text{in}}) \perp\!\!\!\perp \mathbf{Y}_{\text{out}} \mid \mathbf{Z}$, as $X \cup \mathbf{Y}_{\text{in}} \subseteq K$. If $\mathbf{Y}_{\text{in}} = \varnothing$ this is the claim. *Step 4.* Weak union gives $X \perp\!\!\!\perp \mathbf{Y}_{\text{out}} \mid \mathbf{Z} \cup \mathbf{Y}_{\text{in}}$. *Step 5* ($\Leftarrow$). Contraction of Step 4 with $X \perp\!\!\!\perp \mathbf{Y}_{\text{in}} \mid \mathbf{Z}$ yields $X \perp\!\!\!\perp \mathbf{Y}_{\text{in}} \cup \mathbf{Y}_{\text{out}} \mid \mathbf{Z}$. □

Proposition 4 (Ordered tree: H3 by construction)**.** *Let T be built by bucket elimination (Section B.1) along an order $v_1, \ldots, v_m$ of the nodes of D (eliminated in this order) that reverses a topological order of D, let $B(v) = \{v\} \cup N_v$ be the node set of the cluster of v's bucket, and suppose T has the RIP. Let $G_D^*(T)$ be the graph on the nodes in which two nodes are adjacent iff their atoms share a cluster.*

(a) N_v consists of nodes eliminated after v (topological predecessors of v, hence non-descendants), contains $\mathrm{Pa}(v)$, and equals the set of neighbours of v in $G_D^*(T)$ that are eliminated after v.

(b) If p factorizes over the cliques of $G^*(T)$ and satisfies, for every node v,

$$\mathrm{ch}_v \perp\!\!\!\perp \mathrm{atoms}\big(N_v \setminus \mathrm{Pa}(v)\big) \mid \mathrm{pa}_v, \tag{24}$$

then $p(\mathbf{x}) = \prod_v p(\mathbf{x}_{\mathrm{ch}_v} \mid \mathbf{x}_{\mathrm{pa}_v})$ for every $\mathbf{x}$ (each conditional chosen arbitrarily where its conditioning event is null), so p satisfies every d-separation of D between sets of nodes, read at atom level.

(c) Consequently, if every assertion of Λ is a d-separation of D between disjoint sets of whole nodes (check (ii) of Section B.2), the second half of H3 holds; and if each row equation 24 is implied by a single assertion of Λ with the same conditioning set by symmetry and decomposition (check (i)), the first half holds. When the LCN is DAG-structured, the first half also holds semantically: by Lemma 4 the LMC is the directed local Markov property of D, so every distribution satisfying Λ satisfies $v \perp\!\!\!\perp$ (non-descendants $\setminus \mathrm{Pa}(v)$) $\mid \mathrm{Pa}(v)$, and equation 24 follows by decomposition, by (a).

Proof. Write $u \prec v$ when u is eliminated before v.

Step 1 (every node of $B(v)$ other than v is $\succ v$; this holds for any elimination order). By induction along the order. $B(v)$ is the union of the scopes placed in v's bucket and of the messages $B(u) \setminus \{u\}$ of the buckets u whose parent is v. A scope is placed at its $\prec$-first node, here v, so its other nodes are $\succ v$. A message $B(u) \setminus \{u\}$ consists, by induction, of nodes $\succ u$, and its $\prec$-first node is the parent v, so its other nodes are $\succ v$. In particular the cluster of v's parent bucket $u' \succ v$ contains only nodes $\succeq u'$, hence not v.

Step 2 ($\mathrm{Pa}(v) \subseteq N_v$). The family scope $\{v\} \cup \mathrm{Pa}(v)$ is a covered scope. In a reverse topological order v is eliminated before each of its parents, so v is the scope's $\prec$-first node and the scope is placed in v's bucket. Since the order reverses a topological order, every node $\succ v$ is a topological predecessor of v, hence a non-descendant of v.

Step 3 (later neighbours). Let $u \prec v$ with $v \in B(u)$, and let $u_0 = u$, $u_{j+1} =$ the parent of u_j. While $u_j \prec v$, $v \in B(u_j) \setminus \{u_j\}$, the message $B(u_j) \setminus \{u_j\}$ is contained in $B(u_{j+1})$, and u_{j+1} is its $\prec$-first node, so $u_j \prec u_{j+1} \preceq v$; the sequence therefore reaches v. Every node $\succ v$ of $B(u)$ is never the removed node u_j, so it is carried into $B(v)$: $B(u) \cap \{w \succ v\} \subseteq N_v$. The clusters of T are bucket clusters (leaf pruning removes some), and every bucket cluster lies in a cluster of T (a pruned leaf lies in its parent's cluster, recursively). So the neighbours of v that are $\succ v$ in $G^*_D(T)$ are exactly N_v: $\supseteq$ because $B(v)$ lies in a cluster of T, $\subseteq$ by the inclusion just shown (and Step 1 for $B(v)$ itself). Steps 1–3 prove (a).

Step 4 (bucket factorization). By Lemma 5, p factorizes over T; every cluster of T is the atom set of some $B(w)$, so $p = \prod_w \phi_w(\mathbf{x}_{B(w)})$ with non-negative ϕ_w ($\phi_w \equiv 1$ for pruned buckets). Define, along the order, $\lambda_u(\mathbf{x}_{B(u)\setminus\{u\}}) = \sum_{\mathbf{x}_u} \phi_u \prod_{u' :\, \mathrm{parent}(u')=u} \lambda_{u'}$, a function of $B(u) \setminus \{u\}$ because each child message lies in $B(u)$. Let $R_k = \{v_k, \dots, v_m\}$, a prefix of the topological order. We claim

$$p \restriction R_k = \prod_{w \in R_k} \phi_w \prod_{u \in U_k} \lambda_u,$$

where U_k is the set of nodes $u \notin R_k$ whose parent bucket is in R_k or which have no parent (then λ_u is a constant). For $k = 1$ this is the factorization. For the step from k to $k+1$, the factors that mention v_k are ϕ_{v_k} and the λ_u with parent v_k: a ϕ_w with $w \succ v_k$ mentions only nodes $\succeq w$ (Step 1), and a λ_u whose parent is $w \succ v_k$ mentions only nodes $\succeq w$. Summing out $\mathbf{x}_{v_k}$ replaces these factors by λ_{v_k}, which proves the claim. In particular $p \restriction R_k = \psi_k(\mathbf{x}_{B(v_k)})\, F_k(\mathbf{x}_{R_k \setminus \{v_k\}})$ with $\psi_k = \phi_{v_k} \prod_{\mathrm{parent}(u)=v_k} \lambda_u$ and F_k the remaining factors.

Step 5 (one conditional per node). Note $R_k \setminus \{v_k\} = R_{k+1}$ ($R_{m+1} = \varnothing$, with marginal 1) and $\mathrm{Pa}(v_k) \subseteq N_{v_k} \subseteq R_{k+1}$ by (a). Fix $\mathbf{x}$ with $p(\mathbf{x}_{R_{k+1}}) > 0$. Then $p(\mathbf{x}_{v_k} \mid \mathbf{x}_{R_{k+1}}) = \psi_k(\mathbf{x}_{v_k}, \mathbf{x}_{N_{v_k}}) / \sum_{\mathbf{x}'_{v_k}} \psi_k(\mathbf{x}'_{v_k}, \mathbf{x}_{N_{v_k}})$ (the denominator is positive because $p(\mathbf{x}_{R_{k+1}}) = F_k \sum \psi_k > 0$), a function $g(\mathbf{x}_{v_k}, \mathbf{x}_{N_{v_k}})$ of $B(v_k)$ alone. Averaging over the configurations of $R_{k+1} \setminus N_{v_k}$ with weights $p(\mathbf{x}_{R_{k+1}})/p(\mathbf{x}_{N_{v_k}})$ gives $p(\mathbf{x}_{v_k} \mid \mathbf{x}_{N_{v_k}}) = g$ whenever $p(\mathbf{x}_{N_{v_k}}) > 0$. By equation 24 and Definition 6, $p(\mathbf{x}_{v_k} \mid \mathbf{x}_{N_{v_k}}) = p(\mathbf{x}_{v_k} \mid \mathbf{x}_{\mathrm{Pa}(v_k)})$ whenever $p(\mathbf{x}_{N_{v_k}}) > 0$. Hence $p(\mathbf{x}_{v_k} \mid \mathbf{x}_{R_{k+1}}) = p(\mathbf{x}_{v_k} \mid \mathbf{x}_{\mathrm{Pa}(v_k)})$ whenever $p(\mathbf{x}_{R_{k+1}}) > 0$.

Step 6 (the DAG factorization, with zeros). The sets $R_m \subset \cdots \subset R_1$ add one node at a time in topological order. If $p(\mathbf{x}) > 0$, every marginal $p(\mathbf{x}_{R_k})$ is positive, and the chain rule $p(\mathbf{x}) = \prod_k p(\mathbf{x}_{v_k} \mid \mathbf{x}_{R_{k+1}})$ with Step 5 gives $p(\mathbf{x}) = \prod_k p(\mathbf{x}_{v_k} \mid \mathbf{x}_{\mathrm{Pa}(v_k)})$. If $p(\mathbf{x}) = 0$, let k be the largest index with $p(\mathbf{x}_{R_k}) = 0$; it exists because $p(\mathbf{x}_{R_1}) = p(\mathbf{x}) = 0$, and $p(\mathbf{x}_{R_{k+1}}) > 0$ by maximality. Then $p(\mathbf{x}_{\mathrm{Pa}(v_k)}) \geq p(\mathbf{x}_{R_{k+1}}) > 0$, and by Step 5 $p(\mathbf{x}_{v_k} \mid \mathbf{x}_{\mathrm{Pa}(v_k)}) = p(\mathbf{x}_{R_k})/p(\mathbf{x}_{R_{k+1}}) = 0$, so the product vanishes too, whatever values the conditionals take on null events. So p factorizes along D.

Step 7 (d-separation). A distribution that factorizes along a DAG satisfies its d-separation property, without positivity (Lauritzen, 1996, Thm. 3.27); applied to the node variables, every d-separation of D holds, and it holds at atom level because each node variable is the tuple of its atoms. This proves (b).

Step 8 ((c)). If each assertion of Λ is a d-separation of D between sets of whole nodes, (b) gives the second half of H3 for every p that factorizes over the cliques of $G^*(T)$ and satisfies the rows. If each row is obtained from one assertion of Λ by symmetry and decomposition, every distribution satisfying Λ satisfies it (semi-graphoid axioms). The DAG-structured case is Lemma 4 followed by decomposition, since $N_v \setminus \mathrm{Pa}(v)$ consists of non-descendants of v that are not parents (a). □

The implementation checks, per instance, the two conditions (i) and (ii) of Section B.2 that make Proposition 4(c) applicable; in (ii) each of X, $\mathbf{Y}$, $\mathbf{Z}$ must be the union of the atoms of whole nodes, since conditioning on one atom of a compound node is not conditioning on the node. If either check fails, the ordered tree is discarded and the min-fill triage is used. They fail when an assertion splits a compound node, as in the worked example of Section C.1 and in the `smokers` example; for the latter the min-fill triage widens to a single 9-atom cluster, which is exact but no cheaper than the direct program.

Theorem 1 of the main text is restated verbatim below as Theorem 8; Theorem 2 is proved in Section B.4.

Theorem 8 (CJT exactness; Theorem 1 restated)**.** *Let $\mathcal{L}$ be a chain-graph LCN with LMC assertions Λ and $\mathcal{D}_{\mathrm{LCN}} \neq \varnothing$, let T be a cluster tree (or forest) over its atoms, let $G^*(T)$ be the graph on the atoms in which two atoms are adjacent iff they share a cluster of T, and let Λ_T be the independence rows imposed on T. Assume* (H1) *for every atom, the clusters of T containing it form a connected subtree of T;* (H2) *every sentence is hosted by a cluster of T;* (H3) *every row of Λ_T is implied by Λ (it holds for every distribution satisfying Λ), and every distribution that factorizes over the cliques of $G^*(T)$ and satisfies Λ_T satisfies Λ;* (H4) *the query atom a and the evidence atoms $\mathbf{E}$ lie in a common cluster. Then the optima of the CJT program are $\min_{p\in\mathcal{D}_{\mathrm{LCN}}} P_p(a)$ and $\max_{p\in\mathcal{D}_{\mathrm{LCN}}} P_p(a)$, and the infimum and supremum of its Charnes–Cooper form are $\inf$ and $\sup$ of $P_p(a \mid \mathbf{e})$ over $\{p \in \mathcal{D}_{\mathrm{LCN}} : p(\mathbf{e}) > 0\}$, the program being infeasible iff that set is empty. Without H1, H2 and the second half of H3, these values are still outer bounds.*

Proof. Let $\mathcal{G}$ be the cluster-feasible set (Definition 9), which is the feasible set of equation 23, and $\mathcal{H}$ the feasible set of the scaled program of Section C.

Step 1 (compactness). $\mathcal{D}_{\mathrm{LCN}}$ is a closed subset of the simplex (finitely many continuous equalities and non-strict inequalities) and hence compact, so $\min_{\mathcal{D}_{\mathrm{LCN}}} P_p(a)$ and $\max_{\mathcal{D}_{\mathrm{LCN}}} P_p(a)$ are attained ($\mathcal{D}_{\mathrm{LCN}} \neq \varnothing$). Likewise $\mathcal{G}$ is a closed subset of the compact $\prod_C \Delta(2^{|C|})$, so the linear ρ_a attains its minimum and maximum on $\mathcal{G}$ when $\mathcal{G} \neq \varnothing$ (Step 3 shows $\mathcal{G} \neq \varnothing$).

Step 2 (read-off identity). For a joint p and a formula φ with $\mathrm{atoms}(\varphi) \subseteq C$, $\langle A_\varphi^{(C)}, p \restriction C\rangle = P_p(\varphi)$ (Lemma 2, Type-1 case). Hence $\pi(p) \in \mathcal{G}^+$ iff $p(\mathbf{e}) > 0$, and then $\rho_a(\pi(p)) = P_p(a \wedge \mathbf{e})/P_p(\mathbf{e}) = P_p(a \mid \mathbf{e})$; for $\mathbf{E} = \varnothing$, $\rho_a(\pi(p)) = P_p(a)$.

Step 3 (projection; uses only H4 and the first half of H3). Let $p \in \mathcal{D}_{\mathrm{LCN}}$ and $q = \pi(p)$. Each q_C is a distribution. Each separator equality holds, because $q_C \restriction S$ and $q_{C'} \restriction S$ both equal $p \restriction S$ (vacuously for $S = \varnothing$). Each hosted sentence holds on its host C_h: by Lemma 2 its value depends on p only through $p \restriction \mathrm{scope}(c) = q_{C_h} \restriction \mathrm{scope}(c)$, and it holds for p. Each row of Λ_T holds on its host for the same reason, because by the first half of H3 it holds for every distribution satisfying Λ, in particular for p. Hence $\pi(\mathcal{D}_{\mathrm{LCN}}) \subseteq \mathcal{G}$, and by Step 2 $\pi(\mathcal{D}^+_{\mathrm{LCN}}) \subseteq \mathcal{G}^+$ with $\rho_a \circ \pi = P(a \mid \mathbf{e})$ there. So $\min_{\mathcal{G}} \rho_a \leq \min_{\mathcal{D}_{\mathrm{LCN}}} P(a)$ and $\max_{\mathcal{G}} \rho_a \geq \max_{\mathcal{D}_{\mathrm{LCN}}} P(a)$, and likewise $\inf_{\mathcal{G}^+} \rho_a \leq \inf_{\mathcal{D}^+_{\mathrm{LCN}}} P(a \mid \mathbf{e})$ and $\sup_{\mathcal{G}^+} \rho_a \geq \sup_{\mathcal{D}^+_{\mathrm{LCN}}} P(a \mid \mathbf{e})$.

Step 4 (reconstruction; uses H1, H2 and the second half of H3). Let $q \in \mathcal{G}$. By H1 and Lemma 6 there is a joint p with $p \restriction C = q_C$ for every cluster, factorizing over the cliques of $G^*(T)$. (4a) *Sub-scope consistency:* if $A \subseteq C$ then $p \restriction A = (p \restriction C) \restriction A = q_C \restriction A$, since marginalizing in stages agrees with marginalizing at once. In particular any two clusters containing $\mathbf{E}$ give the same $P(\mathbf{e})$, the marginal of p. (4b) *Sentences:* by H2 every sentence s is hosted on some C_h; q_{C_h} satisfies it, $p \restriction \mathrm{atoms}(s) = q_{C_h} \restriction \mathrm{atoms}(s)$ by (4a), so by Lemma 2 p satisfies s. (4c) *Imposed rows:* in the same way p satisfies every row of Λ_T. (4d) *LMC:* p factorizes over the cliques of $G^*(T)$ and satisfies Λ_T, so by the second half of H3 it satisfies Λ. (In CJT this half is supplied on a min-fill tree with nothing relaxed by Lemma 7 — each assertion is separator-entailed or its reduction is a row of Λ_T — and on an ordered tree by Proposition 4(c).) Hence $p \in \mathcal{D}_{\mathrm{LCN}}$ and $\pi(p) = q$: $\pi(\mathcal{D}_{\mathrm{LCN}}) = \mathcal{G}$. If $q \in \mathcal{G}^+$, then $p(\mathbf{e}) = \langle A_{\mathbf{e}}, q_{C_{\mathbf{e}}}\rangle > 0$ by (4a) and Step 2, so $p \in \mathcal{D}^+_{\mathrm{LCN}}$ and $\rho_a(q) = P_p(a \mid \mathbf{e})$: $\pi(\mathcal{D}^+_{\mathrm{LCN}}) = \mathcal{G}^+$.

Step 5 (marginal queries). By Steps 3–4, $\rho_a(\mathcal{G}) = \{P_p(a) : p \in \mathcal{D}_{\mathrm{LCN}}\}$, so the optima coincide, and are attained by Step 1. (Had $\mathcal{D}_{\mathrm{LCN}}$ been empty, Step 4 would give $\mathcal{G} = \varnothing$: the program is infeasible and CJT reports $[0, 1]$.)

Step 6 (conditional queries over $\mathcal{G}^+$). By Steps 3–4, $\rho_a(\mathcal{G}^+) = \{P_p(a \mid \mathbf{e}) : p \in \mathcal{D}^+_{\mathrm{LCN}}\}$, so the infima and the suprema coincide; they need not be attained, since $\mathcal{D}^+_{\mathrm{LCN}}$ need not be closed.

Step 7 (Charnes–Cooper). Define $\Phi(q) = (q/t, 1/t)$ on $\mathcal{G}^+$, with $t = \langle A_{\mathbf{e}}, q_{C_{\mathbf{e}}}\rangle \in (0, 1]$, and $\Psi(y, r) = y/r$ on $\mathcal{H}$. (7a) $\Phi(q) \in \mathcal{H}$: $y = q/t \geq 0$ and $r = 1/t > 0$; $\sum_i y_{C,i} = 1/t = r$ for every C; $\langle A_{\mathbf{e}}, y_{C_{\mathbf{e}}}\rangle = t/t = 1$; every scaled row of Section C is the original row multiplied by r (separator and sentence rows) or by r^2 (assertion rows, in conditional or marginal form), and $r > 0$, so it holds; and the valid inequalities hold, $r = 1/t \geq 1$ and $y_{C,i} = q_{C,i}/t \leq 1/t = r$. (7b) $\Psi(y, r) \in \mathcal{G}^+$: first $r = \sum_i y_{C_{\mathbf{e}},i} \geq \langle A_{\mathbf{e}}, y_{C_{\mathbf{e}}}\rangle = 1 > 0$, because $y \geq 0$ and $A_{\mathbf{e}} \in \{0, 1\}^{2^{|C_{\mathbf{e}}|}}$. Then $q_C = y_C/r \geq 0$ sums to 1; each scaled row divided by r or r^2 is the original row, so $q \in \mathcal{G}$ (the dropped bounds $q \leq 1$ follow from normalization and $q \geq 0$); and $\langle A_{\mathbf{e}}, q_{C_{\mathbf{e}}}\rangle = 1/r > 0$. (7c) $\Psi \circ \Phi = \mathrm{id}$ is immediate, and $\Phi(\Psi(y, r)) = (y, r)$ because $t = \langle A_{\mathbf{e}}, y_{C_{\mathbf{e}}}\rangle/r = 1/r$. So Φ is a bijection $\mathcal{G}^+ \to \mathcal{H}$. (7d) $\rho^{\mathrm{cc}}_a(\Phi(q)) = \langle A_{a\wedge\mathbf{e}}, q_{C_a}\rangle/t$, which equals $\rho_a(q)$ because $t = \langle A_{\mathbf{e}}, q_{C_a}\rangle$: both are $p(\mathbf{e})$ by (4a) under H1, and both are $p(\mathbf{e})$ for $q = \pi(p)$ in any case. Hence $\inf_{\mathcal{H}} \rho^{\mathrm{cc}}_a = \inf_{\mathcal{G}^+} \rho_a$ and $\sup_{\mathcal{H}} \rho^{\mathrm{cc}}_a = \sup_{\mathcal{G}^+} \rho_a$, which with Step 6 proves the conditional claim; and $\mathcal{H} = \varnothing$ iff $\mathcal{G}^+ = \varnothing$ iff $\mathcal{D}^+_{\mathrm{LCN}} = \varnothing$, in which case the program is infeasible and CJT reports $[0, 1]$, the value of Definition 7.

Step 8 (outer bound). Steps 2, 3, (7a) and (7d) for $q = \pi(p)$ use neither H1, H2 nor the second half of H3. They show that every value $P_p(a)$, $p \in \mathcal{D}_{\mathrm{LCN}}$ (resp. $P_p(a \mid \mathbf{e})$, $p \in \mathcal{D}^+_{\mathrm{LCN}}$), is attained by the program (resp. by its scaled form at $\Phi(\pi(p))$), so its optima are outer bounds. □

Remark 8 (Reading the proof)**.** *(a) The argument never mentions chains: exactness is a property of the* junction tree*, via RIP, hosting, separator-entailment and locality. It is in particular* not *a statement about singly-connected topologies — the worked example of Section C.1 is loopy and exact. (b) The projection half needs only that every imposed row follows from the LMC, which the triage guarantees by construction (relaxation included); so the global optimum of the* CJT *program is never unsound — its interval always contains the truth (as do SCIP's dual bounds) — and H1, H2 and the second half of H3 decide only tightness, for marginal and conditional queries alike. (c) H3 is* sufficient, not necessary*, and it is checked mechanically: the implementation records for each instance whether H1 and H3 hold, how many assertions were separator-entailed, reduced and hosted, implied by ordered rows, relaxed or dropped, and, per atom, whether the tree of its route (shared or augmented) satisfies H1–H3.*

Remark 9 (Why the conditional query is scaled)**.** *A natural alternative linearizes $P(a \mid \mathbf{e})$ by an auxiliary t with $t\,P(\mathbf{e}) = P(a \wedge \mathbf{e})$ and a floor $P(\mathbf{e}) \geq \varepsilon$ to exclude the degenerate $P(\mathbf{e}) = 0$. The floor removes the members of $\mathcal{D}_{\mathrm{LCN}}$ with $P(\mathbf{e}) < \varepsilon$, and in floating point it fails in a worse way: once ε is near the solver's feasibility tolerance, $t\,P(\mathbf{e}) = P(a \wedge \mathbf{e})$ holds within tolerance for every t. On the instance $0 \leq P(E) \leq 0.3$, $0.2 \leq P(A \mid E) \leq 0.4$, $0.6 \leq P(A \mid \neg E) \leq 0.8$ with $\varepsilon = 10^{-6}$, SCIP certifies $P(A \mid E) \in [0, 1]$, whereas the exact bound is $[0.2, 0.4]$. The scaled form equation 22 has neither defect and returns $[0.2, 0.4]$.*

D FAMILY-REALIZABLE SENTENCES

Which sentences can a *single* family's local set enforce? A family's degrees of freedom are its rows $\theta_{v,\pi} = P(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi)$; the parents' marginal $P(\mathrm{pa}_v)$ is not one of them — it is determined elsewhere in the network, and is a free variable during the local solve. A sentence that constrains a quantity depending on $P(\mathrm{pa}_v)$ therefore cannot be enforced locally, and the free product equation 11 may violate it. This is the mechanism behind Gap B, and the following definition isolates it. Throughout, a sentence s is *attached* to family v if $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$, and for a joint p and an assignment π of pa_v with $p(\pi) > 0$ we write $P_p(\mathrm{ch}_v \mid \pi)$ for the row that p induces.

Definition 10 (Family-realizable sentence)**.** *Let $\mathcal{L}$ be a chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$, and let s be a sentence with functional g_s. Say s is* family-realizable in family v *if s is attached to v and the value $g_s(p)$ is determined by the conditional table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$* alone*: for any two joints p, p' whose rows $P_p(\mathrm{ch}_v \mid \pi)$ and $P_{p'}(\mathrm{ch}_v \mid \pi)$ agree on every π with $p(\pi) > 0$ and $p'(\pi) > 0$, $g_s(p) = g_s(p')$ (for a Type-2 sentence, whenever $P(\psi) > 0$ and $P'(\psi) > 0$), whatever the parents' marginals $P(\mathrm{pa}_v)$ and $P'(\mathrm{pa}_v)$ may be. In particular, s is family-realizable in v if it has one of the two forms:*

(R1) Type-1 on a root block: *s is $\ell \leq P(\varphi) \leq u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$ and $\mathrm{pa}_v = \varnothing$. Then $P(\varphi)$ is a function of $P(\mathrm{ch}_v)$, which* is *the family's table.*

(R2) Type-2 pinning one full parent configuration: *s is $\ell \le P(\varphi \mid \psi) \le u$ with* $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$, $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, *and ψ satisfied by* exactly one *assignment $\pi^\star$ of* pa_v. *Then* $P(\varphi \mid \psi) = P(\varphi \mid \mathrm{pa}_v{=}\pi^\star)$ *is a single row of the table.*

A sentence is non-realizable *if it is family-realizable in no family. The LCN is* fully realizable *if every sentence is family-realizable in some family.*

R1 and R2 are the forms our problem generator emits. They are sufficient, not necessary: for example, $P(C \mid D)$ on a root block $\{C, D\}$, $P(C \mid D \wedge U)$ with $\mathrm{pa}_v = \{U\}$, and $P(C \wedge U \mid U)$ are functions of one row but have neither form. The next lemma characterizes realizability exactly, so it can be decided syntactically. For an assignment π of pa_v write φ_π for the formula over ch_v obtained by fixing $\mathrm{pa}_v = \pi$ in φ (and likewise ψ_π), $w_\pi = P(\mathrm{pa}_v{=}\pi)$ for the parents' marginal, and, for a row $\theta_{v,\pi}$ (a distribution over the states c of ch_v) and a formula χ over ch_v, $\theta_{v,\pi}(\chi) = \sum_{c \models \chi} \theta_{v,\pi}(c)$.

Lemma 8 (Characterization)**.** *Let s be a sentence attached to family v.*

(i) *If s is Type-1, $\ell \le P(\varphi) \le u$, then s is family-realizable in v iff* $\mathrm{pa}_v = \varnothing$ *or φ is constant (a tautology or a contradiction) on* $\mathrm{ch}_v \cup \mathrm{pa}_v$.

(ii) *If s is Type-2, $\ell \le P(\varphi \mid \psi) \le u$, let $\Pi_\psi = \{\pi : \psi_\pi$ is satisfiable$\}$. Then s is family-realizable in v iff $|\Pi_\psi| \le 1$, or the conditional value is forced: $\psi_\pi \models \varphi_\pi$ for every $\pi \in \Pi_\psi$ (value 1), or $\psi_\pi \models \neg\varphi_\pi$ for every $\pi \in \Pi_\psi$ (value 0).*

In particular, (R1) and (R2) are realizable, and every realizable sentence is a function of a single row of the table, is constant, or (if $\Pi_\psi = \varnothing$) is vacuous.

Proof. Step 1 (decomposition). Let p be a joint on $\mathrm{ch}_v \cup \mathrm{pa}_v$ (by Lemma 2 the value of an attached sentence depends on p only through this marginal), with weights $w_\pi = p(\pi)$ and rows $\theta_{v,\pi} = P_p(\mathrm{ch}_v \mid \pi)$ where $w_\pi > 0$ (any distribution where $w_\pi = 0$). Then $p(c, \pi) = w_\pi\, \theta_{v,\pi}(c)$, so for every formula χ over the scope

$$P(\chi) = \sum_\pi w_\pi\, \theta_{v,\pi}(\chi_\pi). \tag{25}$$

Step 2 (free choice). Conversely, every weight vector w in the simplex over the assignments of pa_v and every choice of rows define, by $p(c, \pi) = w_\pi \theta_{v,\pi}(c)$, a joint with these weights, whose rows are the chosen ones wherever $w_\pi > 0$. Weights and rows can thus be chosen independently. *Moving w from π_1 to π_2* means: fix the rows, and compare the joints p^1, p^2 with weights $w^i = (1-\epsilon)\delta_{\pi_i} + \epsilon\,\nu$, where ν is the uniform distribution over the assignments of pa_v and $0 < \epsilon < 1$. Both have full support and the same rows, so a realizable s must have $g_s(p^1) = g_s(p^2)$. The values in question are affine in w (in (i)) or linear-fractional in w with a denominator bounded away from 0 (in (ii), where the rows below have $\theta_{v,\pi_i}(\psi_{\pi_i}) > 0$), hence continuous in ϵ; so if they differ at $\epsilon = 0$ (the point masses $\delta_{\pi_1}, \delta_{\pi_2}$) they differ for some small $\epsilon > 0$, and s is not realizable.

Step 3 ((i), "if"). If $\mathrm{pa}_v = \varnothing$ there is a single, empty configuration, positive under every joint, and $P(\varphi) = \theta_{v,\varnothing}(\varphi)$ by equation 25; two joints with the same row agree. If φ is constant on $\mathrm{ch}_v \cup \mathrm{pa}_v$, $P(\varphi) \in \{0, 1\}$ is the same for every joint.

Step 4 ((i), "only if"). Suppose $\mathrm{pa}_v \neq \varnothing$ and φ is not constant. There are at least two configurations. If some φ_{π_1} is non-constant on ch_v, take any $\pi_2 \neq \pi_1$ and fix the row at π_2; the row at π_1 can give $\theta_{v,\pi_1}(\varphi_{\pi_1})$ the value 0 and the value 1, so one of them differs from $\theta_{v,\pi_2}(\varphi_{\pi_2})$. Otherwise every φ_π is constant, and since φ is not, two configurations have $\varphi_{\pi_1} \equiv \top$ and $\varphi_{\pi_2} \equiv \bot$. In both cases some rows have $\theta_{v,\pi_1}(\varphi_{\pi_1}) \neq \theta_{v,\pi_2}(\varphi_{\pi_2})$, i.e. different values of $P(\varphi)$ at the point masses; by Step 2 s is not realizable.

Step 5 ((ii), mixture form). By equation 25, the terms with $\pi \notin \Pi_\psi$ vanish in $P(\psi)$ and $P(\varphi \wedge \psi)$, so on $\{P(\psi) > 0\}$

$$P(\varphi \mid \psi) = \sum_{\pi \in \Pi_\psi} \lambda_\pi\, R_\pi, \qquad R_\pi = \frac{\theta_{v,\pi}(\varphi_\pi \wedge \psi_\pi)}{\theta_{v,\pi}(\psi_\pi)}, \qquad \lambda_\pi = \frac{w_\pi\, \theta_{v,\pi}(\psi_\pi)}{\sum_{\pi' \in \Pi_\psi} w_{\pi'}\, \theta_{v,\pi'}(\psi_{\pi'})}, \tag{26}$$

where the sum runs over the π with $w_\pi \theta_{v,\pi}(\psi_\pi) > 0$ (at least one, since $P(\psi) > 0$).

Step 6 ((ii), "if"). If $|\Pi_\psi| = 0$, ψ is unsatisfiable on the scope, $P(\psi) = 0$ for every joint, and the condition of Definition 10 is vacuous: s is (vacuously) realizable, and its cleared rows are $0 \geq 0$. If $\Pi_\psi = \{\pi^\star\}$, then $P(\psi) > 0$ forces $w_{\pi^\star} > 0$, so $\pi^\star$ is positive under both joints, their rows at $\pi^\star$ agree, and the value $R_{\pi^\star}$ is the same. If the value is forced, every R_π in equation 26 equals the same constant (1 or 0) for every row, so the value is that constant.

Step 7 ((ii), "only if"). Suppose $|\Pi_\psi| \geq 2$ and the value is not forced. If some R_{π_1} is not constant over rows (neither $\psi_{\pi_1} \models \varphi_{\pi_1}$ nor $\psi_{\pi_1} \models \neg\varphi_{\pi_1}$), take $\pi_2 \in \Pi_\psi \setminus \{\pi_1\}$ and fix a row at π_2 with $\theta_{v,\pi_2}(\psi_{\pi_2}) > 0$. The row at π_1 can still give R_{π_1} two different values, with $\theta_{v,\pi_1}(\psi_{\pi_1}) > 0$, one of which differs from R_{π_2}. Otherwise every R_π is 0 or 1, and since the value is not forced, two configurations have $R_{\pi_1} = 1 \neq 0 = R_{\pi_2}$. Either way the point masses $\delta_{\pi_1}, \delta_{\pi_2}$ give the values $R_{\pi_1} \neq R_{\pi_2}$, with positive denominators, so by Step 2 s is not realizable.

Step 8 (R1, R2). For (R1), $\mathrm{pa}_v = \varnothing$ and (i) applies. For (R2), $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$ makes every ψ_π constant, so $|\Pi_\psi|$ is the number of parent assignments satisfying ψ, which is one, and (ii) applies. □

Our implementation decides realizability by exactly this test (`realizability.realizable_in_family`); a randomized check against numerical $P(\mathrm{pa}_v)$-invariance agrees on 150 random families and sentences.

D.1 A FIVE-VARIABLE CHAIN-GRAPH EXAMPLE

We next give an example of family-realizability on a five-atom chain graph.

Example 5. *Consider the following family-realizable k-tree LCN with n=5 atoms and k=3:*

$s_0 : 0.01 \leq P(x_3) \leq 0.399975$
$s_1 : 0.01 \leq P(x_4 \mid x_3) \leq 0.442867$
$s_2 : 0.638553 \leq P(x_2 \mid \neg x_3 \wedge x_4) \leq 1.0$
$s_3 : 0.01 \leq P(\neg x_1 \mid x_3 \wedge x_4 \wedge x_2) \leq 0.307066$
$s_4 : 0.673756 \leq P(\neg x_0 \mid x_3 \wedge x_4 \wedge \neg x_1) \leq 1.0$
$s_5 : 0.748845 \leq P(\neg x_3) \leq 0.954666$
$s_6 : 0.141404 \leq P(x_3) \leq 0.247704$

The chain graph has five singleton-block families and there is a single LMC assertion:

$$x_3 \mid \varnothing, \quad x_4 \mid x_3, \quad x_2 \mid x_3, x_4, \quad x_1 \mid x_2, x_3, x_4, \quad x_0 \mid x_1, x_3, x_4; \qquad (x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4).$$

Table 6 classifies every sentence. Two points are worth drawing out. First, the attached *column (what the scope-subset test collects) is much larger than the* realizable-in *column: the root marginals s_0, s_5, s_6 are attached to all five families but realizable only in the root family x_3, which is exactly why Definition 10 quantifies existentially over families. Second, each of s_1, s_2, s_3, s_4 conditions on a* full *conjunction of signed literals over all of its family's parents, so $|\Pi_\psi| = 1$ and (R2) applies — this is what the random problem generator guarantees for every family-realizable problem class.*

Table 6: Sentence classification. Every sentence is family-realizable in exactly one family, so the instance is fully realizable.

sentence	form	attached to families	realizable in	why
s_0, s_5, s_6	Type-1 $P(x_3)$, $P(\neg x_3)$	all five	$x_3 \mid \varnothing$	(R1) root block
s_1	Type-2, $\psi = x_3$	x_4, x_2, x_1, x_0	$x_4 \mid x_3$	(R2) $\lvert\Pi_\psi\rvert$=1
s_2	Type-2, $\psi = \neg x_3 \wedge x_4$	x_2, x_1	$x_2 \mid x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_3	Type-2, $\psi = x_3 \wedge x_4 \wedge x_2$	x_1	$x_1 \mid x_2, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_4	Type-2, $\psi = x_3 \wedge x_4 \wedge \neg x_1$	x_0	$x_0 \mid x_1, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1

Measured bounds. *The exact method with a global nonlinear solver (e.g., SCIP) certifies all ten solves (gap 0), and the new `CJT` reproduces every endpoint:*

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$	$P(x_4)$
`Exact` *(global)*	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$
`CJT` *(SCIP)*	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$

The vacuous $[0,1]$ *on* x_0, x_1, x_2 *is genuine, not a solver artifact. For* x_1*, for instance: the row* $P(x_2 \mid x_3 \wedge x_4)$ *is free (the only sentence on* x_2*'s family,* s_2*, conditions on* $\neg x_3 \wedge x_4$*), and setting it to* 0 *gives* $P(x_3 \wedge x_4 \wedge x_2) = 0$*, so* s_3*'s conditioning event is unreachable and every row of* x_1 *with positive weight is unconstrained; choosing them all equal to* $\delta_{x_1=0}$ *or all to* $\delta_{x_1=1}$ *gives* $P(x_1) = 0$ *and* $P(x_1) = 1$*. (Conversely, while* $P(x_3 \wedge x_4 \wedge x_2) > 0$*,* s_3 *keeps* $P(x_1)$ *away from* 0 *and* 1*, so the endpoints require the unreachable condition.) The junction tree has maximal cliques* $\{x_0, x_1, x_3, x_4\}$ *and* $\{x_1, x_2, x_3, x_4\}$ *— treewidth* 3 *— and the separator between them is exactly* $\{x_1, x_3, x_4\}$*, the conditioning set of the lone LMC assertion, so that assertion is separator-entailed (its conditioning set separates* x_0 *from* x_2 *in* $G^*(T)$*). The tree satisfies running intersection (H1), hosts every sentence (H2) and discharges its only LMC assertion without relaxation (H3), and every atom shares a cluster with the (empty) evidence (H4), so Theorem 1 applies.*

E THE LCN SET VERSUS THE STRONG EXTENSION

We now relate $\mathcal{D}_{\mathrm{LCN}}$ and $\mathcal{D}_{\mathrm{SE}}$. Call a chain-graph LCN *factorizing* if every $p \in \mathcal{D}_{\mathrm{LCN}}$ is a product of family tables along the component DAG, $p = \prod_v \theta_v$ as in equation 10. Every DAG-structured LCN is factorizing (Lemma 4). For general chain graphs this is guaranteed only for the strictly positive members of $\mathcal{D}_{\mathrm{LCN}}$ (Cozman, 2023): the atom-wise LMC assertions $(C \perp\!\!\!\perp \mathbf{Y} \mid D, \mathbf{Z})$, $(D \perp\!\!\!\perp \mathbf{Y} \mid C, \mathbf{Z})$ of a component $\{C, D\}$ give the component-wise $(\{C, D\} \perp\!\!\!\perp \mathbf{Y} \mid \mathbf{Z})$ only through the intersection property, which may fail when p has zeros (Section E.4); Remark 2 gives the weaker statement that holds without factorization.

E.1 LOCAL CREDAL SETS

Definition 11 (Local credal set)**.** *Fix a node* v*, the set* $\mathcal{S}_v$ *of sentences attached to* v*, and an assignment* π *of* pa_v*. The* local program *at* (v, π) *has the feasible set*

$$F_{v,\pi} = \big\{\rho \in \Delta(2^{\mathrm{ch}_v \cup \mathrm{pa}_v}) : \; c_s^-(\rho) \geq 0, \; c_s^+(\rho) \geq 0 \;\; \forall s \in \mathcal{S}_v, \quad \rho(\mathrm{pa}_v{=}\pi) > 0\big\},$$

where $c_s^{\pm}$ *are the cleared rows of* s *(Section 2); it is* feasible *if* $F_{v,\pi} \neq \varnothing$*. No LMC equality is imposed: neither the cross-family ones nor those whose atoms lie inside the family scope. The* local credal set *is*

$$K_{v,\pi} = \big\{\rho(\mathrm{ch}_v \mid \pi) \;:\; \rho \in F_{v,\pi}\big\} \;\; \textit{if the program is feasible,} \qquad K_{v,\pi} = \Delta(2^{\mathrm{ch}_v}) \;\; \textit{otherwise,}$$

and $K_v = \prod_\pi K_{v,\pi}$*. For a root (*$\mathrm{pa}_v = \varnothing$*) there is one, empty, configuration and* $K_v = F_{v,\varnothing}$*. For comparison, the set of rows actually realized by the LCN is* $K_{v,\pi}^{\mathrm{true}} = \{P_p(\mathrm{ch}_v \mid \pi) : \; p \in \mathcal{D}_{\mathrm{LCN}}, \; p(\pi) > 0\}$.

The simplex for an infeasible program is a *convention*: the set of attained rows is then empty. It is sound for the outer direction, because no $p \in \mathcal{D}_{\mathrm{LCN}}$ gives π positive probability (otherwise p's marginal on the family scope would lie in $F_{v,\pi}$; Step 2 of the proof of Proposition 5), so the row of p at π is irrelevant and any row may be used. It is not exact: the free product may then put mass on π with an arbitrary row (Section E.4).

Properties and computation.

1. $K_{v,\pi}$ *is a polytope.* If the program is infeasible this holds by definition. Otherwise substitute $y = \rho/\rho(\pi)$, $t = 1/\rho(\pi)$ (Charnes–Cooper). The Type-1 rows $\langle A_\varphi, \rho\rangle \geq \ell$ (with $\sum \rho = 1$) become $\langle A_\varphi, y\rangle \geq \ell t$ and $\sum y = t$; the Type-2 rows are homogeneous and unchanged; and $\rho(\pi) > 0$ becomes $\sum_{\mathrm{pa}_v = \pi} y = 1$. Let Y be the resulting polyhedron in $(y, t) \geq 0$. Every $(y, t) \in Y$ has $t = \sum y \geq \sum_{\mathrm{pa}_v = \pi} y = 1$, so $\rho = y/t$ is a distribution with $\rho(\pi) = 1/t > 0$ satisfying the attached sentences, i.e. $\rho \in F_{v,\pi}$, with row $y(\cdot, \pi)$; conversely $\rho \in F_{v,\pi}$ gives $(\rho/\rho(\pi), 1/\rho(\pi)) \in Y$. Hence $K_{v,\pi}$ is the linear projection of Y onto the coordinates $y(\cdot, \pi)$, a polyhedron (Fourier–Motzkin elimination) contained in the simplex, hence a polytope, and in particular closed and non-empty.
2. *Single-atom child.* The row is $(1 - x, x)$ and $K_{v,\pi}$ is the interval between the minimum and the maximum of x over Y, two LPs after the Charnes–Cooper substitution (solved by HiGHS; a non-clean HiGHS result falls back to a local solver, which suffices because local

optima of linear and linear-fractional programs are global, and a missing bound is replaced by 0 or 1, which only enlarges the set). This is exact up to the LP tolerance.

3. *Compound child.* The per-state interval box $\{r \in \Delta(2^{\mathrm{ch}_v}) : \underline{r}_c \le r(c) \le \overline{r}_c\}$ contains $K_{v,\pi}$ and can be strictly larger (Section E.4). The compiler computes the vertices of $K_{v,\pi}$ itself (`row_polytopes.row_polytope_vertices`) by the convex-hull method with an LP oracle over Y: it maximizes coordinate and random directions, then every facet normal of the current hull, until no facet moves by more than 10^{-9}. The result is exact up to that tolerance and the LP tolerances (in practice within about 10^{-7}, on the inner side). If the oracle fails or the round cap is reached, the box is kept, which is outer; such rows are reported (`CredalNetwork.inexact_rows`, and `exact_local_sets` in `realizability_report`).
4. *Infeasibility* is detected by a phase-1 LP on Y (`locally_feasible`); the row is stored as vacuous and listed in `CredalNetwork.infeasible_rows`. If HiGHS is unavailable or the check fails, a warning is issued and the row is treated as feasible; the results below assume the check succeeded.

The paper covers the default compilation (`method="linear"`, `merge_budget=1`: one family per chain component, exactly as in Definition 11). The optional `linear-tight` variant adds the in-scope LMC equalities and solves the resulting bilinear local program with a local NLP solver, which can return inner intervals; it is not covered by the results below.

Lemma 9 (Family marginals of products)**.** *Let $q = \prod_v \theta_v$, i.e. $q(\mathbf{x}) = \prod_v \theta_{v,\mathbf{x}_{\mathrm{pa}_v}}(\mathbf{x}_{\mathrm{ch}_v})$, be a product of family tables along the component DAG D, each row $\theta_{v,\pi}$ a distribution over the states of ch_v. Then for every node v:*

(i) $q(\mathrm{ch}_v{=}c, \mathrm{pa}_v{=}\pi) = q(\mathrm{pa}_v{=}\pi)\,\theta_{v,\pi}(c)$ *for all c, π;*

(ii) *if $q(\mathrm{pa}_v{=}\pi) = 0$, replacing $\theta_{v,\pi}$ by any distribution leaves q unchanged.*

Proof. Step 1 (ancestral sets). Call a set B of nodes *ancestral* if it contains the parents $\mathrm{Pa}(u)$ of each of its members u. For ancestral B, the marginal of q on the atoms of B is $\prod_{u\in B}\theta_u$. Indeed, sum out the nodes outside B one at a time in reverse topological order. When node $w \notin B$ is summed out, no remaining node is a child of w: a child in B would make $w \in B$, and a child outside B comes later in the topological order and has been removed already. So $\mathbf{x}_{\mathrm{ch}_w}$ appears only in θ_w, and $\sum_c \theta_{w,\pi}(c) = 1$ removes the factor.

Step 2. Let A be the set of proper ancestors of v in D. Both A and $B = A \cup \{v\}$ are ancestral, pa_v consists of atoms of A, and the atoms ch_v belong to no node of A (components are disjoint and D is acyclic). By Step 1, $q(\mathbf{x}_B) = \big(\prod_{u\in A}\theta_u\big)\,\theta_{v,\mathbf{x}_{\mathrm{pa}_v}}(\mathbf{x}_{\mathrm{ch}_v}) = q(\mathbf{x}_A)\,\theta_{v,\mathbf{x}_{\mathrm{pa}_v}}(\mathbf{x}_{\mathrm{ch}_v})$.

Step 3 ((i)). Sum Step 2 over the atoms of A outside pa_v, with $\mathbf{x}_{\mathrm{pa}_v} = \pi$ and $\mathbf{x}_{\mathrm{ch}_v} = c$: $q(c,\pi) = q(\mathrm{pa}_v{=}\pi)\,\theta_{v,\pi}(c)$.

Step 4 ((ii)). Let q' be the product with $\theta_{v,\pi}$ replaced. The factors of A do not involve θ_v, so by Step 1 $q'(\mathbf{x}_A) = q(\mathbf{x}_A)$ and $q'(\mathrm{pa}_v{=}\pi) = q(\mathrm{pa}_v{=}\pi) = 0$. For $\mathbf{x}$ with $\mathbf{x}_{\mathrm{pa}_v} \neq \pi$ the two products have the same factors, so $q'(\mathbf{x}) = q(\mathbf{x})$; for $\mathbf{x}$ with $\mathbf{x}_{\mathrm{pa}_v} = \pi$, $0 \le q'(\mathbf{x}) \le q'(\mathrm{pa}_v{=}\pi) = 0$ and likewise $q(\mathbf{x}) = 0$. No division and no positivity is used. □

E.2 Inclusion

Proposition 5 (Inclusion)**.** *Let $\mathcal{L}$ be a factorizing chain-graph LCN, compiled as in Definition 11 (with exact local sets or box fallbacks). Then $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{SE}}$, and $K^{\mathrm{true}}_{v,\pi} \subseteq K_{v,\pi}$ for every v, π (this part needs no factorization). Consequently, for every marginal query $[\underline{P}_{\mathrm{LCN}}(a), \overline{P}_{\mathrm{LCN}}(a)] \subseteq [\underline{P}_{\mathrm{SE}}(a), \overline{P}_{\mathrm{SE}}(a)]$, and for every conditional query $P(a \mid \mathbf{e})$ (Definition 7) with $\overline{P}_{\mathrm{LCN}}(\mathbf{e}) > 0$ the interval over $\mathcal{D}_{\mathrm{LCN}}$ is contained in the interval over $\mathcal{D}_{\mathrm{SE}}$. Hence any engine that is exact or outer w.r.t. $\mathcal{D}_{\mathrm{SE}}$ is a valid* outer *bound w.r.t. $\mathcal{D}_{\mathrm{LCN}}$ for these queries.*

Proof. If $\mathcal{D}_{\mathrm{LCN}} = \varnothing$ there is nothing to prove. Let $p \in \mathcal{D}_{\mathrm{LCN}}$.

Step 1 (factorization). Since $\mathcal{L}$ is factorizing, $p = \prod_v \theta_v$ for some tables. By Lemma 9(i), $\theta_{v,\pi} = P_p(\mathrm{ch}_v \mid \pi)$ whenever $p(\pi) > 0$.

Step 2 (rows of positive weight). Fix v and π with $p(\pi) > 0$, and let ρ be the marginal of p on $\mathrm{ch}_v \cup \mathrm{pa}_v$. Every $s \in \mathcal{S}_v$ holds at p, and by Lemma 2 its cleared rows depend on p only through ρ, so $c_s^{\pm}(\rho) = c_s^{\pm}(p) \geq 0$; and $\rho(\pi) = p(\pi) > 0$. Hence $\rho \in F_{v,\pi}$, the program is feasible, and $\theta_{v,\pi} = \rho(\mathrm{ch}_v \mid \pi) \in K_{v,\pi}$. (A box fallback contains $K_{v,\pi}$, so the conclusion holds for it as well.) This step uses only $p \in \mathcal{D}_{\mathrm{LCN}}$ and $p(\pi) > 0$, which proves $K_{v,\pi}^{\mathrm{true}} \subseteq K_{v,\pi}$.

Step 3 (rows of zero weight). For each (v, π) with $p(\pi) = 0$, replace $\theta_{v,\pi}$ by an element of $K_{v,\pi}$, which is non-empty (property 1 if the program is feasible; the simplex otherwise). By Lemma 9(ii) the product is still p, so the replacements can be made one at a time and every weight $p(\pi)$ is unchanged.

Step 4. Now $p = \prod_v \theta_v$ with every row in its local set, i.e. $p \in \mathcal{P}_{\mathrm{SE}}$ equation 11; and $\mathcal{P}_{\mathrm{SE}} \subseteq \mathrm{CH}(\mathcal{P}_{\mathrm{SE}}) = \mathcal{D}_{\mathrm{SE}}$.

Step 5 (bounds). A marginal $P(a)$ is minimized and maximized over a larger set, which gives a wider interval. For $P(a \mid \mathbf{e})$ with $\overline{P}_{\mathrm{LCN}}(\mathbf{e}) > 0$, the inclusion gives $\mathcal{D}_{\mathrm{LCN}} \cap \{p(\mathbf{e}) > 0\} \subseteq \mathcal{D}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\} \neq \varnothing$, so the infimum (supremum) over $\mathcal{D}_{\mathrm{SE}}$ is at most (at least) the one over $\mathcal{D}_{\mathrm{LCN}}$. If $\overline{P}_{\mathrm{LCN}}(\mathbf{e}) = 0$ the LCN answer is $[0, 1]$ by convention, and a strong-extension interval need not contain it; such queries are excluded. □

When $\mathcal{L}$ is not known to be factorizing (compound components), Steps 2–4 apply to every strictly positive member of $\mathcal{D}_{\mathrm{LCN}}$, which factorizes (Cozman, 2023), and give the weaker inclusion of Remark 2.

E.3 Sufficiency

The sufficiency result needs every local row to enforce the sentences realizable at it ("no Gap A"). Local feasibility makes this hold.

Definition 12 (Local feasibility)**.** *A chain-graph LCN is* locally feasible *if, for every family v and parent configuration π, the local program of Definition 11 is feasible: some joint on the family scope $\mathrm{ch}_v \cup \mathrm{pa}_v$ satisfies the attached sentences and gives $\mathrm{pa}_v{=}\pi$ positive probability.*

Local feasibility fails, for example, when two sentences on the same row contradict each other. $\mathcal{D}_{\mathrm{LCN}}$ then forces $P(\mathrm{pa}_v{=}\pi) = 0$, a constraint on the parents that no parent's local set sees, and the convention $K_{v,\pi} = \Delta(2^{\mathrm{ch}_v})$ lets the free product put mass on π (Section E.4). It is not needed for non-emptiness of the local sets, which holds by the convention; it is used only to exclude that convention. It is also stronger than necessary (e.g. an infeasible row is harmless when the parents' local sets already force $q(\pi) = 0$ in every product), but it is what the compiler checks.

Row-wise form of an attached sentence. Let s be attached to v, and let p be any joint whose family marginal has the form $p(\mathrm{ch}_v{=}c, \mathrm{pa}_v{=}\pi) = w_\pi\, \theta_{v,\pi}(c)$ with w a distribution over the assignments of pa_v and rows $\theta_{v,\pi}$ (by Lemma 9, every product of family tables has this form, with $w = q(\mathrm{pa}_v)$). Define Π_s, $L_{s,\pi}^{\pm}$ and constants $\kappa_s^{\pm}$ as follows.

- Type-1, φ constant on the scope with value $c_s \in \{0, 1\}$: $\Pi_s = \varnothing$, $\kappa_s^- = c_s - \ell_s$, $\kappa_s^+ = u_s - c_s$.
- Type-1, φ not constant: Π_s is the set of all assignments of pa_v ($\{\varnothing\}$ for a root), $\kappa_s^{\pm} = 0$, $L_{s,\pi}^-(\theta) = \theta(\varphi_\pi) - \ell_s$, $L_{s,\pi}^+(\theta) = u_s - \theta(\varphi_\pi)$.
- Type-2: $\Pi_s = \Pi_\psi$ (possibly empty), $\kappa_s^{\pm} = 0$, $L_{s,\pi}^-(\theta) = \theta(\varphi_\pi \wedge \psi_\pi) - \ell_s\, \theta(\psi_\pi)$ and $L_{s,\pi}^+(\theta) = u_s\, \theta(\psi_\pi) - \theta(\varphi_\pi \wedge \psi_\pi)$.

Then, by equation 25 (using $\sum_\pi w_\pi = 1$ for Type-1, and that the terms with $\pi \notin \Pi_\psi$ vanish for Type-2),

$$c_s^{\pm}(p) \;=\; \kappa_s^{\pm} + \sum_{\pi \in \Pi_s} w_\pi\, L_{s,\pi}^{\pm}\big(\theta_{v,\pi}\big), \tag{27}$$

where each $L_{s,\pi}^{\pm}$ is affine in the row at π alone. This holds for every attached sentence. Realizability (Lemma 8) restricts the shape: a realizable non-constant Type-1 sentence has $|\Pi_s| = 1$ (a root), and a realizable Type-2 sentence has $|\Pi_s| \leq 1$ or a forced value $c \in \{0, 1\}$, in which case $\theta(\varphi_\pi \wedge \psi_\pi) = c\, \theta(\psi_\pi)$ for every row, so $L_{s,\pi}^- = (c - \ell_s)\, \theta(\psi_\pi)$ and $L_{s,\pi}^+ = (u_s - c)\, \theta(\psi_\pi)$.

Lemma 10 (Row-wise enforcement)**.** *Let s be family-realizable in family v, let $\pi \in \Pi_s$, and suppose the local program at (v, π) is feasible and $K_{v,\pi}$ is the exact set of Definition 11 (not a box fallback). Then every row $r \in K_{v,\pi}$ satisfies $L^{\pm}_{s,\pi}(r) \geq 0$.*

Proof. Step 1 (rows are attained). Since the program is feasible and $K_{v,\pi}$ is exact, $r = \rho(\mathrm{ch}_v \mid \pi)$ for some $\rho \in F_{v,\pi}$ (Definition 11).

Step 2 (the attained joint satisfies s). s is attached to v, so $c^{\pm}_s(\rho) \geq 0$. By Step 1 of the proof of Lemma 8, ρ has the form required by equation 27 with $w_{\pi'} = \rho(\pi')$ and rows $\rho(\mathrm{ch}_v \mid \pi')$ (arbitrary where $\rho(\pi') = 0$), and $\kappa^{\pm}_s = 0$ because $\Pi_s \neq \varnothing$. Hence $\sum_{\pi' \in \Pi_s} \rho(\pi')\, L^{\pm}_{s,\pi'} \geq 0$.

Step 3 (cases). If $\Pi_s = \{\pi\}$, the sum is the single term $\rho(\pi) L^{\pm}_{s,\pi}(r)$ with $\rho(\pi) > 0$, so $L^{\pm}_{s,\pi}(r) \geq 0$. Otherwise $|\Pi_s| \geq 2$, so by Lemma 8 s is Type-2 with a forced value c, and $L^{-}_{s,\pi'} = (c - \ell_s)\theta(\psi_{\pi'})$, $L^{+}_{s,\pi'} = (u_s - c)\theta(\psi_{\pi'})$. If $c \in [\ell_s, u_s]$, both are non-negative for every row. If $c < \ell_s$, every term $\rho(\pi')(c - \ell_s)\rho(\psi_{\pi'} \mid \pi')$ of the sum for L^- is ≤ 0 and the sum is ≥ 0, so every term vanishes; at π, $\rho(\pi) > 0$ gives $r(\psi_\pi) = 0$, whence $L^{-}_{s,\pi}(r) = L^{+}_{s,\pi}(r) = 0$. The case $c > u_s$ is symmetric (use the sum for L^+). □

Without feasibility the lemma fails: $K_{v,\pi}$ is then the whole simplex, whose rows need not satisfy $L^{\pm}_{s,\pi} \geq 0$. With a box fallback it can fail too, since the box contains rows that are not attained.

Proposition 6 (Sufficiency)**.** *Let $\mathcal{L}$ be a chain-graph LCN, compiled with exact local sets (Definition 11; no inexact rows), that is consistent, locally feasible (Definition 12), factorizing, and* product-Markov*: every product $\prod_v \theta_v$ of family tables along the component DAG, with arbitrary rows, satisfies every LMC assertion of $\mathcal{L}$ (Definition 6). (A DAG-structured LCN is factorizing and product-Markov, Lemma 4.) If every sentence is family-realizable in some family (Definition 10), then $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, and $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same lower and upper probabilities for every marginal and every conditional query $P(a \mid \mathbf{e})$ (Definition 7).* Single-connectedness of the primal graph is not required.

Proof. Step 1 ($\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{P}_{\mathrm{SE}}$). Proposition 5, since $\mathcal{L}$ is factorizing.

Step 2 (LMC at a product). Let $q = \prod_v \theta_v$ with every $\theta_{v,\pi} \in K_{v,\pi}$ be an arbitrary element of $\mathcal{P}_{\mathrm{SE}}$. It is a product of family tables along D, so $h_\iota(q) = 0$ for every assertion ι because $\mathcal{L}$ is product-Markov. Loops of the primal graph play no role here (Example 6).

Step 3 (sentences at a product). Let s be a sentence, family-realizable in some family v (full realizability). By Lemma 2 $c^{\pm}_s(q)$ depends on q only through its marginal on $\mathrm{ch}_v \cup \mathrm{pa}_v$, which by Lemma 9(i) is $q(\mathrm{pa}_v{=}\pi)\, \theta_{v,\pi}(c)$; so equation 27 applies with $w = q(\mathrm{pa}_v)$. Three cases. (a) s is a constant Type-1 sentence: $c^{\pm}_s(q) = \kappa^{\pm}_s$ does not depend on q. By consistency some $p \in \mathcal{D}_{\mathrm{LCN}}$ exists, and $c^{\pm}_s(p) = \kappa^{\pm}_s \geq 0$. This is the only use of consistency. (b) $\Pi_s = \varnothing$ otherwise, i.e. Type-2 with $|\Pi_\psi| = 0$: $c^{\pm}_s(q) = 0$, and the sentence is vacuous. (c) $\Pi_s \neq \varnothing$: $\kappa^{\pm}_s = 0$, every (v, π) is feasible by local feasibility, so by Lemma 10 every $L^{\pm}_{s,\pi}(\theta_{v,\pi}) \geq 0$, and $q(\pi) \geq 0$; hence $c^{\pm}_s(q) \geq 0$. So q satisfies every sentence and every assertion: $q \in \mathcal{D}_{\mathrm{LCN}}$, and $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$.

Step 4 (bounds). $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{SE}}) = \mathrm{CH}(\mathcal{D}_{\mathrm{LCN}})$, and $\mathcal{P}_{\mathrm{SE}}$ is compact (Lemma 3). For a marginal, Lemma 3 gives equal minima and maxima over $\mathcal{D}_{\mathrm{LCN}}$ and $\mathcal{D}_{\mathrm{SE}}$. For $P(a \mid \mathbf{e})$, $p(\mathbf{e})$ is linear, so $\sup_{\mathcal{D}_{\mathrm{SE}}} p(\mathbf{e}) = \sup_{\mathcal{D}_{\mathrm{LCN}}} p(\mathbf{e})$: either both are 0 and both answers are $[0, 1]$, or both are positive and the conditional part of Lemma 3 gives equal infima and suprema over the members with $p(\mathbf{e}) > 0$. No hypothesis $\underline{P}(\mathbf{e}) > 0$ is needed. □

The local sets are computed up to the oracle tolerance (property 3 above), so the equality holds up to that tolerance. The product-Markov, locally feasible and fully realizable hypotheses, the exact local sets, and (for Proposition 5) factorization are each needed: Section E.4 gives an example for each.

Testing the hypotheses. `realizability.realizability_report` returns `fully_realizable` (by Lemma 8), `locally_feasible` and `infeasible_rows`, `exact_local_sets` and `inexact_rows`, `dag_structured`, and `product_markov`; factorization is not tested. The product-Markov test (`lmc_factorized`) returns *true* without testing when every component is a single atom (justified by Lemma 4). Otherwise, for LCNs

with at most 14 atoms, it evaluates every LMC assertion in the joint form of Definition 6, over all configurations, on 3 products of family tables with random Dirichlet rows (fixed seed), with absolute tolerance 10^{-9}; above 14 atoms it returns *unknown*. The residuals are polynomials in the table entries, and a nonzero polynomial vanishes on a set of measure zero, so a false *true* has probability zero in exact arithmetic; a *false* is a numerical certificate of failure.

E.4 COUNTEREXAMPLES

The following LCNs show that the hypotheses of Propositions 5 and 6 cannot be dropped. They are in `examples/fr_counter_*.lcn` and checked by `tests/test_realizability.py`; the hypotheses stated for each are those returned by `realizability_report`, and the $\mathcal{D}_{\mathrm{LCN}}$ bounds are certified by SCIP. All but the third (the non-factorizing one) are fully realizable, every sentence having form R1 or R2.

A compound block: the interval box widens the local set; the exact polytope does not. Let $A \to \{C, D\}$ with $0.3 \le P(A) \le 0.6$, and on each row two R2 sentences: $0.9 \le P(\neg(C \oplus D) \mid A) \le 1, 0.2 \le P(C \mid A) \le 0.3, 0.4 \le P(\neg(C \oplus D) \mid \neg A) \le 0.6, 0.5 \le P(C \mid \neg A) \le 0.7$ (with τ, so that $\{C, D\}$ is one chain component). The local set $K_{v,\pi}$ at each row is a polytope cut out by both sentences. The per-state interval box around it is larger: at $A{=}1$ it admits $P(\neg(C \oplus D)) \in [0.8, 1]$ and $P(C) \in [0.1, 0.4]$. With box local sets CVE, exact for the resulting strong extension, gives $P(C) \in [0.03, 0.82]$, while $\mathcal{D}_{\mathrm{LCN}}$ gives $[0.32, 0.58]$. The family is complete (A is adjacent to C and D, and $C - D$), so there is no LMC assertion and the LCN is product-Markov; it is also locally feasible and fully realizable. With the exact row polytopes (12 and 6 vertices at $A{=}0$ and $A{=}1$, against 11 and 8 for the boxes) CVE gives $[0.32, 0.58]$, as Proposition 6 predicts.

An incomplete component: free rows violate the LMC. Let C, D, E form a root component $C - D - E$ through the R1 sentences $0.6 \le P(C \oplus D) \le 0.9$ and $0.6 \le P(D \oplus E) \le 0.9$ (with τ). The LCN is fully realizable, locally feasible, and factorizing (a single root node: every p is its own table). The LMC is $(C \perp\!\!\!\perp E \mid D)$, a nonlinear constraint inside the component's single row, so K^{true}, the set of rows of members of $\mathcal{D}_{\mathrm{LCN}}$, is the polytope K intersected with a curved set, and is not convex. The compiler stores the exact polytope K (56 vertices), which ignores the assertion; its vertices violate it by up to 0.25 (in $\max_d |P(C, E \mid d) - P(C \mid d)P(E \mid d)|$). Hence $\mathcal{P}_{\mathrm{SE}} \not\subseteq \mathcal{D}_{\mathrm{LCN}}$, and the LCN is not product-Markov. This gap is at the level of *sets* only: here both $\mathcal{D}_{\mathrm{LCN}}$ and CVE give $[0, 1]$ for every atom. Exact polytope rows cannot close it in general: they contain the curved set, whose convex hull need not be a polytope (a convex-hull computation with a global oracle that includes the independence keeps cutting facets), so no finite credal network represents it. The gap reaches the *bounds* once evidence is added (`fr_counter_lmc_evidence.lcn`): with the R1 sentences $P(D) \in [0.45, 0.55]$, $P(C \wedge D), P(E \wedge D) \in [0.35, 0.45]$ and $P(C \wedge \neg D), P(E \wedge \neg D) \in [0.05, 0.1]$ on the same component, the polytope rows (104 vertices) give the $\mathcal{D}_{\mathrm{LCN}}$ bounds on every marginal without evidence, but with evidence $C{=}1$ they give $P(E \mid C) \in [0.333, 1]$, against $[0.5196, 0.9182]$ for $\mathcal{D}_{\mathrm{LCN}}$. The two-level chain-graph factorization that would restore the Markov property (Lauritzen, 1996) constrains each component's conditional, which a credal-network row does not.

A non-factorizing compound LCN: the strong extension is not outer. Let $Z \to Y \to \{C, D\}$ with $0.4 \le P(Z) \le 0.6$, $0.4 \le P(Y \mid Z) \le 0.6$, $0.3 \le P(C \mid Y) \le 0.7$ and $0.3 \le P(C \vee D) \le 0.9$ (with τ, so that $\{C, D\}$ is one chain component; `examples/fr_counter_factorizing.lcn`). This example targets Proposition 5, and several hypotheses of Proposition 6 fail as well: the LCN is locally feasible with exact local sets, but it is not fully realizable ($s_4 = P(C \vee D)$ is a non-constant Type-1 sentence on the non-root block $\{C, D\}$, realizable in no family by Lemma 8(i)), not DAG-structured, and not product-Markov (its assertion $(D \perp\!\!\!\perp Z, Y \mid C)$ fails for products whose rows $P(C, D \mid Y{=}0)$ and $P(C, D \mid Y{=}1)$ give different $P(D \mid C)$). The LMC is $(C \perp\!\!\!\perp Z \mid Y, D)$ and $(D \perp\!\!\!\perp Z, Y \mid C)$. Take $Z \sim \mathrm{Bern}(\frac{1}{2})$, Y independent of Z with $P(Y) = \frac{1}{2}$, and $C = D = \neg Z$. Every sentence holds, and both assertions hold because C and D determine each other; this is where the intersection property fails. So $p \in \mathcal{D}_{\mathrm{LCN}}$, but p does not factorize along $Z \to Y \to C$-D, since C-D depends on Z given Y. The LCN is not factorizing, and Proposition 5 does not apply: p gives $P(C \mid Z{=}1) = 0$ (and $\min P(C \mid Z{=}1) = 0$ over $\mathcal{D}_{\mathrm{LCN}}$), while CVE, exact for $\mathcal{D}_{\mathrm{SE}}$, gives $[0.12, 0.88]$, which is *not* an outer bound. Without evidence the strong-extension bounds happen to contain the LCN ones ($P(C)$: $[0.048, 0.952]$ against $[0.048, 0.9]$).

Hence for compound LCNs the outer guarantee of the compiled methods requires factorization; without it they bound only the closure of the strictly positive part of $\mathcal{D}_{\mathrm{LCN}}$ (Remark 2).

Local feasibility. Let $U \to X$ with $0 \le P(U) \le 1$, $0.8 \le P(X \mid U) \le 1$, $0 \le P(X \mid U) \le 0.2$ and $0.3 \le P(X \mid \neg U) \le 0.6$. The LCN is DAG-structured (hence factorizing and product-Markov), fully realizable, and has exact local sets, but it is not locally feasible: the two sentences on the row $U{=}1$ contradict each other, so every member of $\mathcal{D}_{\mathrm{LCN}}$ has $P(U) = 0$, which no local set encodes. The compiler detects the row with the phase-1 LP, stores it as the simplex (Definition 11) and reports it in `infeasible_rows`. Both bounds of $P(U)$ in $\mathcal{D}_{\mathrm{LCN}}$ are 0, and $P(X) \in [0.3, 0.6]$; CVE gives $P(U) \in [0, 1]$ and $P(X) \in [0, 1]$, which is outer (Proposition 5) but not exact.

We now give an example of a loopy LCN on which CVE is exact.

Example 6. *Single-connectedness is not necessary for exactness. Consider the following family-realizable k-tree LCN with $n{=}4$ atoms and $k{=}2$ (`examples/ktree_fr_k2_n4_example.lcn`):*

$$s_0\text{: } 0.084382 \le P(x_1) \le 0.684382 \qquad s_2\text{: } 0.092785 \le P(x_3 \mid x_1 \wedge x_2) \le 0.692785$$
$$s_1\text{: } 0.01 \le P(\neg x_2 \mid \neg x_1) \le 0.572656 \quad s_3\text{: } 0.068242 \le P(x_0 \mid x_1 \wedge x_3) \le 0.668242$$

*The seed 3-clique is $\{x_1, x_2, x_3\}$ and x_0 attaches to the 2-clique $\{x_1, x_3\}$, so the moralized graph contains the triangle $x_1x_2x_3$: the instance is **loopy**, decidedly not singly connected. It is also fully realizable (each ψ is a full conjunction over its family's parents; s_0 is a root marginal), DAG-structured and locally feasible, so Proposition 6 applies. Measured, with all eight SCIP solves `confirmed` at gap 0:*

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$
`Exact` *(global)*	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
`CJT` *(SCIP)*	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
CVE	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$

CVE is exact on every atom *of a loopy LCN, as Proposition 6 predicts.*

F Detailed Description of Credal Variable Elimination

This appendix details Credal Variable Elimination (CVE), which extends classical variable elimination Dechter (1996) to credal networks by propagating potentials made of finite sets of *pairs* of non-negative functions (Mauá et al., 2012; Mauá & Cozman, 2020). Interval Belief Propagation and CDVE are detailed in Appendices G and H.

F.1 Credal Networks

The compiled **credal network** is a pair $\langle D, \mathcal{K} \rangle$, where D is the component DAG over the nodes $\mathbf{V} = \{V_1, \ldots, V_m\}$, each with a finite domain Ω_i (the assignments to the atoms of the component, $|\Omega_i| = 2^{|\mathrm{ch}_{V_i}|}$), and $\mathcal{K}$ collects one local credal set $K_{i,\pi} \subseteq \Delta(\Omega_i)$ for each node V_i and each assignment π of its parent nodes $\mathrm{Pa}(V_i)$ (Definition 11). Each $K_{i,\pi}$ is a polytope (an interval for a single-atom node, a row polytope or, if that computation fails, the reported box for a compound node), given by its vertex set $\mathrm{ext}K_{i,\pi}$.

F.2 Strong Extension

A choice θ of one row $\theta_{i,\pi} \in K_{i,\pi}$ for every node and parent assignment defines the product $p_\theta(\mathbf{x}) = \prod_{i=1}^{m} \theta_{i,\pi_i(\mathbf{x})}(x_i)$, where $\pi_i(\mathbf{x})$ is the restriction of $\mathbf{x}$ to $\mathrm{Pa}(V_i)$. The product set is $\mathcal{P}_{\mathrm{SE}} = \{p_\theta\}$, with the rows chosen independently, and the **strong extension** is

$$\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{SE}}) = \mathrm{CH}(\mathcal{P}_{\mathrm{ext}}), \qquad \mathcal{P}_{\mathrm{ext}} = \{p_\theta : \theta_{i,\pi} \in \mathrm{ext}K_{i,\pi} \text{ for all } i, \pi\}. \tag{28}$$

The second equality holds because p_θ is affine in each row: every term of $p_\theta(\mathbf{x})$ contains exactly one entry of one row of each node. Writing one row as a convex combination of its vertices writes p_θ as

the same convex combination of products, so replacing the rows one at a time by vertices shows that every member of $\mathcal{P}_{\mathrm{SE}}$ is a convex combination of members of the finite set $\mathcal{P}_{\mathrm{ext}}$ (Mauá & Cozman, 2020).

F.3 Marginal Inference

Let a be an atom and $\mathbf{e}$ evidence on the atoms $\mathbf{E}$. Following Definition 7, the target is

$$\underline{P}(a \mid \mathbf{e}) = \inf\Big\{ \frac{p(a,\mathbf{e})}{p(\mathbf{e})} : p \in \mathcal{D}_{\mathrm{SE}},\ p(\mathbf{e}) > 0 \Big\}, \tag{29}$$

$\overline{P}(a \mid \mathbf{e})$ likewise with sup, and the interval is $[0,1]$ when $\overline{P}(\mathbf{e}) = \sup_{p \in \mathcal{D}_{\mathrm{SE}}} p(\mathbf{e}) = 0$.

Lemma 11 (Vertex products)**.** $\overline{P}(\mathbf{e}) = \max_{p \in \mathcal{P}_{\mathrm{ext}}} p(\mathbf{e})$. *If $\overline{P}(\mathbf{e}) > 0$, the infimum and supremum in equation 29 equal the minimum and maximum of $p(a,\mathbf{e})/p(\mathbf{e})$ over the finite nonempty set $\{p \in \mathcal{P}_{\mathrm{ext}} : p(\mathbf{e}) > 0\}$, and likewise over $\{p \in \mathcal{P}_{\mathrm{SE}} : p(\mathbf{e}) > 0\}$.*

Proof. (1) $p \mapsto p(\mathbf{e})$ is linear, so its supremum over $\mathcal{D}_{\mathrm{SE}} = \mathrm{CH}(\mathcal{P}_{\mathrm{ext}})$ (Eq. 28) is its maximum over the finite set $\mathcal{P}_{\mathrm{ext}}$. (2) Hence, if $\overline{P}(\mathbf{e}) > 0$, some member of $\mathcal{P}_{\mathrm{ext}}$ has $p(\mathbf{e}) > 0$. (3) Apply Lemma 3(b) with $\mathcal{M} = \mathcal{P}_{\mathrm{ext}}$ (finite, so compact), b the indicator of $\mathbf{e}$ and a the indicator of $a \wedge \mathbf{e}$: the infimum and supremum of the ratio over $\mathcal{D}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ equal those over $\mathcal{P}_{\mathrm{ext}} \cap \{p(\mathbf{e}) > 0\}$, a finite nonempty set by (2), where they are attained. (4) Since $\mathcal{P}_{\mathrm{ext}} \subseteq \mathcal{P}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{SE}}$, the bounds over $\mathcal{P}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ are squeezed between the two and coincide with them. □

F.4 Pair Potentials

Definition 13 (Pair potential)**.** *A **potential** $\phi(\mathbf{Y})$ over nodes $\mathbf{Y}$ is a finite set of pairs (p,q) of nonnegative functions on the joint domain $\Omega_{\mathbf{Y}}$ of $\mathbf{Y}$. Each pair corresponds to one choice of vertices of the rows that have been multiplied into it (Mauá et al., 2012; Mauá & Cozman, 2020).*

Product and sum-marginal. The product of $\phi(\mathbf{Y})$ and $\psi(\mathbf{Z})$ is $\phi \cdot \psi = \{(pr,\, qs) : (p,q) \in \phi,\ (r,s) \in \psi\}$ over $\mathbf{Y} \cup \mathbf{Z}$, and the sum-marginal sums both components, $\sum_X \phi = \{(\sum_x p,\ \sum_x q) : (p,q) \in \phi\}$.

Input potentials. A node V_i with parent assignments $\pi_1, \ldots, \pi_r$ contributes $\phi_{V_i} = \{(f,f)\}$ over $\{V_i\} \cup \mathrm{Pa}(V_i)$, one function $f(x,\pi_j) = \theta_{i,\pi_j}(x)$ per element $(\theta_{i,\pi_1}, \ldots, \theta_{i,\pi_r})$ of $\mathrm{ext}K_{i,\pi_1} \times \cdots \times \mathrm{ext}K_{i,\pi_r}$ (a complete table with one vertex in every row), so $|\phi_{V_i}| \le \prod_j |\mathrm{ext}K_{i,\pi_j}|$. The potential is an explicit enumeration of the family's contribution to $\mathcal{P}_{\mathrm{ext}}$. Evidence on atoms of V_i multiplies the $0/1$ indicator δ_i of the states of V_i that agree with $\mathbf{e}$ into both components of ϕ_{V_i}. For an atom a of the node Z with $0/1$ indicator t over Ω_Z, the *query potential* is $\{(1-t,\ t)\}$.

Read-off. After every node has been eliminated, each pair is a pair of numbers (p,q), and

$$\underline{P}(a \mid \mathbf{e}) = \min_{(p,q),\ p+q>0} \frac{q}{p+q}, \qquad \overline{P}(a \mid \mathbf{e}) = \max_{(p,q),\ p+q>0} \frac{q}{p+q},$$

with $[0,1]$ when no pair has $p+q>0$ (`PairPotential.bounds`). The ratio is taken per pair, which is the conditioning step: it is *not* the invalid ratio $\min P(a,\mathbf{e}) / \max P(\mathbf{e})$ of a separately optimized numerator and denominator.

Definition 14 (Pareto pruning)**.** *Let ϕ be a set of distinct pairs. For the upper bound, (p,q) is* dominated *when another pair $(r,s) \neq (p,q)$ of ϕ has $r \le p$ and $s \ge q$ pointwise; for the lower bound the order is reversed ($r \ge p$, $s \le q$). $\mathrm{prune}(\phi)$ removes the pairs with $p = q = 0$ and the pairs dominated in* both *orders, so one set serves both bounds. CVE applies it only to the final scalar pairs (`CredalVE._query`), where it is equivalent to taking the minimum and maximum directly: it saves nothing, but is harmless (Lemma 12(1)).*

Pareto pruning is useless before the query enters: there every pair has the form (f,f), and (f,f) dominates (g,g) in the upper order only if $f \le g$ and $f \ge g$, i.e. $f = g$. The query-free potentials are reduced by their hulls instead.

Definition 15 (Hull pruning)**.** *For a potential whose pairs all have $p = q$ (no query multiplied in yet),* $\mathrm{prune}_{\mathrm{hull}}(\phi)$ *first rescales every pair to* $\max f = 1$ *and drops the pairs with $f = 0$ and the duplicates, and then keeps the pairs (f, f) whose ray $\hat{f} = f/\sum f$ is a vertex of the convex hull of $\{\hat{f} : (f, f) \in \phi\}$ (`PairPotential.prune_query_free`). The rescaling keeps long products of probabilities in floating-point range and changes no ratio $q/(p + q)$.*

Definition 16 (α-quotient (Mauá et al., 2012))**.** *For $\alpha > 1$, two pairs are* α-equivalent *when, after scaling each pair to $\sum(p + q) = 1$, every coordinate of both components is zero in both or positive in both with equal $\lfloor \log_\alpha \cdot \rfloor$. The quotient ϕ/α drops the pairs with $p = q = 0$ and keeps, of each equivalence class, the members with the lowest and the highest share $\sum q/\sum(p + q)$ (one member if they coincide; `PairPotential.quotient`). One member per class would suffice for the covering argument (Lemma 12(3)); keeping two at most doubles the size bound.*

Numerical tolerances. All three engines work on the enumerated vertices, which are exact only to floating-point precision: the LRS vertices of the alarm example sum to one only to about $4 \cdot 10^{-7}$. The removal decisions of $\mathrm{prune}_{\mathrm{hull}}$ (`hull_vertices`) use the following thresholds; each fallback keeps rows, and keeping a row is always sound.

- Rays are deduplicated after rounding to 15 decimals.
- The rays are projected onto their affine hull by an SVD; singular values below $10^{-12} \cdot \mathrm{scale} \cdot \max(1, \sqrt{k})$ are cut, with k the number of rows and $\mathrm{scale} = \max(1, \max|\hat{f}|)$. Rank 0 keeps one row and rank 1 keeps the two endpoints (closed form).
- qhull is used up to affine dimension 7; dimensions 6 and 7 only with at most 5000 rows.
- Otherwise, or if qhull fails, Clarkson's LP vertex test is used for at most 1000 rows. A row is removed only if a convex combination of other rows reproduces it to relative residual 10^{-10}. Above 1000 rows every row is kept.

IVBP (Appendix G) merges message vertices that agree to 12 decimals, and raises an error when one update would enumerate more than 200 000 corners.

F.5 Algorithm

CVE answers all atom marginals of one node Z by a single bucket-elimination pass in which Z is eliminated *last* (Algorithm 2). The variables of the elimination are the nodes of the credal network, not atoms: a node carries the atoms $\mathrm{atoms}(Z)$ and has $2^{|\mathrm{atoms}(Z)|}$ states (most significant bit first). Because the query enters only after every other node is gone, all of Z's atoms share the elimination. Each atom then costs only the final product with its query potential, so a compound node needs no projection of per-state bounds onto its atoms. Every query-free potential that the pass creates, including every intermediate product of a bucket, is reduced by $\mathrm{prune}_{\mathrm{hull}}$ (`CredalVE._node_potential`), so every pairwise product combines two potentials that are each an input potential or a reduced one.

All marginals of the network are obtained by running Algorithm 2 once per node, each with its own ordering. Nothing is shared between nodes.

F.6 Elimination Ordering

Two heuristics are supported; both hold Z out of the ordering.

- **Reverse topological order** of the DAG, restricted to $\mathbf{V} \setminus \{Z\}$.
- **Min-fill**: the greedy interaction-graph heuristic, eliminating at each step the node that adds the fewest fill-in edges, with an alphabetical tiebreak. Z still counts as a neighbour when fill is scored. This aims at a small induced width w^*.

F.7 Running Example

Example 7 (Alarm Network)**.** *The LCN `examples/alarm.lcn` over atoms A, B, C, D, E factorizes into four credal-network nodes: the singletons B (Burglary), E (Earthquake), A (Alarm),*

Algorithm 2 Credal Variable Elimination (CVE) and its FPTAS CVE_ε

Require: Vertices $\mathrm{ext}K_{i,\pi}$ of every row of every node V_i; query node Z; evidence $\mathbf{e}$; $\varepsilon \geq 0$
Ensure: $[\underline{P}(a \mid \mathbf{e}),\ \overline{P}(a \mid \mathbf{e})]$ for each non-evidence atom a of Z

1: $\Gamma \leftarrow \{\phi_{V_i} : V_i \in \mathbf{V}\}$ with the evidence indicators multiplied in, zero pairs and duplicates dropped
2: $\mathbf{o} \leftarrow$ an elimination ordering of $\mathbf{V} \setminus \{Z\}$ (min-fill)
3: $N \leftarrow$ number of pairwise products and summations below, plus one; $\alpha \leftarrow (1+\varepsilon)^{1/(2N)}$ ▷ $\alpha = 1$: exact
4: **for** each V_i in ordering $\mathbf{o}$ **do**
5: $\quad \Gamma_{V_i} \leftarrow \{\phi \in \Gamma : V_i \in \mathrm{scope}(\phi)\};\ \Gamma \leftarrow \Gamma \setminus \Gamma_{V_i}$
6: $\quad \psi \leftarrow$ the product of Γ_{V_i}, one pairwise product at a time, each followed by $/\alpha$ and $\mathrm{prune}_{\mathrm{hull}}$
7: $\quad \psi \leftarrow \mathrm{prune}_{\mathrm{hull}}\big((\sum_{V_i} \psi)/\alpha\big)$ ▷ $p = q$ throughout
8: $\quad \Gamma \leftarrow \Gamma \cup \{\psi\}$
9: $\phi_Z \leftarrow$ the product of Γ (each pairwise product followed by $/\alpha$ and $\mathrm{prune}_{\mathrm{hull}}$) ▷ scope $\{Z\}$
10: **for** each atom a of Z not in the evidence **do**
11: $\quad \phi \leftarrow \mathrm{prune}\big(\big(\sum_Z \phi_Z \cdot \{(1-t_a,\ t_a)\}\big)/\alpha\big)$
12: $\quad$ report $\big[\min_\phi \frac{q}{p+q},\ \max_\phi \frac{q}{p+q}\big]$ over the pairs with $p+q>0$, or $[0,1]$ if there is none

and the compound *node C-D carrying the two atoms C, D. They are merged because the sentence $0.5 \leq P(\neg(C \oplus D) \mid A) \leq 0.7$ constrains them jointly. The compound node has $2^2 = 4$ states, ordered $CD \in \{00, 01, 10, 11\}$ most-significant bit first. The DAG is*

$$B \to A, \quad E \to A, \quad A \to C\text{-}D.$$

The local credal sets are listed below (LRS vertex enumeration for the single-atom nodes; the exact row polytope of Section 4 for C-D):

Node	*Parent Config*	*#*	*Vertices*
B	—	*2*	$[0.8, 0.2]$, $[0.9, 0.1]$
E	—	*2*	$[0.9, 0.1]$, $[0.95, 0.05]$
A	B=0, E=0	*2*	$[0.956, 0.044]$, $[1.0, 0.0]$
A	B=1, E=0	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
A	B=0, E=1	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
A	B=1, E=1	*2*	$[0.0, 1.0]$, $[1.0, 0.0]$
C-D	A=0	*4*	$[0,0,0,1]$, $[0,0,1,0]$, $[0,1,0,0]$, $[1,0,0,0]$
C-D	A=1	*8*	$[0,0,0.3,0.7]$, $[0,0,0.5,0.5]$, …, $[0.7,0,0,0.3]$

Initial potentials. *The potential sizes are the* products *of the counts above: $|\phi_B| = 2$, $|\phi_E| = 2$, $|\phi_A| = 2^4 = 16$, $|\phi_{C\text{-}D}| = 4 \times 8 = 32$.*

Query: the atom B (no evidence). *Min-fill with B held out gives $\mathbf{o} = (C\text{-}D, A, E)$.*

Step 1: eliminate C-D. *The bucket is $\phi_{C\text{-}D}$ alone. Summing out C-D maps each of its 32 functions to the constant 1 over $\{A\}$ (each is a conditional table), and hull pruning leaves a single function.*

Step 2: eliminate A. *The bucket is ϕ_A (16 functions over $\{A, B, E\}$) and the constant potential. Their product has 16 pairs, all of them hull vertices, and summing out A maps all of them to the constant 1 over $\{B, E\}$. Hull pruning therefore keeps a single function, up to floating-point rounding: the enumerated vertices sum to one only to about $4 \cdot 10^{-7}$, and two near-duplicates of the constant are kept, which is harmless.*

Step 3: eliminate E. *The product of these two functions with ϕ_E has 4 pairs, and summing out E again leaves two near-constant functions over $\{B\}$. Their product with ϕ_B has 4 pairs, and hull pruning keeps 2: up to rounding, the rescaled vertices $(1, 0.25)$ and $(1, \frac{1}{9})$ of ϕ_B. So the query step sees ϕ_B alone, up to rounding.*

Result. *The product with the query potential* $\{(1-t_B,\, t_B)\}$*, summed over* B*, gives pairs whose ratios* $q/(p+q)$ *range over*

$$\underline{P}(B{=}1) = 0.10, \qquad \overline{P}(B{=}1) = 0.20,$$

exactly the interval of the sentence $0.1 \le P(B) \le 0.2$*, as it must be since* B *is a root. The largest product formed in this pass has* 16 *pairs (the largest input potential,* $\phi_{C\text{-}D}$*, has* 32*).*

All marginals. *The four passes give* $P(A{=}1) \in [0, 0.312]$*,* $P(B{=}1) \in [0.1, 0.2]$*,* $P(E{=}1) \in [0.05, 0.1]$ *and* $P(C{=}1), P(D{=}1) \in [0, 1]$ *(the largest product,* 64 *pairs reduced to* 36*, occurs in the pass for* C*-*D*). The last two come directly from the query indicators of the compound node, and are vacuous because* $A{=}0$ *leaves* C*-*D *unconstrained. The LCN is not fully realizable (*s_3 *conditions on a disjunction of parent configurations and* s_5 *is a Type-1 sentence on the non-root* A*), so these bounds are outer for* $\mathcal{D}_{\mathrm{LCN}}$*; for instance,* $P(A) \in [0, 0.312]$ *although a sentence states* $0.05 \le P(A) \le 0.1$ *— the compiled network ignores that sentence. With evidence* $C{=}1$ *CVE returns* $[0, 1]$ *for every atom, for two reasons that combine. The rows of* A *given* $B \vee E$ *are vacuous (vertices* $[0, 1]$ *and* $[1, 0]$*), so a vertex product can make* A *equal to* $B \vee E$ *or force* $A{=}0$ *whenever* $B \vee E$ *holds; and the row of* C*-*D *given* $A{=}0$ *is the whole simplex, so* $C{=}1$ *can be made impossible or certain given* $A{=}0$*. With* $C{=}1$ *impossible given* $A{=}0$*, letting* $A{=}1$ *exactly when* B *holds gives* $P(B{=}1 \mid C{=}1) = 1$*, and forcing* $A{=}0$ *whenever* B *holds (*$A{=}1$ *when* E *holds and* B *does not) gives* $P(B{=}1 \mid C{=}1) = 0$*; the other atoms are vacuous in the same way. CVE*$_\varepsilon$ *with* $\varepsilon = 0.1$ *returns the same intervals to four decimals.*

F.8 Proofs of Theorems 3, 4 and Proposition 3

Proof of Theorem 3. Step 1 (one pass). A pass for the query node Z starts with m input potentials. Every pairwise product replaces two live potentials by one and a summation replaces one by one, so a pass performs exactly $m-1$ pairwise products and $m-1$ summations (Algorithm 2). By construction (Section F.5) both operands of a product are input potentials or outputs of $\mathrm{prune}_{\mathrm{hull}}$, so each has at most C pairs and the product at most C^2, over at most w^*+1 nodes of domain size at most d: time and space $O(C^2 d^{w^*+1})$. A summation is linear in the size of its operand. Hence the pass costs $O(m\, C^2 d^{w^*+1})$; the at most m live potentials, each of at most C pairs over at most w^*+1 nodes, take $O(m\, C\, d^{w^*+1})$ space. The query step costs $O(C\, d)$ per atom.

Step 2 (reductions, extra). $\mathrm{prune}_{\mathrm{hull}}$ of a set of $k \le C^2$ rays of length $|\Omega| \le d^{w^*+1}$ costs an SVD, $O(k|\Omega|^2)$, plus qhull or Clarkson's test (about $k+h$ LPs with h columns, h the number of vertices). The quotient costs $O(k|\Omega|)$ with hashing, and the final Pareto prune is quadratic in the number of final pairs. These costs are not included in the bound.

Step 3 (all marginals). Algorithm 2 runs once per node, a factor m.

Step 4 (CVE$_\varepsilon$*).* After scaling a pair to mass 1 every positive entry x lies in $[t, 1]$, where t is the smallest positive entry after this scaling, so $\lfloor \log_\alpha x \rfloor$ takes one of the $1 - \lfloor \log_\alpha t \rfloor$ values $\lfloor \log_\alpha t \rfloor, \ldots, 0$; with the zero class, each of the $2|\Omega|$ coordinates of (p, q) takes at most $2 - \lfloor \log_\alpha t \rfloor$ values. There are therefore at most $(2 - \lfloor \log_\alpha t \rfloor)^{2|\Omega|}$ classes, and $|\phi/\alpha| \le 2(2 - \lfloor \log_\alpha t \rfloor)^{2|\Omega|}$. The scaling makes t independent of the rescalings of Definition 15. Before scaling, an entry is a sum of at most d^m products of at most m input probabilities and indicators, so a positive entry is at least 2^{-bm} (b the number of bits of an input probability) and the mass is at most $2|\Omega| d^m$; hence $\ln(1/t) \le m\, b \ln 2 + m \ln d + \ln(2|\Omega|)$. With $\alpha = (1+\varepsilon)^{1/(2N)}$, $N \le 2m$, and $\ln(1+\varepsilon) \ge \varepsilon/(1+\varepsilon)$, $-\lfloor \log_\alpha t \rfloor \le 1 + 2N(1+\varepsilon)\ln(1/t)/\varepsilon$, which is polynomial in m, b, $\log d$ and $1/\varepsilon$; for bounded $|\Omega| \le d^{w^*+1}$ the class count is polynomial. An input potential has at most $\prod_\pi |\mathrm{ext} K_{i,\pi}| \le e^{d^{w^*}}$ pairs, e the maximum number of vertices of a row, which is polynomial in the size of the compiled network (vertices included) for bounded d and w^*. Hence C is polynomial (Proposition 7 and Theorem 11 of Mauá et al. 2012). The exact prunings only remove pairs. □

Exactness rests on one invariant, shared by the three reductions. For non-negative weight vectors u, v on a scope's domain and a pair (p, q) with $\langle u, q\rangle + \langle v, p\rangle > 0$, let

$$R_{u,v}(p, q) = \frac{\langle u, q\rangle}{\langle u, q\rangle + \langle v, p\rangle};$$

$R_{u,v}(p,q)$ is *undefined* when the denominator is zero. The weights stand for everything the rest of the elimination does to a pair: fix one element of every other potential, and the final pair of numbers is $(\langle v,p\rangle, \langle u,q\rangle)$, with v and u the products and sums contributed by those elements. Its read-off $q/(p+q)$ is $R_{u,v}(p,q)$. Two facts are used throughout: $R_{u,v}(cp,cq) = R_{u,v}(p,q)$ for $c>0$ (the factor cancels, and definedness is unchanged), and $R_{u,v}$ is non-decreasing in q and non-increasing in p where defined: for $x \ge x_0$, $y \le y_0$ with $x+y, x_0+y_0 > 0$, $\frac{x}{x+y} - \frac{x_0}{x_0+y_0} = \frac{xy_0 - x_0 y}{(x+y)(x_0+y_0)} \ge 0$.

Definition 17 (Covering)**.** *For $c \ge 1$, a set S' of pairs c-*covers *a set S if every element of S' is a positive multiple of an element of S, and for all $u, v \ge 0$ and every $(p,q) \in S$ with $R_{u,v}(p,q)$ defined there are $(r,s), (r',s') \in S'$ with $R_{u,v}$ defined and*

$$R_{u,v}(r,s) \;\ge\; R_{u,v}(p,q)/c, \qquad R_{u,v}(r',s') \;\le\; c\,R_{u,v}(p,q).$$

Lemma 12 (Covering calculus)**.** *1.* Pareto pruning *(Def. 14), the removal of exact duplicates and of pairs with $p=q=0$, and the rescaling of query-free pairs* 1*-cover.*

2. Hull pruning *(Def. 15) of a potential with $p=q$* 1*-covers.*

3. The α-quotient *(Def. 16)* α^2*-covers.*

4. If S' c-covers S and T' d-covers T, then $S' \cdot T'$ cd-covers $S \cdot T$; $\sum_X S'$ c-covers $\sum_X S$; and if S'' c'-covers S', which c-covers S, then S'' cc'-covers S.

Proof. (1) *Duplicates, zero pairs, rescaling.* Removing a duplicate keeps an identical pair. A pair with $p=q=0$ has no defined $R_{u,v}$, so nothing must be covered for it. A rescaled pair (cf, cf), $c>0$, has the same $R_{u,v}$ as (f,f) and the same definedness.

Pareto pruning. Write $(r,s) \succ_{\max} (p,q)$ when $(r,s) \ne (p,q)$, $r \le p$ and $s \ge q$, and $\succ_{\min}$ for the reverse order. Each is a strict partial order on the finite set of distinct pairs (irreflexive and transitive), so every pair dominated in an order lies below a maximal element of that order, and a maximal element is not dominated in that order, hence not removed. Let (p,q) be removed and $R_{u,v}(p,q)$ defined; write $x_0 = \langle u,q\rangle$, $y_0 = \langle v,p\rangle$, $x_0 + y_0 > 0$. Take a kept $(r,s) \succ_{\max} (p,q)$ and a kept $(r',s') \succ_{\min} (p,q)$.

(a) If $x_0 > 0$: $\langle u,s\rangle \ge x_0 > 0$, so $R_{u,v}(r,s)$ is defined, and $R_{u,v}(r,s) \ge R_{u,v}(p,q)$ by monotonicity.

(b) If $x_0 = 0$: then $y_0 > 0$ and $R_{u,v}(p,q) = 0$; $\langle v,r'\rangle \ge y_0 > 0$, so $R_{u,v}(r',s')$ is defined and is $\ge 0 = R_{u,v}(p,q)$. The upper requirement is met by (r',s') (its value ≥ 0 suffices), even if $R_{u,v}(r,s)$ is undefined.

Symmetrically, for the lower requirement: if $y_0 > 0$, (r',s') has $R_{u,v}$ defined and $\le R_{u,v}(p,q)$; if $y_0 = 0$, then $R_{u,v}(p,q) = 1$ and (r,s) has $\langle u,s\rangle \ge x_0 > 0$, so it is defined and ≤ 1. Kept pairs are members, so prune 1-covers.

(2) *Hull pruning.* By (1) the rescaling and the removal of zero rows and duplicates 1-cover, so let every pair have $\sum f > 0$. With $p=q=f$, $R_{u,v}(f,f) = \langle u,f\rangle / \langle u+v,f\rangle$ is defined iff $\langle u+v,f\rangle > 0$ and depends only on the ray $\hat{f} = f/\sum f$. Let (f,f) be removed, so $\hat{f} = \sum_k \lambda_k \hat{f}_k$ is a convex combination of kept rays, and let $R_{u,v}(\hat{f})$ be defined. Then $\langle u+v,\hat{f}\rangle = \sum_k \lambda_k \langle u+v,\hat{f}_k\rangle > 0$, and by Lemma 3(b) (with $b = u+v$, $a = u$, $\mathcal{M}$ the kept rays) $R_{u,v}(\hat{f})$ is a convex combination of the defined values $R_{u,v}(\hat{f}_k)$, with weights $\mu_k \propto \lambda_k \langle u+v,\hat{f}_k\rangle$. So some kept pair has a defined value at least, and some at most, $R_{u,v}(f,f)$. (The kept set is computed with the tolerances listed above; each fallback keeps more rows, which preserves the covering.)

(3) *Quotient.* Pairs with $p=q=0$ are handled by (1). Let (p,q) be removed and (r,s) a kept member of its class, both scaled to mass 1 (which changes no $R_{u,v}$). On every coordinate both entries are zero, or both positive with the same $\lfloor \log_\alpha \cdot \rfloor$, so their ratio lies in $(1/\alpha, \alpha)$: $r \le \alpha p$, $r \ge p/\alpha$, $s \le \alpha q$ and $s \ge q/\alpha$ pointwise. The zero patterns agree, so $R_{u,v}(r,s)$ is defined iff $R_{u,v}(p,q)$ is. Write $x = \langle u,q\rangle$ and $y = \langle v,p\rangle$. Then $R_{u,v}(r,s) \ge \frac{x/\alpha}{x/\alpha + \alpha y} = \frac{x}{x+\alpha^2 y} \ge \frac{1}{\alpha^2}\frac{x}{x+y}$ and $R_{u,v}(r,s) \le \frac{\alpha x}{\alpha x + y/\alpha} = \frac{\alpha^2 x}{\alpha^2 x + y} \le \alpha^2 \frac{x}{x+y}$. A single kept member therefore serves both requirements, and kept pairs are members.

(4) *Product.* Let $\sigma = (p, q) \in S$ over $\mathbf{Y}$, $\tau = (p', q') \in T$ over $\mathbf{Z}$, and $u, v \geq 0$ on $\Omega_{\mathbf{Y}\cup\mathbf{Z}}$ with $R_{u,v}(\sigma \cdot \tau)$ defined. Multiply u by q' and v by p' and marginalize onto $\mathbf{Y}$, ϕ's scope: $u^\tau(\mathbf{y}) = \sum_{\mathbf{x}:\, \mathbf{x}_\mathbf{Y}=\mathbf{y}} u(\mathbf{x})\, q'(\mathbf{x}_\mathbf{Z})$ and v^τ likewise with p'. Then $\langle u, qq' \rangle = \langle u^\tau, q \rangle$ and $\langle v, pp' \rangle = \langle v^\tau, p \rangle$, so $R_{u,v}(\sigma \cdot \tau) = R_{u^\tau, v^\tau}(\sigma)$, with $u^\tau, v^\tau \geq 0$. Since S' c-covers S, some $\sigma' \in S'$ has $R_{u,v}(\sigma' \cdot \tau) = R_{u^\tau, v^\tau}(\sigma') \geq R_{u,v}(\sigma \cdot \tau)/c$, defined. Fix σ' and apply the same argument in the second argument with the weights $u^{\sigma'}, v^{\sigma'}$ on $\mathbf{Z}$: some $\tau' \in T'$ has $R_{u,v}(\sigma' \cdot \tau') \geq R_{u,v}(\sigma' \cdot \tau)/d \geq R_{u,v}(\sigma \cdot \tau)/(cd)$. The lower requirement is symmetric. A product of positive multiples of members is a positive multiple of a member of $S \cdot T$, and $\sum_X (c\sigma) = c \sum_X \sigma$.

Sum-marginal. For u, v on $\Omega_{\mathbf{Y}\setminus\{X\}}$ let $\tilde{u}, \tilde{v}$ be their extensions to $\Omega_\mathbf{Y}$, constant along X. Then $\langle u, \sum_X q \rangle = \langle \tilde{u}, q \rangle$, so $R_{u,v}(\sum_X \sigma) = R_{\tilde{u},\tilde{v}}(\sigma)$, and the covering of S by S' with the weights $\tilde{u}, \tilde{v}$ gives the claim with the same factor c.

Transitivity. For $\sigma \in S$ pick $\sigma' \in S'$ with $R_{u,v}(\sigma') \geq R_{u,v}(\sigma)/c$, then $\sigma'' \in S''$ with $R_{u,v}(\sigma'') \geq R_{u,v}(\sigma')/c'$; likewise for the lower requirement. Positive multiples compose. □

Lemma 13 (Covering the brute-force set)**.** *Let $F_0 = \{(p_\theta(\neg a, \mathbf{e}), p_\theta(a, \mathbf{e})) : p_\theta \in \mathcal{P}_{\text{ext}}\}$ and F the final set of scalar pairs of a pass of Algorithm 2 for the atom a. Then F c-covers F_0, where c is the product of the factors of the reductions applied in the pass: $c = 1$ for CVE and $c = \alpha^{2N} = 1 + \varepsilon$ for CVE$_\varepsilon$.*

Proof. Step 1 (brute force). Run the same sequence of products and summations on the input potentials and the query potential without any reduction. A pair of the result corresponds to one choice of one element of every input potential, that is, of one vertex $\theta_{i,\pi}$ of every row (the evidence and query potentials have one element each). For a fixed choice the computation is ordinary variable elimination on fixed non-negative factors, which is correct by distributivity: the final pair is $(\sum_\mathbf{x} p_\theta(\mathbf{x})\delta_\mathbf{e}(\mathbf{x})(1 - t(\mathbf{x}_Z)), \sum_\mathbf{x} p_\theta(\mathbf{x})\delta_\mathbf{e}(\mathbf{x})t(\mathbf{x}_Z)) = (p_\theta(\neg a, \mathbf{e}), p_\theta(a, \mathbf{e}))$. So the brute-force result is F_0.

Step 2 (each potential is consumed once). Every potential, input or created, enters exactly one bucket product or the final product (it is removed from Γ when used). The computation is therefore a tree whose leaves are the input potentials; the subtrees of the two operands of a product are disjoint.

Step 3 (induction over the tree). For every node ψ of the tree let ψ^0 be the potential that the brute force computes at the same node. We show that ψ c_ψ-covers ψ^0, with c_ψ the product of the factors of the reductions in the subtree of ψ. Leaves: the input potentials, after dropping zero pairs and duplicates, 1-cover themselves (Lemma 12(1)). A product $\psi_1 \cdot \psi_2$ $c_{\psi_1} c_{\psi_2}$-covers $\psi_1^0 \cdot \psi_2^0$ (Lemma 12(4)); a summation preserves the factor; and the reduction that follows multiplies it by its own factor (Lemma 12(1)–(3) and transitivity). By Step 2 the factors of disjoint subtrees multiply, and no factor is counted twice.

Step 4 (the factor). For CVE only reductions (1) and (2) are applied, so $c = 1$. For CVE$_\varepsilon$ the pass applies N quotients: one after every pairwise product and every summation, including the final products of the remaining potentials, and one in the query step; each α^2-covers and the exact reductions have factor 1, so $c = \alpha^{2N} = 1 + \varepsilon$. □

Proof of Theorem 4. Step 1 (target). By Lemma 11, if $\overline{P}(\mathbf{e}) > 0$ the target interval is the minimum and maximum of $p_\theta(a, \mathbf{e})/p_\theta(\mathbf{e})$ over the vertex products with $p_\theta(\mathbf{e}) > 0$, i.e. of $q/(p + q)$ over the pairs of F_0 with $p + q > 0$ (note $p + q = p_\theta(\mathbf{e})$).

Step 2 (covering). By Lemma 13 with $c = 1$, applied with the scalar weights $u = v = 1$, for every pair of F_0 with $p + q > 0$ there are pairs of F with $p + q > 0$ whose ratio is at least, and at most, its ratio. Every pair of F is a positive multiple of a pair of F_0, so its ratio, when defined, is the ratio of a vertex product with $p_\theta(\mathbf{e}) > 0$. Hence the minimum and maximum of $q/(p + q)$ over $F \cap \{p + q > 0\}$ equal those over $F_0 \cap \{p + q > 0\}$, which by Step 1 are the target bounds. The final Pareto prune is part of F's computation and is covered by Lemma 12(1).

Step 3 ($\overline{P}(\mathbf{e}) = 0$). By Lemma 11 every pair of F_0 is then $(0, 0)$, so every pair of F, a positive multiple of one, is zero, and the zero pairs are dropped: F has no pair with $p + q > 0$ and CVE returns $[0, 1]$, the convention of Definition 7. Conversely, if $\overline{P}(\mathbf{e}) > 0$ Step 2 provides a pair with $p + q > 0$, so $[0, 1]$ is never returned spuriously.

Step 4 (compound atoms). The query potential is the indicator of the atom over its node's states, so atoms of compound nodes are handled directly, without projection.

Step 5 (box fallback). If a compound row keeps the box $K'_{i,\pi} \supseteq K_{i,\pi}$, Steps 1–4 hold for the compiled sets, whose strong extension $\mathcal{D}'_{\mathrm{SE}}$ contains $\mathcal{D}_{\mathrm{SE}}$ built on the exact sets. An infimum over the larger set $\mathcal{D}'_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ is at most, and a supremum at least, the one over $\mathcal{D}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$, so the bounds are outer for the latter whenever it is nonempty.

Step 6 ($\mathcal{D}_{\mathrm{LCN}}$). Under the hypotheses of Proposition 6 (consistent, locally feasible, factorizing, product-Markov, fully realizable, with exact local sets), $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, and by Lemma 11 the bounds over $\mathcal{P}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ are those of Step 1, so they are the bounds with respect to $\mathcal{D}_{\mathrm{LCN}}$.

Step 7 (precision). The argument is exact for the vertices the compiler enumerated; the computed values carry their floating-point error (about $4 \cdot 10^{-7}$ for LRS) and the hull tolerances listed above, whose fallbacks only keep pairs. □

Proof of Proposition 3. Step 1. By Lemma 13, F $(1+\varepsilon)$-covers F_0. *Step 2.* Take the pair of F_0 attaining $\overline{P}$ (Theorem 4, Step 1). Applied with $u = v = 1$, the covering gives a pair of F with $q/(p+q) \geq \overline{P}/(1+\varepsilon)$; symmetrically some pair of F has $q/(p+q) \leq (1+\varepsilon)\underline{P}$. *Step 3.* Every pair of F is a positive multiple of a pair of F_0, since the reductions keep members, rescale by positive factors and never synthesize pairs, so its ratio, when defined, lies in $[\underline{P}, \overline{P}]$. Hence $\underline{P} \leq \underline{P}_\varepsilon \leq (1+\varepsilon)\underline{P}$ and $\overline{P}/(1+\varepsilon) \leq \overline{P}_\varepsilon \leq \overline{P}$. □

Lemma 14 records, for the classical single-function algorithm, why dominance pruning removes nothing before the query: without evidence every intermediate function is a conditional table, and distinct conditional tables never dominate one another.

Lemma 14 (Evidence-free potentials are conditional tables)**.** *Run bucket elimination on the credal network with no evidence, along any elimination order, starting from the potentials $\phi_X = \{\theta_X\}$ whose functions are the vertex choices of X's rows. For every potential ϕ created during elimination there is a partition of its scope into a* child block C_ϕ *and a* parent block R_ϕ*, determined by the elimination history alone, such that every $f \in \phi$ satisfies $\sum_{\mathbf{c}} f(\mathbf{c}, \mathbf{r}) = 1$ for every assignment $\mathbf{r}$ of R_ϕ. Consequently no function of ϕ is strictly dominated by another.*

Proof. Let $\mathcal{A}_\phi$ be the set of nodes whose own table has been multiplied into ϕ, $C_\phi = \mathcal{A}_\phi \cap \mathrm{scope}(\phi)$ and $R_\phi = \mathrm{scope}(\phi) \setminus C_\phi$. A function of ϕ is $f = \sum_{\mathbf{x}_{\mathcal{A}_\phi \setminus \mathrm{scope}(\phi)}} \prod_{X \in \mathcal{A}_\phi} \theta_X$ for one vertex choice of every table.

Step 1 (summed nodes are absorbed). A node W is summed out only in its own bucket, which contains every potential mentioning W, in particular the one carrying θ_W. So every node summed out in ϕ's history lies in $\mathcal{A}_\phi$.

Step 2 (outside parents are in scope). Let $W \notin \mathcal{A}_\phi$ be a parent of some $X \in \mathcal{A}_\phi$. The factor θ_X mentions W, and by Step 1 W has not been summed out, so $W \in R_\phi$.

Step 3 (summing a product of tables). For a fixed assignment of the parents outside $\mathcal{A}_\phi$, sum $\prod_{X \in \mathcal{A}_\phi} \theta_X$ over all nodes of $\mathcal{A}_\phi$, from the topologically last to the first. At each step the remaining factors belong to topologically earlier nodes, none of which has the current node X as a parent, so they are constant in x_X, and $\sum_{x_X} \theta_X(x_X \mid \pi) = 1$. The total is 1.

Step 4. Summing f over C_ϕ completes the sum of Step 3 (the nodes of $\mathcal{A}_\phi \setminus \mathrm{scope}(\phi)$ are already summed), with the outside parents fixed by $\mathbf{r}$ (Step 2). So $\sum_{\mathbf{c}} f(\mathbf{c}, \mathbf{r}) = 1$, with the same split for all functions of ϕ.

Step 5 (no strict dominance). If $g \geq f$ pointwise with $g \neq f$, pick $\mathbf{r}$ with $g(\cdot, \mathbf{r}) \neq f(\cdot, \mathbf{r})$; then $\sum_{\mathbf{c}} g(\mathbf{c}, \mathbf{r}) > \sum_{\mathbf{c}} f(\mathbf{c}, \mathbf{r}) = 1$, a contradiction. □

The lemma concerns the unscaled functions. The implementation stores positive multiples of them (rescaled to $\max f = 1$, Definition 15), which are not literally conditional tables; since every ratio $R_{u,v}$ is invariant to positive rescaling of a pair (Lemma 12(1)), the lemma applies to the unscaled functions that the stored ones represent, and the computed bounds are unaffected.

G Detailed Description: Interval Belief Propagation

G.1 Factor Graph and Credal Messages

Given the credal network $\langle D, \mathcal{K} \rangle$, the factor graph has one variable node per node V (with domain Ω_V, of size $2^{|\mathrm{atoms}(V)|}$ for a chain-graph component), one *family factor* f_V with scope $\{V\} \cup \mathrm{Pa}(V)$ storing the vertex sets $\mathrm{ext}K_{V,\pi}$ of every row, and one *evidence factor* e_V for every V with observed atoms, storing the $0/1$ indicator δ_V over Ω_V of the states that agree with every observed atom of V. On a polytree credal network the factor graph is a tree.

Definition 18 (Credal message)**.** *A **credal message** on a node V is a finite set $M \subset \Delta(\Omega_V)$ of normalized vectors, stored as the vertices of its convex hull; it represents the set* $\mathrm{conv}(M)$*. The empty set is the* empty message*. For a binary node a nonempty message is an interval on the second coordinate.*

For a fixed choice of rows, the sum-product messages equation 6 are defined up to scale. As the rows range over their local sets, the normalized messages range over a set, which IVBP represents by its hull vertices. All messages start at the neutral message $\{\mathbf{1}/|\Omega_V|\}$; an evidence factor always sends $\{\delta_V / \sum \delta_V\}$.

G.2 Message Updates

Write $\mathrm{nrm}(m) = m/\sum m$ and $\mathrm{hull}(\cdot)$ for the reduction to hull vertices. Vectors m with $\sum m = 0$ are dropped (regular conditioning: the evidence has probability zero for that choice); an update whose candidates are all dropped returns the empty message.

Variable to factor (product). For the incoming credal messages $M_1, \ldots, M_r$ on V from the factors other than the receiving one,

$$M_{V \to f} = \mathrm{hull}\big\{\mathrm{nrm}(m_1 \circ \cdots \circ m_r) : m_j \in M_j\big\},$$

computed pairwise with a hull reduction after each product; it is empty if some M_j is.

Factor to child (π-message). For each *corner* $(q_1, \ldots, q_k)$, one vertex of the message of each parent $U_1, \ldots, U_k$, the weights $w(\pi) = \prod_j q_j(\pi_j)$ sum to one, and

$$\mu_{f_V \to V} = \sum_{\pi} w(\pi)\, \theta_{V,\pi}, \qquad \theta_{V,\pi} \in \mathrm{ext}K_{V,\pi},$$

so the set of these messages is the Minkowski sum $\sum_\pi w(\pi)\, \mathrm{ext}K_{V,\pi}$, already normalized (each row sums to one). Its vertices are sums of row vertices and are computed incrementally, one row with $w(\pi) > 0$ at a time, with a hull reduction after each addition. $M_{f_V \to V}$ is the hull of the union over corners; it is empty if some parent message is.

Factor to parent (λ-message). For the parent $U = U_i$ and each corner (λ, q_{-i}), one vertex λ of the message from V into f_V (the product of V's other incoming messages, including evidence) and one vertex of each other parent's message,

$$m(u) = \sum_{\pi : \pi_i = u} w_{-i}(\pi)\, \langle \lambda, \theta_{V,\pi} \rangle, \qquad w_{-i}(\pi) = \prod_{j \neq i} q_j(\pi_j).$$

Each coordinate $m(u)$ depends only on the rows with $\pi_i = u$, so as the rows range over their local sets, m ranges over the box $\prod_u [\underline{m}(u), \overline{m}(u)]$, with $\underline{m}(u) = \sum_{\pi:\pi_i=u} w_{-i}(\pi) \min_{v \in \mathrm{ext}K_{V,\pi}} \langle \lambda, v \rangle$ and $\overline{m}(u)$ with $\max$. Only the coordinates with $\underline{m}(u) < \overline{m}(u)$ are varied, so a corner yields at most $2^{|\Omega_U|}$ box corners. $M_{f_V \to U}$ is the hull of the normalized box corners of positive mass over all corners (λ, q_{-i}). If none has positive mass, the evidence below the edge is impossible for every choice of rows, and the message is empty (`IntervalBP._factor_to_parent`).

Beliefs and atom bounds. The belief B_V of V is the product of all its incoming credal messages, including evidence, and the bounds of an atom a of V are $\min_b \sum_{s \models a} b(s)$ and $\max_b \sum_{s \models a} b(s)$ over the belief's vertices b. The functional is linear, so its extrema over the hull are attained at vertices, and compound nodes need no further projection. If some belief is empty, IVBP reports $[0, 1]$ for every atom and does not flag the result as exact.

Algorithm 3 Interval Belief Propagation (IVBP)

Require: credal network $\langle D, \mathcal{K} \rangle$ (from the chain-graph factorization of $\mathcal{L}$); evidence $\mathbf{e}$; sweeps T, tolerance τ
Ensure: $[\underline{P}(a \mid \mathbf{e}), \overline{P}(a \mid \mathbf{e})]$ for every non-evidence atom a
build the factor graph: family factors f_V, evidence factors e_V
$M_{f \to v} \leftarrow \{\mathbf{1}/|\Omega_v|\}$ for every edge
for $t = 1, \ldots, T$ ($\max(T, 2m+2)$ on a polytree) **do**
  for each V in topological order (t odd) or reverse order (t even) **do**
    $M_{U \to f_V} \leftarrow$ product of the messages into U except from f_V, for $U \in \mathrm{Pa}(V)$, and $M_{V \to f_V}$ likewise
    $M_{f_V \to V} \leftarrow \pi$-message; $M_{f_V \to U} \leftarrow \lambda$-message, for $U \in \mathrm{Pa}(V)$
  if no message changed by more than τ (10^{-12} on a polytree) **then**
    break
$B_V \leftarrow$ product of all messages into V, including e_V
if some B_V is empty **then**
  return $[0, 1]$ for every atom
return $\left[\min_{b \in B_V} \sum_{s \models a} b(s),\ \max_{b \in B_V} \sum_{s \models a} b(s)\right]$ for each atom a of each V

Schedule. The factors are updated in topological order, then in reverse topological order, alternating, until a sweep changes no message (Hausdorff distance of the vertex sets at most 10^{-12}; at most $\max(T, 2m+2)$ sweeps). On a loopy graph the tolerance is $\tau = 10^{-6}$ and at most $T = 100$ sweeps are run. Message vertices are merged at 12 decimals, and an update that would enumerate more than 200 000 corners raises an error.

G.3 Running Example

Consider the chain $A \to B \to C$ over binary atoms with $P(A) \in [0.4, 0.6]$, $P(B \mid A) \in [0.8, 0.9]$, $P(B \mid \neg A) \in [0.1, 0.2]$, $P(C \mid B) \in [0.5, 0.7]$ and $P(C \mid \neg B) \in [0.2, 0.3]$. Every row has two vertices, the endpoints of its interval, and all messages are intervals on $P(\cdot{=}1)$.

No evidence. The π-messages are $f_A \to A = [0.4, 0.6]$, $f_B \to B = [0.1 + 0.7 \cdot 0.4,\ 0.2 + 0.7 \cdot 0.6] = [0.38, 0.62]$ and $f_C \to C = [0.314, 0.548]$, the extremes of $P(C{=}1 \mid \neg B) + (P(C{=}1 \mid B) - P(C{=}1 \mid \neg B))\, P(B{=}1)$ over the endpoints. Each λ-message is the neutral $\{(\frac{1}{2}, \frac{1}{2})\}$, because every row sums to one, so the beliefs are the π-messages: the exact intervals. The forward sweep computes them and a second sweep confirms the fixed point.

Evidence $C{=}1$. The evidence factor sends $(0, 1)$ into C, so $\lambda = (0, 1)$ in f_C, and $m(b) = P(C{=}1 \mid b)$. The box is $[0.2, 0.3] \times [0.5, 0.7]$, and its normalized corners give $f_C \to B = [0.5/0.8,\ 0.7/0.9] = [0.625, 0.778]$. In f_B each vertex λ of this message gives $m(a) = \langle \lambda, P(B \mid a) \rangle$, and the corners give $f_B \to A = [0.575, 0.722]$. The beliefs are the products with the π-messages:

$$P(A{=}1 \mid C{=}1) \in [0.4742, 0.7959], \qquad P(B{=}1 \mid C{=}1) \in [0.5053, 0.851].$$

The lower bound of A combines the lower endpoints, $\frac{0.4 \cdot 0.575}{0.4 \cdot 0.575 + 0.6 \cdot 0.425} = 0.4742$, and the upper one the upper endpoints. Both agree with the enumeration of all $2 \cdot 4 \cdot 4$ vertex products; three sweeps are run. The previous downward-only scheme returned $P(A{=}1 \mid C{=}1) \in [0.4, 0.6]$.

Compound nodes. In `examples/ivbp_compound_mid.lcn` the chain-graph component $\{B, C\}$ is a 4-state node with parent U and children D, E. Its row given $U{=}1$ has 10 vertices (the exact row polytope), and the π-message into it has 19. With evidence $C{=}0$ the evidence factor of B-C selects the states $\{00, 10\}$, and the λ-message to U moves $P(U{=}1)$ from $[0.3, 0.6]$ to $[0.109, 1]$, matching the enumeration of all vertex products. The examples `examples/ivbp_compound_parents.lcn` (two compound parents of one child) and `alarm.lcn` (compound leaf C-D) are checked in the test suite in the same way.

Zero evidence probability. In `examples/ivbp_zero_evidence.lcn` ($A \to B$, $A \to C$, $P(B \mid A) = P(B \mid \neg A) = 0$) with evidence $B{=}1$, every box corner of the λ-message from f_B to A has zero mass, the message is empty, and IVBP returns $[0, 1]$ for A and C, as CVE and CDVE do.

G.4 Proofs of Theorems 5 and 6

Proof of Theorem 5. Step 1 (π-message). The message of f_V to V enumerates the C_V^π corners of the parent messages. For each corner it adds the rows one at a time; adding a row to a partial hull of h vectors forms $h \cdot e_V$ sums of length at most d, so a corner costs $O(M_V e_V d)$, with M_V the total size of the partial hulls.

Step 2 (λ-message). The message of f_V to U enumerates the $C_{V,U}^\lambda$ corners of the child message and of the other parents' messages. For each corner it evaluates one inner product of length at most d per row vertex, $O(E_V d)$, and normalizes at most $2^{|\Omega_U|} \le 2^d$ box corners, $O(2^d d)$.

Step 3 (sweep). A sweep updates every family factor once: one π-message and one λ-message per parent. The variable-to-factor products and the hull reductions are extra. C_V^π, $C_{V,U}^\lambda$ and M_V are products and sums of message and row vertex counts, and can be exponential in the number of parents and in d.

Step 4 (number of sweeps). Define the depth of a factor-to-variable message $f \to V$ as 1 if its update reads no family-factor message (e.g. a root's π-message), and otherwise as 1 plus the largest depth of the family-factor messages $g \to W$, $W \in \mathcal{N}(f) \setminus \{V\}$, $g \neq f$, that it reads (through the variable-to-factor messages, which have the depth of their deepest input). On a tree the depth is at most the number D of family factors on a path of the factor graph, so $D \le m$. If every message of depth less than t has its final value at the end of sweep $t-1$, then every message of depth t is recomputed in sweep t from final inputs and is final at its end, whatever the order. By induction, D sweeps reach the fixed point, and the next sweep changes nothing and stops the loop; $D+1 \le m+1 < 2m+2$, so the cap is never reached on a polytree. Alternating the direction of the sweeps lets one sweep finalize a whole chain in its direction: on all polytree benchmarks ($n \le 100$) and on a zigzag polytree of colliders we observed at most four sweeps, the last confirming the fixed point. □

Lemma 15 (Projective images of hulls)**.** *Let $X_1, \dots, X_r$ be arbitrary (possibly infinite) sets of non-negative vectors and F a map, linear in each argument separately, with non-negative values on non-negative arguments. Let $g = \mathrm{nrm} \circ F$ on the points where $\sum F > 0$, and for a set S of argument tuples write $S^+ = \{x \in S : \sum F(x) > 0\}$. Then*

$$\mathrm{conv}\, g\big((\mathrm{conv}X_1 \times \cdots \times \mathrm{conv}X_r)^+\big) = \mathrm{conv}\, g\big((X_1 \times \cdots \times X_r)^+\big).$$

In particular, if $\sum F > 0$ everywhere, no point is excluded, and if the right-hand set is empty, so is the left-hand one.

Proof. Step 1 ($r = 1$). A point of $\mathrm{conv}X_1$ is a finite convex combination $x = \sum_k \lambda_k x_k$ with $x_k \in X_1$. By linearity $F(x) = \sum_k \lambda_k F(x_k)$, so $\sum F(x) = \sum_k \lambda_k \sum F(x_k)$. If $\sum F(x) > 0$, then $g(x) = \sum_k \mu_k g(x_k)$ over the k with $\sum F(x_k) > 0$, with $\mu_k = \lambda_k \sum F(x_k) / \sum F(x) \ge 0$ summing to one, as in Lemma 3(b); a term with $\sum F(x_k) = 0$ has $F(x_k) = 0$ (non-negative values) and contributes nothing. So $g((\mathrm{conv}X_1)^+) \subseteq \mathrm{conv}\, g(X_1^+)$; the reverse inclusion of the hulls holds because $X_1^+ \subseteq (\mathrm{conv}X_1)^+$.

Step 2 ($r > 1$). Let $x = (x_1, \dots, x_r)$ be in the left-hand product with $\sum F(x) > 0$. Fix $x_2, \dots, x_r$; the map $F(\cdot, x_2, \dots, x_r)$ is linear and non-negative, so by Step 1, $g(x) \in \mathrm{conv}\, g(\{(y_1, x_2, \dots, x_r) : y_1 \in X_1\}^+)$. For each such point fix y_1 and apply Step 1 in the second argument, and so on. Since the hull of a union of hulls is the hull of the union, after r steps $g(x) \in \mathrm{conv}\, g((X_1 \times \cdots \times X_r)^+)$. The reverse inclusion is again immediate. □

Proof of Theorem 6. Let the compiled credal network be a polytree, so its factor graph is a tree. Write θ for a choice of one row $\theta_{V,\pi} \in K_{V,\pi}$ for every node and parent assignment; the rows are chosen independently (separate specification). For fixed θ let μ^θ be the precise sum-product messages equation 6, including the evidence factors.

Step 1 (parameter blocks). Removing the edge between a factor f and a variable V splits the tree in two. Let $\Theta_{f \to V}$ be the rows of the family factors on f's side, f included, and $\Theta_{V \to f}$ those on V's side. $\mu^\theta_{f \to V}$ depends only on $\Theta_{f \to V}$ and $\mu^\theta_{V \to f}$ only on $\Theta_{V \to f}$. At a factor f with neighbours $V, W_1, \dots, W_k$, the blocks $\Theta_{W_j \to f}$ and the rows of f are pairwise disjoint, with union $\Theta_{f \to V}$; at a variable V with neighbouring factors $g_1, \dots, g_l$ the blocks $\Theta_{g_j \to V}$ are pairwise disjoint. Hence, as θ ranges over all choices, the inputs of every update range independently over their own sets.

Step 2 (true message sets). Every update equation 6 is linear in each incoming message and in each row, so rescaling an input rescales the output, and an input of zero mass gives an output of zero mass. For a directed edge e let $S_e = \{\mathrm{nrm}(\mu_e^\theta) : \sum \mu_e^\theta > 0\}$, possibly empty. By Step 1 and the scale invariance, if an update F has inputs on the edges $e_1, \dots, e_r$ and the rows of f, then $S_e = g\big((S_{e_1} \times \cdots \times S_{e_r} \times \prod_\pi K_{V,\pi})^+\big)$ with $g = \mathrm{nrm} \circ F$.

Step 3 (sandwich). Let M_e be the message IVBP stores at the fixed point (Step 6). We show, by induction on the depth of e (Theorem 5, Step 4),

$$M_e \subseteq S_e \subseteq \operatorname{conv} M_e.$$

Base. An evidence factor's message is the point $\mathrm{nrm}(\delta_V)$, attained by every θ. A root's π-message is $\mathrm{hull}(\mathrm{ext}K_{V,()})$, and $\mathrm{ext}K_{V,()} \subseteq K_{V,()} \subseteq \operatorname{conv}\mathrm{ext}K_{V,()}$. A variable with no other incoming factor sends the empty product, the neutral message, which is its precise value.

Upper inclusion. By the induction hypothesis $S_{e_j} \subseteq \operatorname{conv} M_{e_j}$, and $K_{V,\pi} = \operatorname{conv}\mathrm{ext}K_{V,\pi}$. By Step 2 and Lemma 15, $S_e \subseteq g((\operatorname{conv}M_{e_1} \times \cdots \times \prod_\pi \operatorname{conv}\mathrm{ext}K_{V,\pi})^+) \subseteq \operatorname{conv} g((M_{e_1} \times \cdots \times \prod_\pi \mathrm{ext}K_{V,\pi})^+)$. IVBP computes the vertices of this hull. For the product update this is the set of normalized vertex products; the intermediate hull reductions do not change the hull, by Lemma 15 applied to each pairwise product. For the π-message, with the parent vertices fixed, the image is the Minkowski sum of the weighted row-vertex sets, and $\operatorname{conv}(A + B) = \operatorname{conv}(\operatorname{conv}A + B)$ justifies the incremental reduction. For the λ-message to U, with λ and the other parents' vertices fixed, $m(u)$ is a sum of linear functions of the rows with $\pi_U = u$, and distinct u use distinct rows; as the rows range over the convex sets $K_{V,\pi}$, each $m(u)$ ranges over the whole interval $[\underline{m}(u), \overline{m}(u)]$ and the coordinates are independent, so m ranges over the whole box. The box is the hull of its corners, so by Lemma 15 (with F the identity) the hull of its normalized image is the hull of the normalized corners of positive mass.

Lower inclusion: every computed vertex is attained. A vertex of a product is $\mathrm{nrm}(m_1 \circ \cdots \circ m_r)$ with $m_j \in M_{e_j} \subseteq S_{e_j}$, each attained on its own block; the blocks are disjoint, so one θ attains all of them, and the product has positive mass, so the vertex lies in S_e. A vertex of a π-message is $\sum_\pi w(\pi)v_\pi$ with $v_\pi \in \mathrm{ext}K_{V,\pi}$ and the parent vertices attained on disjoint blocks: choose $\theta_{V,\pi} = v_\pi$. A box corner chooses, for every u, the lower or the upper end of $m(u)$; it is attained by giving every row with $\pi_U = u$ its vertex minimizing (or maximizing) $\langle \lambda, \cdot \rangle$, which is possible because distinct u use distinct rows, together with θs attaining λ and the other parents' vertices on their disjoint blocks. Corners of zero mass are dropped, so every kept corner is in S_e. If all candidates are dropped, $M_e = \varnothing$, and by the upper inclusion $S_e = \varnothing$ too, so the empty message is exact; an empty input makes the output empty on both sides.

Step 4 (beliefs). On a tree, $\prod_g \mu^\theta_{g \to V}(x) = p_\theta(V{=}x, \mathbf{e})$, so the normalized belief is $p_\theta(V \mid \mathbf{e})$ whenever $p_\theta(\mathbf{e}) > 0$. The incoming messages of V depend on disjoint blocks, so the argument of Step 3 gives $B_V \subseteq B_V^\star \subseteq \operatorname{conv} B_V$ for $B_V^\star = \{p_\theta(V \mid \mathbf{e}) : p_\theta(\mathbf{e}) > 0\}$. For the linear functional $\ell_a(b) = \sum_{s \models a} b(s)$, $\min_{B_V} \ell_a = \min_{\operatorname{conv} B_V} \ell_a \le \inf_{B_V^\star} \ell_a \le \min_{B_V} \ell_a$, so the computed lower bound is $\inf\{p_\theta(a \mid \mathbf{e}) : p_\theta \in \mathcal{P}_{\mathrm{SE}},\, p_\theta(\mathbf{e}) > 0\}$, and likewise for the upper bound.

Step 5 (from $\mathcal{P}_{\mathrm{SE}}$ to $\mathcal{D}_{\mathrm{SE}}$; $\overline{P}(\mathbf{e}) = 0$). If $\overline{P}(\mathbf{e}) > 0$, the bounds over $\mathcal{P}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ are those over $\mathcal{D}_{\mathrm{SE}} \cap \{p(\mathbf{e}) > 0\}$ by Lemma 11 (equivalently, by Lemma 3(b) with $\mathcal{M} = \mathcal{P}_{\mathrm{SE}}$, compact by Lemma 3(a)); moreover $B_V^\star \neq \varnothing$, hence $B_V \neq \varnothing$, for every V, so no belief is empty. If $\overline{P}(\mathbf{e}) = 0$, every $B_V^\star$ is empty, hence every B_V, and IVBP returns $[0, 1]$, the convention of Definition 7. Conversely, an empty belief certifies $\overline{P}(\mathbf{e}) = 0$. Compound atoms are handled by ℓ_a directly.

Step 6 (fixed point). By Theorem 5, Step 4, the stored messages reach the values defined by the depth recursion after at most D sweeps, and the next sweep certifies it. Conversely, a sweep without change means that the stored messages satisfy the update equations, whose solution on a tree is unique (it is determined by the depth recursion), so a converged run has the messages of Step 3.

Step 7 ($\mathcal{D}_{\mathrm{LCN}}$, box fallback, precision). Under the hypotheses of Proposition 6 with exact local sets, $\mathcal{P}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$, so the bounds of Step 4 are those over $\mathcal{D}_{\mathrm{LCN}}$. With a box fallback the argument holds for the compiled sets, and the bounds are outer for $\mathcal{D}_{\mathrm{SE}}$ on the exact sets, as in Theorem 4, Step 5. The argument is exact for the enumerated vertices; the computed values carry their floating-point error and the merge at 12 decimals. □

Algorithm 4 Coordinate Descent with Variable Elimination (CDVE)

Require: credal network $\langle D, \mathcal{K} \rangle$ (from the chain-graph factorization of $\mathcal{L}$); evidence $\mathbf{e}$; sweeps T; restarts R
Ensure: inner bounds $[\underline{p}_a, \overline{p}_a]$ on $P(a \mid \mathbf{e})$ for every non-evidence atom a

for each node Z and each non-evidence atom a of Z **do**
  $\mathcal{R} \leftarrow$ ancestors of $\{Z\} \cup \{\text{evidence nodes}\}$, themselves included
  for sense $\in \{\min, \max\}$ and start $j = 0, \ldots, R$ **do**
    $r \leftarrow$ vertex averages ($j = 0$) or random vertices ($j > 0$) of every row
    for sweep $t = 1, \ldots, T$ **do**
      for each $V \in \mathcal{R}$ with a row of more than one vertex **do**
        $T_V \leftarrow$ VE of all factors but f_V, with the evidence indicators, onto $\{V\} \cup \mathrm{Pa}(V) \cup \{Z\}$
        $n_\pi \leftarrow \sum_{s \models a} T_V(\cdot, \pi, s), \quad d_\pi \leftarrow \sum_s T_V(\cdot, \pi, s)$
        $r_V \leftarrow$ Dinkelbach step on $\sum_\pi \langle n_\pi, r_\pi \rangle / \sum_\pi \langle d_\pi, r_\pi \rangle$ (sense)
      if no family changed **then break**
    $p_j \leftarrow p_r(a \mid \mathbf{e})$ if $p_r(\mathbf{e}) > 0$; otherwise discard the start
  $\underline{p}_a \leftarrow \min_j p_j$ (sense min), $\quad \overline{p}_a \leftarrow \max_j p_j$ (sense max); $[0, 1]$ if every start was discarded

Each hypothesis is used. On a loopy structure the inputs of an update share parameters and cannot be chosen independently (Step 1 fails), so the computed sets are neither inner nor outer in general. With multi-valued nodes the vertex counts C_V^π, $C_{V,U}^\lambda$ and M_V can grow exponentially, consistently with the NP-hardness of exact polytree inference for ternary variables (Mauá et al., 2014); the theorem bounds the error, not the cost. For binary nodes every message has at most two vertices, and the output coincides with that of 2U.

H Detailed Description: Coordinate Descent with Variable Elimination

CDVE (Algorithm 4) is a coordinate-descent method over the local credal sets in the spirit of ApproxLP (Antonucci et al., 2015), which iteratively solves the linear program obtained by fixing all local credal sets but one. The defaults are $R = 8$ restarts and $T = 50$ sweeps.

Objective. For an atom a of Z (a state set $S_a \subseteq \Omega_Z$ for a compound Z) and evidence $\mathbf{e}$ (an indicator over each observed node's states, which for a compound node selects the states that agree with its observed atoms),

$$p(a \mid \mathbf{e}) = \frac{\sum_{s \in S_a} p(Z{=}s, \mathbf{e})}{\sum_{s \in \Omega_Z} p(Z{=}s, \mathbf{e})}.$$

Families outside the ancestral set $\mathcal{R}$ of $\{Z\} \cup \{\text{evidence nodes}\}$ sum out to one, so they are dropped.

The family step. Every term of $p(Z, \mathbf{e})$ contains exactly one entry of V's table, so with the other families fixed

$$p(Z{=}s, \mathbf{e}) = \sum_{x, \pi} \theta_{V,\pi}(x)\, T_V(x, \pi, s),$$

with T_V the product of all other factors and of the evidence indicators, eliminated onto $\{V\} \cup \mathrm{Pa}(V) \cup \{Z\}$ (if $Z = V$ or $Z \in \mathrm{Pa}(V)$, that axis is shared). The objective is the ratio $N(r)/D(r)$ of $N = \sum_\pi \langle n_\pi, r_\pi \rangle$ and $D = \sum_\pi \langle d_\pi, r_\pi \rangle$, with $0 \le n_\pi \le d_\pi$, over the rows $r_\pi \in K_{V,\pi}$. Since the network is separately specified, the rows range independently. The step precomputes, for every vertex v of every row, the numbers $\langle n_\pi, v \rangle$ and $\langle d_\pi, v \rangle$. Starting from the current ratio $t_0 = N/D$ (the step is skipped if $D = 0$), Dinkelbach's iteration sets every row to the vertex maximizing $\langle n_\pi - t\, d_\pi, v \rangle$ (minimizing, for $\min$) and $t \leftarrow N/D$, until $F(t) = \max_r (N - tD) \le 10^{-15} \max(1, D_0)$, at most 100 times. This is the optimum of the local program, attained at a vertex of every row. Since $0 \le N \le D$ and $t \ge 0$, a positive F (for $\max$: $N > tD \ge 0$; for $\min$: $tD > N \ge 0$) implies $D > 0$, so the ratio stays defined. The move is accepted if it strictly improves t_0, by more than $10^{-12} \max(1, |t_0|)$.

Starts. The first start puts every row at its *vertex average*, the mean of the row's vertices (not the centroid of the polytope). $p(\mathbf{e})$ is multilinear in the independently chosen rows, so its value at the vertex averages is the average of its values over all vertex products, which is positive iff some vertex product gives $\mathbf{e}$ positive probability, i.e. iff $\overline{P}(\mathbf{e}) > 0$ (Lemma 11). At this symmetric point an uninformative child row (for instance $P(C \mid B)$ at $(\frac{1}{2}, \frac{1}{2})$) can hide every move of its parent's family, so a family whose rows are not yet vertices moves to a best vertex choice even when the ratio stays equal, provided it does not decrease and $D > 0$. Each row does so at most once, so this cannot cycle. The other R starts choose one vertex per row uniformly at random, with a fixed seed. Every accepted move keeps $p(\mathbf{e}) = D > 0$, so a start with $p(\mathbf{e}) > 0$ ends at a member of $\mathcal{P}_{\mathrm{SE}}$ with $p(\mathbf{e}) > 0$. A start whose point has $p(\mathbf{e}) = 0$ makes no move (every step is skipped) and is discarded. Hence, if $\overline{P}(\mathbf{e}) > 0$ the vertex-average start survives and the result is the best value over the surviving starts, an inner bound; CDVE returns $[0, 1]$ exactly when every start is discarded, i.e. when $\overline{P}(\mathbf{e}) = 0$. Coordinate descent can stop at a local optimum: reaching the optimum may require changing two families at once.

Output. $[\underline{p}_a, \overline{p}_a]$ for every non-evidence atom; for a compound node, the per-state range of the posteriors $p_r(Z \mid \mathbf{e})$ at the optima found.

H.1 Detailed Example

Consider the chain $A \to B \to C$ of the IVBP example, with $P(A) \in [0.4, 0.6]$, $P(B \mid A) \in [0.8, 0.9]$, $P(B \mid \neg A) \in [0.1, 0.2]$, $P(C \mid B) \in [0.5, 0.7]$, $P(C \mid \neg B) \in [0.2, 0.3]$ and evidence $C{=}1$. All three families are ancestors of $\{A, C\}$. We trace the start at the vertex averages, $P(A{=}1) = 0.5$, $P(B{=}1 \mid A) = 0.85$, $P(B{=}1 \mid \neg A) = 0.15$, $P(C{=}1 \mid B) = 0.6$, $P(C{=}1 \mid \neg B) = 0.25$.

Sense $\max$**, sweep 1.**

- *Family A.* $T_A(a) = P(C{=}1 \mid a)$ at the current rows, so $n = (0,\ 0.5475)$ and $d = (0.3025,\ 0.5475)$ over $A = (0, 1)$. Of the two vertices, $P(A) = (0.4, 0.6)$ gives the larger ratio, $0.6 \cdot 0.5475 / (0.4 \cdot 0.3025 + 0.6 \cdot 0.5475) = 0.7308$.
- *Family B.* The coefficients make the step pick the rows $P(B{=}1 \mid A) = 0.9$ and $P(B{=}1 \mid \neg A) = 0.1$, giving 0.7483.
- *Family C.* The step picks $P(C{=}1 \mid B) = 0.7$ and $P(C{=}1 \mid \neg B) = 0.2$, giving 0.7959.

Sweep 2. No family improves, so the run stops with $\overline{p}_A = 0.7959$. Sense $\min$ picks the other vertices and stops at $\underline{p}_A = 0.4742$.

Result. $P(A{=}1 \mid C{=}1) \in [0.4742,\ 0.7959]$, which here is the exact interval: enumerating all $2 \cdot 4 \cdot 4$ vertex products gives the same values. Without evidence the same run returns the exact $P(A{=}1) \in [0.4, 0.6]$ after one improving sweep and one confirming sweep.

H.2 Proof of Theorem 7

Proof. Step 1 (family step). One variable elimination over the at most m factors costs $O(m\, d^{w^*+1})$, where w^* is the induced width of the cached min-fill order that keeps $\{V\} \cup \mathrm{Pa}(V) \cup \{Z\}$, counting the kept nodes, so that every table has at most d^{w^*+1} entries. Precomputing $\langle n_\pi, v\rangle$ and $\langle d_\pi, v\rangle$ for the E_V row vertices costs $O(E_V d)$. Each of the I Dinkelbach iterations then takes an argmax over the precomputed numbers of every row and two sums, $O(E_V)$.

Step 2 (counting). A sweep performs at most one step per family, and there are at most T sweeps per start. Each start ends with one posterior elimination, $O(m\, d^{w^*+1})$. There are $R{+}1$ starts per sense, two senses and A atoms. Multiplying gives the bound. The space is that of one elimination, $O(m\, d^{w^*+1})$, besides the input.

Step 3 (I is finite). For $\max$, $F(t) > 0$ gives a new ratio $N/D > t$ (and symmetrically for $\min$), so t moves strictly and takes values among the finitely many ratios of vertex choices of the rows; the iteration therefore stops. The implementation caps it at 100 iterations, and a capped step still returns a visited vertex choice with $D > 0$, so the result stays inner.

Step 4 (scope of the guarantee). Every kept point is a member of $\mathcal{P}_{\mathrm{SE}}$ with $p(\mathbf{e}) > 0$ (paragraph *Starts*), so the result is inner for $\mathcal{D}_{\mathrm{SE}}$; it is inner for $\mathcal{D}_{\mathrm{LCN}}$ only when $\mathcal{D}_{\mathrm{SE}}$ and $\mathcal{D}_{\mathrm{LCN}}$ have the same bounds (e.g. Proposition 2). The guarantee is relative to the compiled local sets: if the exact row-polytope computation of a compound row fails, the compiler falls back to the (outer) box for that row and reports it, and CDVE is then not guaranteed inner for $\mathcal{D}_{\mathrm{SE}}$ built on the exact sets. □

I Detailed Experimental Evaluation

This appendix reports every measurement behind Section 5: all four random families at both scales, and the complete per-instance results for the two realistic suites. Throughout, the reference is CredalJT and the budget is 3600 seconds and 30 GB per run.

I.1 A reference that is exact by construction

All approximate methods are scored against CJT, which is exact on an instance only when two conditions hold. (i) *Structural*: the junction tree that CJT builds satisfies H1–H4 of Theorem 8 — running intersection, checked explicitly; every sentence hosted; every LMC assertion discharged by separator-entailment, an exact reduction or the ordered rows, with nothing relaxed (Section B.2); every atom hosted — and its largest cluster has at most 12 atoms. This needs no solve (`cjt_exactness_report`, well under a second per instance). (ii) *Certified*: each endpoint is certified by SCIP (proven optimal, or primal and dual bounds within 10^{-4}, Remark 3).

Condition (i) holds on every benchmark instance: all 480 random, all 260 almost-realizable (Appendix I.2) and all 26 realistic instances, each of which is also a chain graph in the strict sense of Section 2.3. The generator can enforce (i) at generation time (`cjt_exact="structural"`, recorded in a header comment); it did so for the almost-realizable instances, but the random instances were generated without the gate, and (i) was checked afterwards. Condition (ii) is not enforced by the generator, since it is the reference computation itself, and it does not hold everywhere at a 60 s limit per solve: on a sample of 293 instances, 54 had at least one uncertified endpoint, among them 4 of the 60 `dag_small` instances. We therefore score each endpoint separately, and use a CJT endpoint as the reference only when its solve is certified *and* its route is structurally exact; the uncertified endpoints are reported by their (sound) dual bounds but are excluded from every error statistic. A candidate fails (i) only when its tree exceeds the width budget or an assertion would have to be relaxed, for example when an LMC assertion splits an undirected chain component, as in the `smokers` example.

I.2 Almost-realizable benchmarks

To measure the relaxation error of the credal-network compilation under control, we perturb fully realizable instances by adding non-realizable sentences (`Generator.perturb`). The new sentences come in three kinds, one for each way a sentence can fail Definition 4:

- `nr1`, a *non-root marginal*: a Type-1 sentence $P(l)$ with l a literal of an atom that has parents (it fails R1);
- `nr2`, a *mixture conditional*: a Type-2 sentence $P(l \mid \psi)$ with ψ over the child's parents, satisfied by at least two but not all of their configurations (it fails R2, as it bounds a $P(\mathrm{pa}_v)$-weighted mixture of rows). It needs at least two parents, so `tree` instances never get it;
- `nr3`, a *cross-family* sentence: a Type-1 sentence over two atoms that share no family scope, with $\tau = \mathrm{False}$, so it adds no structure edge.

None of the kinds changes the structure graph or the LMC. Each new sentence s gets the bounds $[v_s - \delta,\ v_s + \delta]$, clipped to $[0, 1]$, around its value v_s under the base instance's fully independent product witness. That witness still satisfies every sentence and the LMC, so every perturbed instance is consistent by construction. (The witness makes every atom independent, so all sentences on the same child, such as two `nr2` conditionals, are centred on its marginal.) We checked every shipped instance: each added sentence is non-realizable and of its stated kind, the structure graph and the LMC are unchanged, and condition (i) of Appendix I.1 holds.

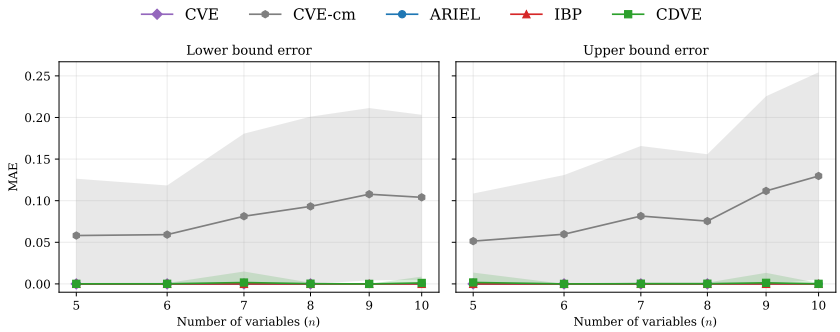
Sets. The bases are the `tree` and `dag` random instances with $n \in \{10, 20\}$ (10 per size). For each base we draw one sequence of 8 sentences, with the kinds in round-robin order (`nr1`, `nr2`, `nr3`, …; `nr1`, `nr3`, … on trees). The instance with m new sentences keeps the first m, for $m \in \{0, 1, 2, 4, 8\}$ and $\delta = 0.1$, so the instances are nested and $m = 0$ is the base itself (100 `tree` and 100 `dag` instances). An ablation set isolates each kind: `dag` bases with $n = 10$, $m = 4$ sentences of a single kind, and $\delta \in \{0.05, 0.3\}$ (60 instances). Clipping to $[0, 1]$ affects at least one sentence in 98 of the 260 instances. Every perturbed file records its base, m, δ, the kinds of its new sentences and its structural CJT report in a header comment.

I.3 Coverage and exclusions

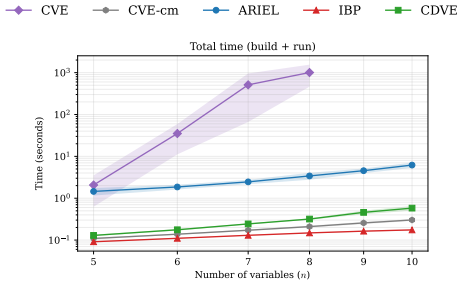
Not every algorithm completes on every instance, and reading the figures without this in mind would overstate coverage, since a curve simply stops where its algorithm ran out of budget. Over the 480 random instances, ARIEL and IVBP complete essentially all of them, and CredalJT's `ipopt` backend completes all of the instances it was run on. The exceptions are concentrated in the CVE family. Plain CVE, without clustering, is the least scalable: it solves $35/60$ of `tree_small`, $30/60$ of `polytree_small` and $24/60$ of `dag_small`, but *none* of `ktree_small` (21 timeouts and 39 errors) and none of `polytree_large`; on the other three large families it was not run at all. It is therefore absent from most plots, and where present it covers only the smallest sizes. Clustering restores coverage at small scale — CVE_{cm} and CVE_{cp} solve all 60 instances of every small family — but degrades on the large loopy ones, where CVE_{cm} reaches only $14/60$ on `ktree_large` and $30/60$ on `dag_large`. Exact inference over the 2^n joint is viable only at the very smallest sizes ($15/60$ on `tree_small`, $13/60$ on `polytree_small`, $2/10$ on the Bayesian-network instances, none of the Supreme Court ones), which is why it is not used as the reference. Three bookkeeping gaps are worth recording: the `ktree_large` CDVE log holds 58 records rather than 60, the Supreme Court exact-inference log holds 12 rather than 16, and the Bayesian-network CVE log holds 7 rather than 10; the corresponding instances are simply missing rather than failed, and are excluded from the aggregates.

I.4 Random LCNs

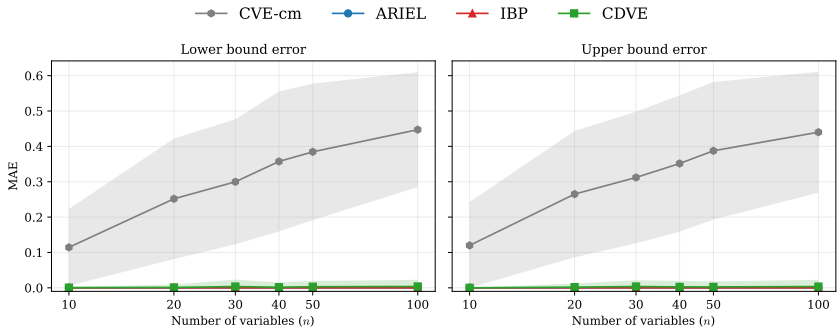
Figures 5–8 give the error and timing curves for all four families at both scales. The tables that follow them give the underlying per-size aggregates; for each family and scale the first table reports means and the second the corresponding standard deviations over the ten instances at each size.



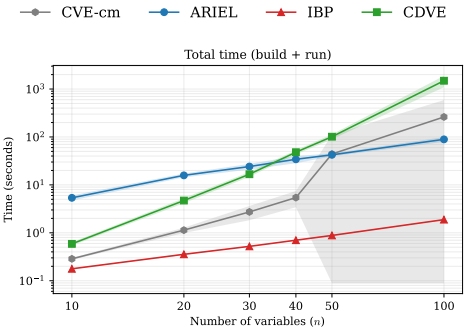
(a) small, error



(b) small, time

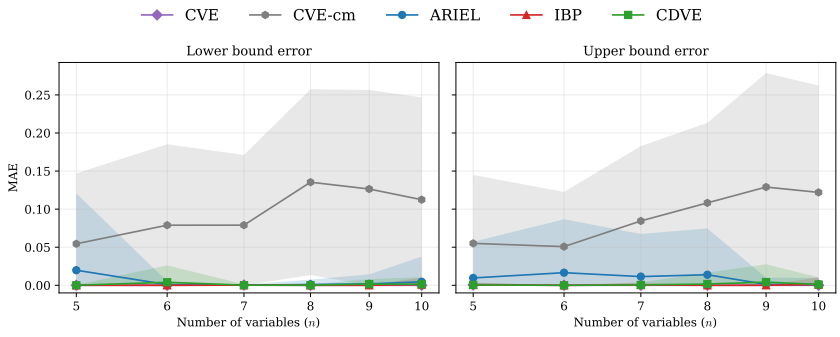


(c) large, error

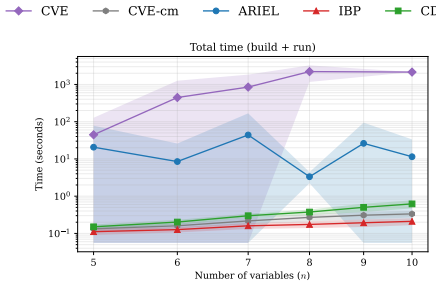


(d) large, time

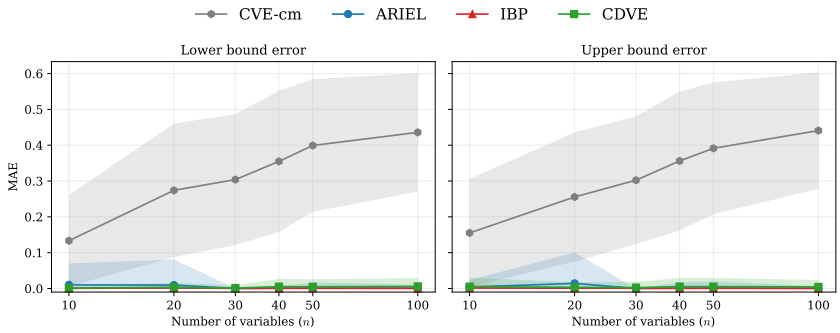
Figure 5: Random `tree` LCNs. Singly connected and fully realizable, so the credal-network relaxation is tight: ARIEL and IVBP are indistinguishable from the reference.



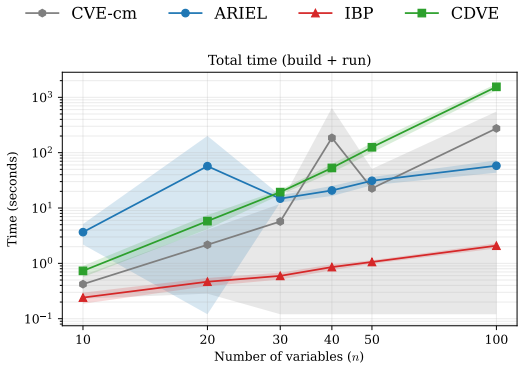
(a) small, error



(b) small, time

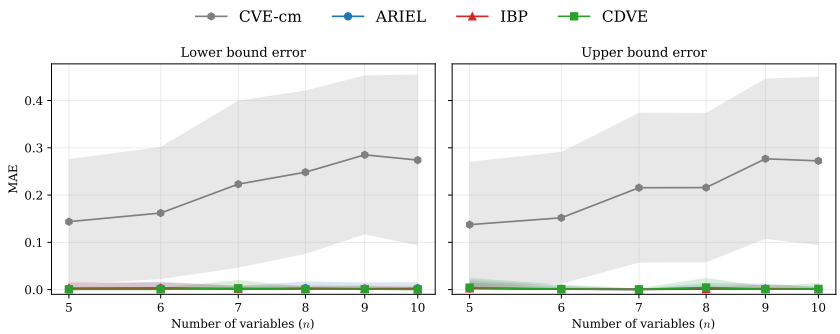


(c) large, error

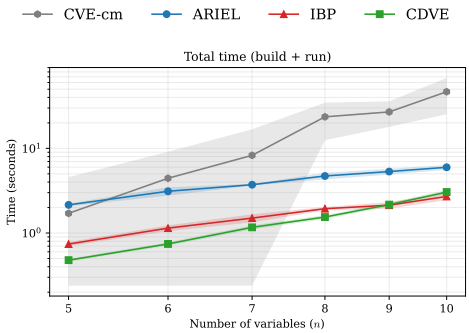


(d) large, time

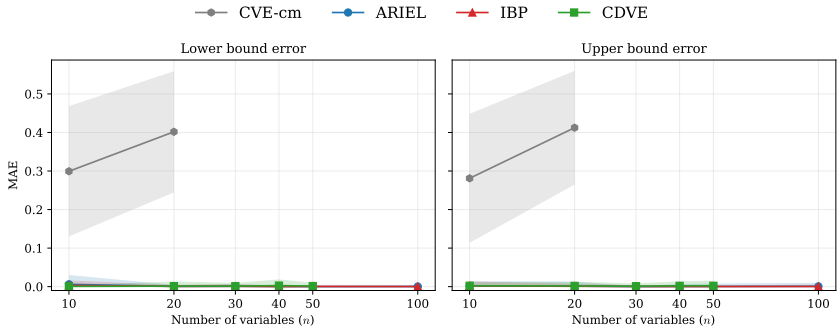
Figure 6: Random `polytree` LCNs. The primal structure is a polytree, but $18/60$ small and $22/60$ large instances compile to a loopy credal network, on which IVBP is not guaranteed exact; the residual ARIEL error at the small scale comes from instances where the reference itself is the tighter engine.



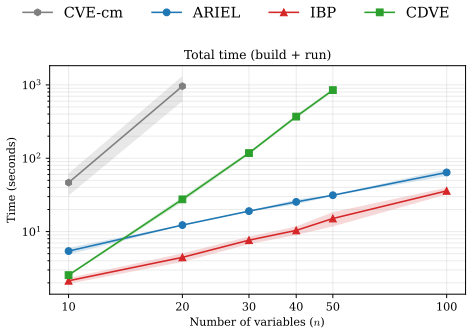
(a) small, error



(b) small, time

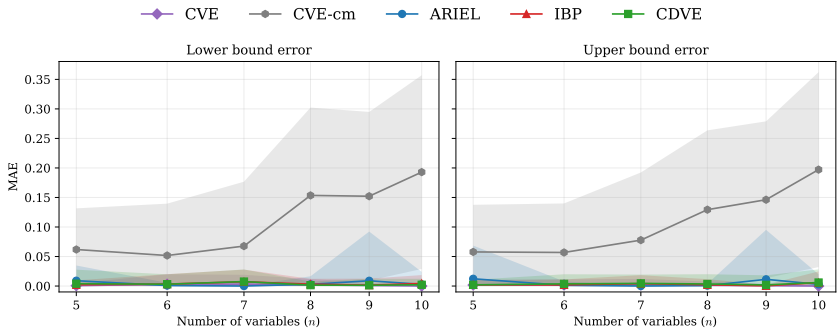


(c) large, error

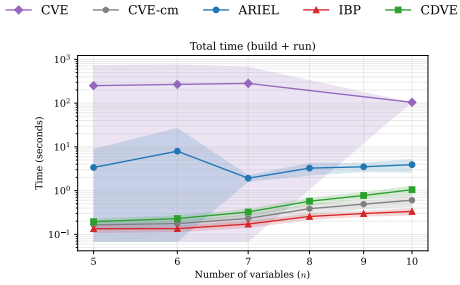


(d) large, time

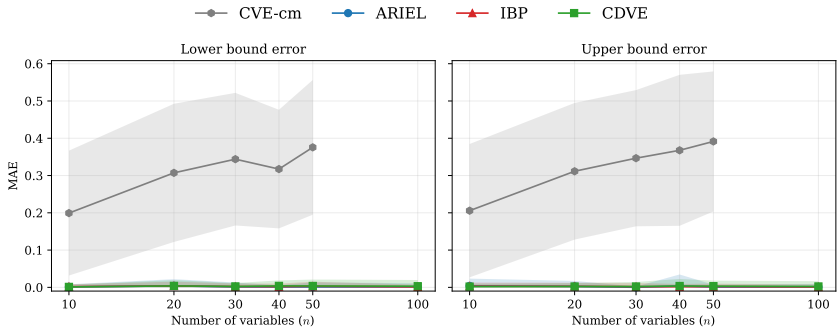
Figure 7: Random `ktree` LCNs (k=3). Loopy despite bounded treewidth, so all credal-network methods give outer bounds only; plain CVE solves none of these instances and CVE$_{cm}$ stops after n=20.



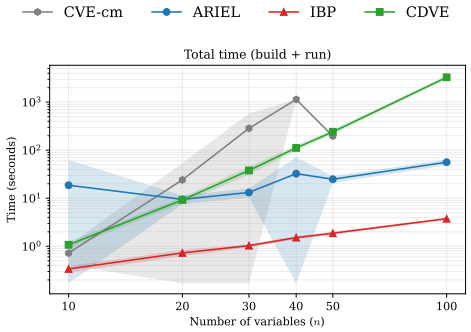
(a) small, error



(b) small, time



(c) large, error



(d) large, time

Figure 8: Random `dag` LCNs (moralized treewidth ≤ 3, at most two parents). The least structured family; CDVE's runtime grows fastest here, exceeding 600 seconds at n=100.

Table 7: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
tree_fr	5	CDVE	0.0001	0.0017	0.08	0.04	0.13	0.31	1.00
tree_fr	5	ARIEL	0.0000	0.0000	0.00	1.45	1.45	0.31	–
tree_fr	5	CVE	0.0000	0.0000	0.10	1.97	2.07	0.31	1.00
tree_fr	5	CVE_{cm}	0.0581	0.0514	0.09	0.02	0.11	0.31	1.00
tree_fr	5	IBP	0.0000	0.0000	0.09	0.00	0.09	0.31	1.00
tree_fr	6	CDVE	0.0001	0.0000	0.10	0.08	0.18	0.28	1.00
tree_fr	6	ARIEL	0.0000	0.0000	0.00	1.86	1.86	0.28	–
tree_fr	6	CVE	0.0000	0.0000	0.12	35.01	35.13	0.28	1.00
tree_fr	6	CVE_{cm}	0.0592	0.0597	0.10	0.04	0.14	0.28	1.00
tree_fr	6	IBP	0.0000	0.0000	0.11	0.00	0.11	0.28	1.00
tree_fr	7	CDVE	0.0016	0.0002	0.12	0.12	0.24	0.33	1.00
tree_fr	7	ARIEL	0.0000	0.0000	0.00	2.46	2.46	0.33	–
tree_fr	7	CVE	0.0000	0.0000	0.12	511.13	511.25	0.33	1.00
tree_fr	7	CVE_{cm}	0.0813	0.0815	0.12	0.05	0.17	0.33	1.00
tree_fr	7	IBP	0.0000	0.0000	0.13	0.00	0.13	0.33	1.00
tree_fr	8	CDVE	0.0002	0.0002	0.14	0.18	0.32	0.39	1.00
tree_fr	8	ARIEL	0.0000	0.0000	0.00	3.41	3.41	0.39	–
tree_fr	8	CVE	0.0000	0.0000	0.14	1006.64	1006.78	0.39	1.00
tree_fr	8	CVE_{cm}	0.0931	0.0754	0.14	0.07	0.21	0.39	1.00
tree_fr	8	IBP	0.0000	0.0000	0.14	0.01	0.15	0.39	1.00
tree_fr	9	CDVE	0.0000	0.0013	0.15	0.31	0.46	0.44	1.00
tree_fr	9	ARIEL	0.0000	0.0000	0.00	4.55	4.55	0.44	–
tree_fr	9	CVE_{cm}	0.1078	0.1117	0.16	0.10	0.26	0.44	1.00
tree_fr	9	IBP	0.0000	0.0000	0.16	0.01	0.16	0.44	1.00
tree_fr	10	CDVE	0.0011	0.0001	0.17	0.41	0.58	0.50	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	6.19	6.19	0.50	–
tree_fr	10	CVE_{cm}	0.1040	0.1297	0.18	0.12	0.30	0.50	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.17	0.50	1.00

Table 8: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
tree_fr	5	CDVE	0.0005	0.0119	0.08	0.04	0.13	0.31	1.00
tree_fr	5	ARIEL	0.0000	0.0000	0.00	1.45	1.45	0.31	–
tree_fr	5	CVE	0.0000	0.0000	0.10	1.97	2.07	0.31	1.00
tree_fr	5	CVE_{cm}	0.0683	0.0572	0.09	0.02	0.11	0.31	1.00
tree_fr	5	IBP	0.0000	0.0000	0.09	0.00	0.09	0.31	1.00
tree_fr	6	CDVE	0.0009	0.0000	0.10	0.08	0.18	0.28	1.00
tree_fr	6	ARIEL	0.0000	0.0000	0.00	1.86	1.86	0.28	–
tree_fr	6	CVE	0.0000	0.0000	0.12	35.01	35.13	0.28	1.00
tree_fr	6	CVE_{cm}	0.0591	0.0711	0.10	0.04	0.14	0.28	1.00
tree_fr	6	IBP	0.0000	0.0000	0.11	0.00	0.11	0.28	1.00
tree_fr	7	CDVE	0.0133	0.0014	0.12	0.12	0.24	0.33	1.00
tree_fr	7	ARIEL	0.0000	0.0000	0.00	2.46	2.46	0.33	–
tree_fr	7	CVE	0.0000	0.0000	0.12	511.13	511.25	0.33	1.00
tree_fr	7	CVE_{cm}	0.0993	0.0842	0.12	0.05	0.17	0.33	1.00
tree_fr	7	IBP	0.0000	0.0000	0.13	0.00	0.13	0.33	1.00
tree_fr	8	CDVE	0.0013	0.0014	0.14	0.18	0.32	0.39	1.00
tree_fr	8	ARIEL	0.0000	0.0000	0.00	3.41	3.41	0.39	–
tree_fr	8	CVE	0.0000	0.0000	0.14	1006.64	1006.78	0.39	1.00
tree_fr	8	CVE_{cm}	0.1078	0.0804	0.14	0.07	0.21	0.39	1.00
tree_fr	8	IBP	0.0000	0.0000	0.14	0.01	0.15	0.39	1.00
tree_fr	9	CDVE	0.0000	0.0120	0.15	0.31	0.46	0.44	1.00
tree_fr	9	ARIEL	0.0000	0.0000	0.00	4.55	4.55	0.44	–
tree_fr	9	CVE_{cm}	0.1035	0.1138	0.16	0.10	0.26	0.44	1.00
tree_fr	9	IBP	0.0000	0.0000	0.16	0.01	0.16	0.44	1.00
tree_fr	10	CDVE	0.0076	0.0009	0.17	0.41	0.58	0.50	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	6.19	6.19	0.50	–
tree_fr	10	CVE_{cm}	0.0992	0.1246	0.18	0.12	0.30	0.50	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.17	0.50	1.00

Table 9: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
tree_fr	10	CDVE	0.0005	0.0000	0.18	0.41	0.59	0.64	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	5.37	5.37	0.64	–
tree_fr	10	CVE_{cm}	0.1145	0.1199	0.18	0.11	0.29	0.64	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.18	0.64	1.00
tree_fr	20	CDVE	0.0011	0.0019	0.35	4.36	4.71	1.43	1.00
tree_fr	20	ARIEL	0.0000	0.0000	0.00	15.80	15.80	1.43	–
tree_fr	20	CVE_{cm}	0.2517	0.2650	0.37	0.77	1.14	1.43	1.00
tree_fr	20	IBP	0.0000	0.0000	0.34	0.02	0.36	1.43	1.00
tree_fr	30	CDVE	0.0036	0.0037	0.50	16.18	16.68	2.58	1.00
tree_fr	30	ARIEL	0.0000	0.0000	0.00	24.13	24.13	2.58	–
tree_fr	30	CVE_{cm}	0.2998	0.3120	0.55	2.18	2.73	2.58	1.00
tree_fr	30	IBP	0.0000	0.0000	0.49	0.03	0.52	2.58	1.00
tree_fr	40	CDVE	0.0017	0.0026	0.66	47.39	48.04	3.90	1.00
tree_fr	40	ARIEL	0.0000	0.0000	0.00	34.21	34.21	3.90	–
tree_fr	40	CVE_{cm}	0.3573	0.3515	0.70	4.72	5.43	3.90	1.00
tree_fr	40	IBP	0.0000	0.0000	0.66	0.04	0.70	3.90	1.00
tree_fr	50	CDVE	0.0031	0.0026	0.83	100.39	101.22	5.95	1.00
tree_fr	50	ARIEL	0.0000	0.0000	0.00	42.41	42.41	5.95	–
tree_fr	50	CVE_{cm}	0.3845	0.3875	0.96	43.09	44.05	5.95	1.00
tree_fr	50	IBP	0.0000	0.0000	0.83	0.05	0.88	5.95	1.00
tree_fr	100	CDVE	0.0037	0.0036	1.67	1487.61	1489.29	16.24	1.00
tree_fr	100	ARIEL	0.0000	0.0000	0.00	89.09	89.09	16.24	–
tree_fr	100	CVE_{cm}	0.4473	0.4401	1.94	259.37	261.31	16.24	1.00
tree_fr	100	IBP	0.0000	0.0000	1.76	0.12	1.88	16.24	1.00

Table 10: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
tree_fr	10	CDVE	0.0026	0.0003	0.18	0.41	0.59	0.64	1.00
tree_fr	10	ARIEL	0.0000	0.0000	0.00	5.37	5.37	0.64	–
tree_fr	10	CVE_{cm}	0.1085	0.1227	0.18	0.11	0.29	0.64	1.00
tree_fr	10	IBP	0.0000	0.0000	0.17	0.01	0.18	0.64	1.00
tree_fr	20	CDVE	0.0083	0.0104	0.35	4.36	4.71	1.43	1.00
tree_fr	20	ARIEL	0.0000	0.0000	0.00	15.80	15.80	1.43	–
tree_fr	20	CVE_{cm}	0.1704	0.1786	0.37	0.77	1.14	1.43	1.00
tree_fr	20	IBP	0.0000	0.0000	0.34	0.02	0.36	1.43	1.00
tree_fr	30	CDVE	0.0195	0.0177	0.50	16.18	16.68	2.58	1.00
tree_fr	30	ARIEL	0.0000	0.0000	0.00	24.13	24.13	2.58	–
tree_fr	30	CVE_{cm}	0.1766	0.1860	0.55	2.18	2.73	2.58	1.00
tree_fr	30	IBP	0.0000	0.0000	0.49	0.03	0.52	2.58	1.00
tree_fr	40	CDVE	0.0134	0.0169	0.66	47.39	48.04	3.90	1.00
tree_fr	40	ARIEL	0.0000	0.0000	0.00	34.21	34.21	3.90	–
tree_fr	40	CVE_{cm}	0.1977	0.1929	0.70	4.72	5.43	3.90	1.00
tree_fr	40	IBP	0.0000	0.0000	0.66	0.04	0.70	3.90	1.00
tree_fr	50	CDVE	0.0164	0.0154	0.83	100.39	101.22	5.95	1.00
tree_fr	50	ARIEL	0.0000	0.0000	0.00	42.41	42.41	5.95	–
tree_fr	50	CVE_{cm}	0.1929	0.1944	0.96	43.09	44.05	5.95	1.00
tree_fr	50	IBP	0.0000	0.0000	0.83	0.05	0.88	5.95	1.00
tree_fr	100	CDVE	0.0189	0.0190	1.67	1487.61	1489.29	16.24	1.00
tree_fr	100	ARIEL	0.0000	0.0000	0.00	89.09	89.09	16.24	–
tree_fr	100	CVE_{cm}	0.1625	0.1711	1.94	259.37	261.31	16.24	1.00
tree_fr	100	IBP	0.0000	0.0000	1.76	0.12	1.88	16.24	1.00

Table 11: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
polytree_fr	5	CDVE	0.0001	0.0005	0.10	0.05	0.15	0.39	1.50
polytree_fr	5	ARIEL	0.0198	0.0096	0.00	20.68	20.68	0.39	–
polytree_fr	5	CVE	0.0000	0.0005	0.11	44.41	44.52	0.39	1.50
polytree_fr	5	CVE_{cm}	0.0545	0.0550	0.11	0.03	0.13	0.39	1.50
polytree_fr	5	IBP	0.0000	0.0005	0.11	0.01	0.11	0.39	1.50
polytree_fr	6	CDVE	0.0039	0.0000	0.12	0.08	0.20	0.44	1.40
polytree_fr	6	ARIEL	0.0010	0.0166	0.00	8.51	8.51	0.44	–
polytree_fr	6	CVE	0.0000	0.0000	0.14	445.58	445.71	0.44	1.40
polytree_fr	6	CVE_{cm}	0.0789	0.0508	0.12	0.04	0.16	0.44	1.40
polytree_fr	6	IBP	0.0000	0.0000	0.12	0.01	0.13	0.44	1.40
polytree_fr	7	CDVE	0.0002	0.0004	0.15	0.15	0.30	0.73	1.70
polytree_fr	7	ARIEL	0.0002	0.0114	0.00	44.04	44.04	0.73	–
polytree_fr	7	CVE	0.0003	0.0000	0.13	849.79	849.92	0.73	1.40
polytree_fr	7	CVE_{cm}	0.0790	0.0845	0.15	0.06	0.22	0.73	1.70
polytree_fr	7	IBP	0.0002	0.0004	0.15	0.01	0.16	0.73	1.70
polytree_fr	8	CDVE	0.0001	0.0017	0.16	0.22	0.38	0.91	1.40
polytree_fr	8	ARIEL	0.0010	0.0139	0.00	3.33	3.33	0.91	–
polytree_fr	8	CVE	0.0000	0.0000	0.15	2213.21	2213.37	0.91	1.00
polytree_fr	8	CVE_{cm}	0.1354	0.1082	0.18	0.09	0.27	0.91	1.40
polytree_fr	8	IBP	0.0000	0.0000	0.17	0.01	0.17	0.91	1.40
polytree_fr	9	CDVE	0.0015	0.0042	0.18	0.32	0.50	0.99	1.30
polytree_fr	9	ARIEL	0.0019	0.0012	0.00	26.20	26.20	0.99	–
polytree_fr	9	CVE_{cm}	0.1264	0.1290	0.19	0.12	0.31	0.99	1.30
polytree_fr	9	IBP	0.0000	0.0000	0.18	0.01	0.19	0.99	1.30
polytree_fr	10	CDVE	0.0014	0.0011	0.19	0.43	0.62	1.11	1.30
polytree_fr	10	ARIEL	0.0046	0.0011	0.00	11.47	11.47	1.11	–
polytree_fr	10	CVE	0.0000	0.0000	0.16	2149.12	2149.28	1.11	1.00
polytree_fr	10	CVE_{cm}	0.1124	0.1220	0.20	0.14	0.33	1.11	1.30
polytree_fr	10	IBP	0.0010	0.0010	0.20	0.01	0.21	1.11	1.30

Table 12: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
polytree_fr	5	CDVE	0.0004	0.0029	0.10	0.05	0.15	0.39	1.50
polytree_fr	5	ARIEL	0.1010	0.0475	0.00	20.68	20.68	0.39	–
polytree_fr	5	CVE	0.0001	0.0029	0.11	44.41	44.52	0.39	1.50
polytree_fr	5	CVE_{cm}	0.0925	0.0898	0.11	0.03	0.13	0.39	1.50
polytree_fr	5	IBP	0.0001	0.0029	0.11	0.01	0.11	0.39	1.50
polytree_fr	6	CDVE	0.0222	0.0000	0.12	0.08	0.20	0.44	1.40
polytree_fr	6	ARIEL	0.0042	0.0702	0.00	8.51	8.51	0.44	–
polytree_fr	6	CVE	0.0000	0.0000	0.14	445.58	445.71	0.44	1.40
polytree_fr	6	CVE_{cm}	0.1063	0.0718	0.12	0.04	0.16	0.44	1.40
polytree_fr	6	IBP	0.0000	0.0000	0.12	0.01	0.13	0.44	1.40
polytree_fr	7	CDVE	0.0012	0.0032	0.15	0.15	0.30	0.73	1.70
polytree_fr	7	ARIEL	0.0009	0.0560	0.00	44.04	44.04	0.73	–
polytree_fr	7	CVE	0.0017	0.0001	0.13	849.79	849.92	0.73	1.40
polytree_fr	7	CVE_{cm}	0.0922	0.0982	0.15	0.06	0.22	0.73	1.70
polytree_fr	7	IBP	0.0012	0.0032	0.15	0.01	0.16	0.73	1.70
polytree_fr	8	CDVE	0.0011	0.0150	0.16	0.22	0.38	0.91	1.40
polytree_fr	8	ARIEL	0.0064	0.0608	0.00	3.33	3.33	0.91	–
polytree_fr	8	CVE	0.0000	0.0000	0.15	2213.21	2213.37	0.91	1.00
polytree_fr	8	CVE_{cm}	0.1220	0.1051	0.18	0.09	0.27	0.91	1.40
polytree_fr	8	IBP	0.0000	0.0000	0.17	0.01	0.17	0.91	1.40
polytree_fr	9	CDVE	0.0067	0.0234	0.18	0.32	0.50	0.99	1.30
polytree_fr	9	ARIEL	0.0126	0.0088	0.00	26.20	26.20	0.99	–
polytree_fr	9	CVE_{cm}	0.1301	0.1497	0.19	0.12	0.31	0.99	1.30
polytree_fr	9	IBP	0.0000	0.0001	0.18	0.01	0.19	0.99	1.30
polytree_fr	10	CDVE	0.0093	0.0094	0.19	0.43	0.62	1.11	1.30
polytree_fr	10	ARIEL	0.0331	0.0088	0.00	11.47	11.47	1.11	–
polytree_fr	10	CVE	0.0000	0.0000	0.16	2149.12	2149.28	1.11	1.00
polytree_fr	10	CVE_{cm}	0.1343	0.1404	0.20	0.14	0.33	1.11	1.30
polytree_fr	10	IBP	0.0092	0.0094	0.20	0.01	0.21	1.11	1.30

Table 13: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
polytree_fr	10	CDVE	0.0010	0.0050	0.23	0.51	0.73	13.77	1.50
polytree_fr	10	ARIEL	0.0100	0.0038	0.00	3.66	3.66	13.77	–
polytree_fr	10	CVE_{cm}	0.1333	0.1551	0.22	0.20	0.42	13.77	1.50
polytree_fr	10	IBP	0.0003	0.0013	0.23	0.01	0.24	13.77	1.50
polytree_fr	20	CDVE	0.0031	0.0031	0.45	5.35	5.80	3.21	1.60
polytree_fr	20	ARIEL	0.0095	0.0143	0.00	57.07	57.07	3.21	–
polytree_fr	20	CVE_{cm}	0.2740	0.2555	0.44	1.73	2.17	3.21	1.60
polytree_fr	20	IBP	0.0012	0.0006	0.43	0.03	0.47	3.21	1.60
polytree_fr	30	CDVE	0.0012	0.0023	0.54	18.84	19.38	5.22	1.10
polytree_fr	30	ARIEL	0.0000	0.0000	0.00	14.78	14.78	5.22	–
polytree_fr	30	CVE_{cm}	0.3039	0.3023	0.58	5.12	5.71	5.22	1.10
polytree_fr	30	IBP	0.0001	0.0003	0.56	0.04	0.59	5.22	1.10
polytree_fr	40	CDVE	0.0051	0.0052	0.76	52.22	52.98	12.10	1.40
polytree_fr	40	ARIEL	0.0007	0.0009	0.00	20.76	20.76	12.10	–
polytree_fr	40	CVE_{cm}	0.3548	0.3560	0.78	184.60	185.37	12.10	1.40
polytree_fr	40	IBP	0.0003	0.0002	0.80	0.06	0.86	12.10	1.40
polytree_fr	50	CDVE	0.0047	0.0051	1.10	124.13	125.23	38.34	1.50
polytree_fr	50	ARIEL	0.0008	0.0012	0.00	31.07	31.07	38.34	–
polytree_fr	50	CVE_{cm}	0.3992	0.3913	0.93	21.67	22.60	38.34	1.50
polytree_fr	50	IBP	0.0005	0.0003	0.99	0.07	1.06	38.34	1.50
polytree_fr	100	CDVE	0.0056	0.0043	1.95	1545.82	1547.77	68.64	1.40
polytree_fr	100	ARIEL	0.0005	0.0003	0.00	58.08	58.08	68.64	–
polytree_fr	100	CVE_{cm}	0.4359	0.4408	1.86	272.65	274.51	68.64	1.33
polytree_fr	100	IBP	0.0002	0.0002	1.93	0.15	2.08	68.64	1.40

Table 14: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
polytree_fr	10	CDVE	0.0041	0.0251	0.23	0.51	0.73	13.77	1.50
polytree_fr	10	ARIEL	0.0598	0.0200	0.00	3.66	3.66	13.77	–
polytree_fr	10	CVE_{cm}	0.1278	0.1507	0.22	0.20	0.42	13.77	1.50
polytree_fr	10	IBP	0.0013	0.0105	0.23	0.01	0.24	13.77	1.50
polytree_fr	20	CDVE	0.0127	0.0143	0.45	5.35	5.80	3.21	1.60
polytree_fr	20	ARIEL	0.0714	0.0862	0.00	57.07	57.07	3.21	–
polytree_fr	20	CVE_{cm}	0.1862	0.1804	0.44	1.73	2.17	3.21	1.60
polytree_fr	20	IBP	0.0060	0.0043	0.43	0.03	0.47	3.21	1.60
polytree_fr	30	CDVE	0.0076	0.0161	0.54	18.84	19.38	5.22	1.10
polytree_fr	30	ARIEL	0.0000	0.0000	0.00	14.78	14.78	5.22	–
polytree_fr	30	CVE_{cm}	0.1829	0.1783	0.58	5.12	5.71	5.22	1.10
polytree_fr	30	IBP	0.0014	0.0027	0.56	0.04	0.59	5.22	1.10
polytree_fr	40	CDVE	0.0218	0.0244	0.76	52.22	52.98	12.10	1.40
polytree_fr	40	ARIEL	0.0052	0.0162	0.00	20.76	20.76	12.10	–
polytree_fr	40	CVE_{cm}	0.1969	0.1938	0.78	184.60	185.37	12.10	1.40
polytree_fr	40	IBP	0.0019	0.0019	0.80	0.06	0.86	12.10	1.40
polytree_fr	50	CDVE	0.0210	0.0242	1.10	124.13	125.23	38.34	1.50
polytree_fr	50	ARIEL	0.0160	0.0190	0.00	31.07	31.07	38.34	–
polytree_fr	50	CVE_{cm}	0.1852	0.1840	0.93	21.67	22.60	38.34	1.50
polytree_fr	50	IBP	0.0049	0.0022	0.99	0.07	1.06	38.34	1.50
polytree_fr	100	CDVE	0.0235	0.0195	1.95	1545.82	1547.77	68.64	1.40
polytree_fr	100	ARIEL	0.0100	0.0060	0.00	58.08	58.08	68.64	–
polytree_fr	100	CVE_{cm}	0.1651	0.1624	1.86	272.65	274.51	68.64	1.33
polytree_fr	100	IBP	0.0025	0.0025	1.93	0.15	2.08	68.64	1.40

Table 15: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	5	CDVE	0.0006	0.0042	0.33	0.14	0.48	0.29	3.00
ktree_fr_k3	5	ARIEL	0.0015	0.0028	0.00	2.15	2.15	0.29	–
ktree_fr_k3	5	CVE_{cm}	0.1437	0.1374	0.33	1.38	1.71	0.29	3.00
ktree_fr_k3	5	IBP	0.0030	0.0027	0.33	0.41	0.74	0.29	3.00
ktree_fr_k3	6	CDVE	0.0006	0.0012	0.45	0.29	0.74	0.36	3.00
ktree_fr_k3	6	ARIEL	0.0029	0.0011	0.00	3.11	3.11	0.36	–
ktree_fr_k3	6	CVE_{cm}	0.1620	0.1519	0.45	3.99	4.43	0.36	3.00
ktree_fr_k3	6	IBP	0.0030	0.0008	0.45	0.69	1.14	0.36	3.00
ktree_fr_k3	7	CDVE	0.0027	0.0005	0.57	0.59	1.17	0.42	3.00
ktree_fr_k3	7	ARIEL	0.0015	0.0002	0.00	3.72	3.72	0.42	–
ktree_fr_k3	7	CVE_{cm}	0.2231	0.2155	0.60	7.68	8.27	0.42	3.00
ktree_fr_k3	7	IBP	0.0018	0.0005	0.58	0.92	1.50	0.42	3.00
ktree_fr_k3	8	CDVE	0.0008	0.0044	0.64	0.90	1.54	0.51	3.00
ktree_fr_k3	8	ARIEL	0.0028	0.0020	0.00	4.72	4.72	0.51	–
ktree_fr_k3	8	CVE_{cm}	0.2484	0.2158	0.69	22.88	23.57	0.51	3.00
ktree_fr_k3	8	IBP	0.0018	0.0011	0.71	1.23	1.94	0.51	3.00
ktree_fr_k3	9	CDVE	0.0011	0.0009	0.78	1.39	2.17	0.60	3.00
ktree_fr_k3	9	ARIEL	0.0019	0.0017	0.00	5.32	5.32	0.60	–
ktree_fr_k3	9	CVE_{cm}	0.2851	0.2768	0.82	26.13	26.95	0.60	3.00
ktree_fr_k3	9	IBP	0.0015	0.0020	0.76	1.38	2.13	0.60	3.00
ktree_fr_k3	10	CDVE	0.0002	0.0015	0.95	2.10	3.05	0.68	3.00
ktree_fr_k3	10	ARIEL	0.0027	0.0011	0.00	5.98	5.98	0.68	–
ktree_fr_k3	10	CVE_{cm}	0.2740	0.2723	0.96	45.69	46.65	0.68	3.00
ktree_fr_k3	10	IBP	0.0013	0.0009	0.97	1.74	2.70	0.68	3.00

Table 16: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	5	CDVE	0.0021	0.0206	0.33	0.14	0.48	0.29	3.00
ktree_fr_k3	5	ARIEL	0.0105	0.0171	0.00	2.15	2.15	0.29	–
ktree_fr_k3	5	CVE_{cm}	0.1324	0.1330	0.33	1.38	1.71	0.29	3.00
ktree_fr_k3	5	IBP	0.0125	0.0122	0.33	0.41	0.74	0.29	3.00
ktree_fr_k3	6	CDVE	0.0034	0.0094	0.45	0.29	0.74	0.36	3.00
ktree_fr_k3	6	ARIEL	0.0137	0.0071	0.00	3.11	3.11	0.36	–
ktree_fr_k3	6	CVE_{cm}	0.1399	0.1394	0.45	3.99	4.43	0.36	3.00
ktree_fr_k3	6	IBP	0.0119	0.0037	0.45	0.69	1.14	0.36	3.00
ktree_fr_k3	7	CDVE	0.0174	0.0022	0.57	0.59	1.17	0.42	3.00
ktree_fr_k3	7	ARIEL	0.0063	0.0013	0.00	3.72	3.72	0.42	–
ktree_fr_k3	7	CVE_{cm}	0.1767	0.1585	0.60	7.68	8.27	0.42	3.00
ktree_fr_k3	7	IBP	0.0069	0.0027	0.58	0.92	1.50	0.42	3.00
ktree_fr_k3	8	CDVE	0.0050	0.0202	0.64	0.90	1.54	0.51	3.00
ktree_fr_k3	8	ARIEL	0.0144	0.0114	0.00	4.72	4.72	0.51	–
ktree_fr_k3	8	CVE_{cm}	0.1728	0.1580	0.69	22.88	23.57	0.51	3.00
ktree_fr_k3	8	IBP	0.0071	0.0041	0.71	1.23	1.94	0.51	3.00
ktree_fr_k3	9	CDVE	0.0067	0.0057	0.78	1.39	2.17	0.60	3.00
ktree_fr_k3	9	ARIEL	0.0131	0.0110	0.00	5.32	5.32	0.60	–
ktree_fr_k3	9	CVE_{cm}	0.1681	0.1694	0.82	26.13	26.95	0.60	3.00
ktree_fr_k3	9	IBP	0.0068	0.0089	0.76	1.38	2.13	0.60	3.00
ktree_fr_k3	10	CDVE	0.0012	0.0127	0.95	2.10	3.05	0.68	3.00
ktree_fr_k3	10	ARIEL	0.0129	0.0052	0.00	5.98	5.98	0.68	–
ktree_fr_k3	10	CVE_{cm}	0.1808	0.1783	0.96	45.69	46.65	0.68	3.00
ktree_fr_k3	10	IBP	0.0054	0.0039	0.97	1.74	2.70	0.68	3.00

Table 17: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	10	CDVE	0.0007	0.0023	0.40	2.14	2.55	0.65	3.00
ktree_fr_k3	10	ARIEL	0.0060	0.0017	0.00	5.42	5.42	0.65	–
ktree_fr_k3	10	CVE_{cm}	0.2993	0.2810	0.40	46.11	46.51	0.65	3.00
ktree_fr_k3	10	IBP	0.0032	0.0021	0.40	1.74	2.13	0.65	3.00
ktree_fr_k3	20	CDVE	0.0013	0.0017	0.85	26.67	27.52	1.57	3.00
ktree_fr_k3	20	ARIEL	0.0006	0.0014	0.00	12.26	12.26	1.57	–
ktree_fr_k3	20	CVE_{cm}	0.4017	0.4124	0.81	959.08	959.88	1.57	3.00
ktree_fr_k3	20	IBP	0.0007	0.0009	0.83	3.62	4.45	1.57	3.00
ktree_fr_k3	30	CDVE	0.0013	0.0011	1.28	116.25	117.53	2.91	3.00
ktree_fr_k3	30	ARIEL	0.0009	0.0001	0.00	19.01	19.01	2.91	–
ktree_fr_k3	30	IBP	0.0009	0.0004	1.26	6.38	7.64	2.91	3.00
ktree_fr_k3	40	CDVE	0.0027	0.0020	1.71	367.85	369.55	4.80	3.00
ktree_fr_k3	40	ARIEL	0.0003	0.0004	0.00	25.41	25.41	4.80	–
ktree_fr_k3	40	IBP	0.0003	0.0003	1.68	8.70	10.38	4.80	3.00
ktree_fr_k3	50	CDVE	0.0015	0.0022	3.68	845.10	848.78	6.89	3.00
ktree_fr_k3	50	ARIEL	0.0001	0.0004	0.00	31.32	31.32	6.89	–
ktree_fr_k3	50	IBP	0.0004	0.0003	3.64	11.44	15.08	6.89	3.00
ktree_fr_k3	100	ARIEL	0.0003	0.0007	0.00	64.03	64.03	22.71	–
ktree_fr_k3	100	IBP	0.0002	0.0005	10.55	25.34	35.89	22.71	3.00

Table 18: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
ktree_fr_k3	10	CDVE	0.0031	0.0095	0.40	2.14	2.55	0.65	3.00
ktree_fr_k3	10	ARIEL	0.0242	0.0128	0.00	5.42	5.42	0.65	–
ktree_fr_k3	10	CVE_{cm}	0.1690	0.1673	0.40	46.11	46.51	0.65	3.00
ktree_fr_k3	10	IBP	0.0128	0.0113	0.40	1.74	2.13	0.65	3.00
ktree_fr_k3	20	CDVE	0.0122	0.0111	0.85	26.67	27.52	1.57	3.00
ktree_fr_k3	20	ARIEL	0.0041	0.0116	0.00	12.26	12.26	1.57	–
ktree_fr_k3	20	CVE_{cm}	0.1572	0.1476	0.81	959.08	959.88	1.57	3.00
ktree_fr_k3	20	IBP	0.0038	0.0068	0.83	3.62	4.45	1.57	3.00
ktree_fr_k3	30	CDVE	0.0085	0.0075	1.28	116.25	117.53	2.91	3.00
ktree_fr_k3	30	ARIEL	0.0055	0.0011	0.00	19.01	19.01	2.91	–
ktree_fr_k3	30	IBP	0.0055	0.0032	1.26	6.38	7.64	2.91	3.00
ktree_fr_k3	40	CDVE	0.0161	0.0119	1.71	367.85	369.55	4.80	3.00
ktree_fr_k3	40	ARIEL	0.0034	0.0065	0.00	25.41	25.41	4.80	–
ktree_fr_k3	40	IBP	0.0029	0.0036	1.68	8.70	10.38	4.80	3.00
ktree_fr_k3	50	CDVE	0.0087	0.0133	3.68	845.10	848.78	6.89	3.00
ktree_fr_k3	50	ARIEL	0.0010	0.0077	0.00	31.32	31.32	6.89	–
ktree_fr_k3	50	IBP	0.0042	0.0030	3.64	11.44	15.08	6.89	3.00
ktree_fr_k3	100	ARIEL	0.0037	0.0090	0.00	64.03	64.03	22.71	–
ktree_fr_k3	100	IBP	0.0024	0.0048	10.55	25.34	35.89	22.71	3.00

Table 19: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	5	CDVE	0.0039	0.0020	0.13	0.06	0.20	0.42	2.00
dag_fr_tw3	5	ARIEL	0.0090	0.0123	0.00	3.38	3.38	0.42	–
dag_fr_tw3	5	CVE	0.0006	0.0017	0.13	250.50	250.64	0.42	2.00
dag_fr_tw3	5	CVE_{cm}	0.0619	0.0578	0.13	0.03	0.16	0.42	2.00
dag_fr_tw3	5	IBP	0.0021	0.0020	0.13	0.01	0.13	0.42	2.00
dag_fr_tw3	6	CDVE	0.0031	0.0037	0.14	0.09	0.23	0.49	1.90
dag_fr_tw3	6	ARIEL	0.0008	0.0012	0.00	7.96	7.96	0.49	–
dag_fr_tw3	6	CVE	0.0031	0.0019	0.15	267.68	267.84	0.49	1.90
dag_fr_tw3	6	CVE_{cm}	0.0517	0.0569	0.14	0.04	0.18	0.49	1.90
dag_fr_tw3	6	IBP	0.0033	0.0020	0.13	0.01	0.14	0.49	1.90
dag_fr_tw3	7	CDVE	0.0076	0.0045	0.17	0.15	0.32	0.79	1.80
dag_fr_tw3	7	ARIEL	0.0000	0.0000	0.00	1.92	1.92	0.79	–
dag_fr_tw3	7	CVE	0.0031	0.0020	0.13	282.36	282.49	0.79	1.50
dag_fr_tw3	7	CVE_{cm}	0.0677	0.0778	0.17	0.07	0.23	0.79	1.80
dag_fr_tw3	7	IBP	0.0073	0.0041	0.16	0.01	0.17	0.79	1.80
dag_fr_tw3	8	CDVE	0.0015	0.0035	0.24	0.33	0.57	1.10	2.20
dag_fr_tw3	8	ARIEL	0.0029	0.0007	0.00	3.26	3.26	1.10	–
dag_fr_tw3	8	CVE_{cm}	0.1535	0.1293	0.24	0.14	0.39	1.10	2.20
dag_fr_tw3	8	IBP	0.0032	0.0021	0.24	0.02	0.26	1.10	2.20
dag_fr_tw3	9	CDVE	0.0013	0.0019	0.28	0.49	0.77	1.23	2.20
dag_fr_tw3	9	ARIEL	0.0089	0.0114	0.00	3.50	3.50	1.23	–
dag_fr_tw3	9	CVE_{cm}	0.1521	0.1462	0.27	0.22	0.49	1.23	2.20
dag_fr_tw3	9	IBP	0.0016	0.0004	0.28	0.02	0.30	1.23	2.20
dag_fr_tw3	10	CDVE	0.0021	0.0060	0.31	0.73	1.05	1.62	2.20
dag_fr_tw3	10	ARIEL	0.0029	0.0029	0.00	3.92	3.92	1.62	–
dag_fr_tw3	10	CVE	0.0000	0.0000	0.20	104.02	104.21	1.62	2.00
dag_fr_tw3	10	CVE_{cm}	0.1930	0.1972	0.30	0.30	0.60	1.62	2.20
dag_fr_tw3	10	IBP	0.0044	0.0058	0.31	0.03	0.33	1.62	2.20

Table 20: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	5	CDVE	0.0238	0.0085	0.13	0.06	0.20	0.42	2.00
dag_fr_tw3	5	ARIEL	0.0258	0.0563	0.00	3.38	3.38	0.42	–
dag_fr_tw3	5	CVE	0.0041	0.0085	0.13	250.50	250.64	0.42	2.00
dag_fr_tw3	5	CVE_{cm}	0.0697	0.0798	0.13	0.03	0.16	0.42	2.00
dag_fr_tw3	5	IBP	0.0074	0.0083	0.13	0.01	0.13	0.42	2.00
dag_fr_tw3	6	CDVE	0.0168	0.0163	0.14	0.09	0.23	0.49	1.90
dag_fr_tw3	6	ARIEL	0.0047	0.0067	0.00	7.96	7.96	0.49	–
dag_fr_tw3	6	CVE	0.0168	0.0094	0.15	267.68	267.84	0.49	1.90
dag_fr_tw3	6	CVE_{cm}	0.0878	0.0830	0.14	0.04	0.18	0.49	1.90
dag_fr_tw3	6	IBP	0.0168	0.0092	0.13	0.01	0.14	0.49	1.90
dag_fr_tw3	7	CDVE	0.0205	0.0150	0.17	0.15	0.32	0.79	1.80
dag_fr_tw3	7	ARIEL	0.0000	0.0001	0.00	1.92	1.92	0.79	–
dag_fr_tw3	7	CVE	0.0160	0.0102	0.13	282.36	282.49	0.79	1.50
dag_fr_tw3	7	CVE_{cm}	0.1090	0.1145	0.17	0.07	0.23	0.79	1.80
dag_fr_tw3	7	IBP	0.0206	0.0143	0.16	0.01	0.17	0.79	1.80
dag_fr_tw3	8	CDVE	0.0061	0.0165	0.24	0.33	0.57	1.10	2.20
dag_fr_tw3	8	ARIEL	0.0136	0.0031	0.00	3.26	3.26	1.10	–
dag_fr_tw3	8	CVE_{cm}	0.1487	0.1343	0.24	0.14	0.39	1.10	2.20
dag_fr_tw3	8	IBP	0.0091	0.0096	0.24	0.02	0.26	1.10	2.20
dag_fr_tw3	9	CDVE	0.0109	0.0163	0.28	0.49	0.77	1.23	2.20
dag_fr_tw3	9	ARIEL	0.0834	0.0838	0.00	3.50	3.50	1.23	–
dag_fr_tw3	9	CVE_{cm}	0.1428	0.1327	0.27	0.22	0.49	1.23	2.20
dag_fr_tw3	9	IBP	0.0110	0.0018	0.28	0.02	0.30	1.23	2.20
dag_fr_tw3	10	CDVE	0.0101	0.0223	0.31	0.73	1.05	1.62	2.20
dag_fr_tw3	10	ARIEL	0.0211	0.0170	0.00	3.92	3.92	1.62	–
dag_fr_tw3	10	CVE	0.0000	0.0000	0.20	104.02	104.21	1.62	2.00
dag_fr_tw3	10	CVE_{cm}	0.1640	0.1650	0.30	0.30	0.60	1.62	2.20
dag_fr_tw3	10	IBP	0.0142	0.0190	0.31	0.03	0.33	1.62	2.20

Table 21: Mean error metrics vs. CJT reference

Type	n	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	10	CDVE	0.0012	0.0020	0.32	0.76	1.07	1.53	2.40
dag_fr_tw3	10	ARIEL	0.0013	0.0038	0.00	18.64	18.64	1.53	–
dag_fr_tw3	10	CVE_{cm}	0.1992	0.2057	0.32	0.41	0.72	1.53	2.40
dag_fr_tw3	10	IBP	0.0019	0.0030	0.31	0.03	0.34	1.53	2.40
dag_fr_tw3	20	CDVE	0.0039	0.0023	0.67	8.53	9.20	4.17	2.60
dag_fr_tw3	20	ARIEL	0.0040	0.0026	0.00	9.46	9.46	4.17	–
dag_fr_tw3	20	CVE_{cm}	0.3072	0.3114	0.65	23.43	24.08	4.17	2.60
dag_fr_tw3	20	IBP	0.0037	0.0026	0.66	0.07	0.73	4.17	2.60
dag_fr_tw3	30	CDVE	0.0025	0.0023	1.03	36.88	37.90	10.25	2.80
dag_fr_tw3	30	ARIEL	0.0013	0.0001	0.00	13.20	13.20	10.25	–
dag_fr_tw3	30	CVE_{cm}	0.3439	0.3465	0.95	284.01	284.96	10.25	2.75
dag_fr_tw3	30	IBP	0.0021	0.0018	0.94	0.09	1.03	10.25	2.80
dag_fr_tw3	40	CDVE	0.0039	0.0036	1.43	110.19	111.62	14.22	3.00
dag_fr_tw3	40	ARIEL	0.0009	0.0034	0.00	32.43	32.43	14.22	–
dag_fr_tw3	40	CVE_{cm}	0.3173	0.3677	1.09	1135.19	1136.29	14.22	3.00
dag_fr_tw3	40	IBP	0.0019	0.0018	1.35	0.17	1.51	14.22	3.00
dag_fr_tw3	50	CDVE	0.0037	0.0032	1.65	238.85	240.50	23.21	3.00
dag_fr_tw3	50	ARIEL	0.0011	0.0008	0.00	24.86	24.86	23.21	–
dag_fr_tw3	50	CVE_{cm}	0.3756	0.3915	1.54	194.40	195.94	23.21	3.00
dag_fr_tw3	50	IBP	0.0027	0.0020	1.67	0.20	1.87	23.21	3.00
dag_fr_tw3	100	CDVE	0.0030	0.0024	3.00	3278.99	3281.98	80.21	3.00
dag_fr_tw3	100	ARIEL	0.0013	0.0010	0.00	56.21	56.21	80.21	–
dag_fr_tw3	100	IBP	0.0013	0.0012	3.34	0.40	3.75	80.21	3.00

Table 22: Standard deviation of error metrics vs. CJT reference

Type	n	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
dag_fr_tw3	10	CDVE	0.0060	0.0079	0.32	0.76	1.07	1.53	2.40
dag_fr_tw3	10	ARIEL	0.0070	0.0196	0.00	18.64	18.64	1.53	–
dag_fr_tw3	10	CVE_{cm}	0.1675	0.1789	0.32	0.41	0.72	1.53	2.40
dag_fr_tw3	10	IBP	0.0066	0.0099	0.31	0.03	0.34	1.53	2.40
dag_fr_tw3	20	CDVE	0.0155	0.0093	0.67	8.53	9.20	4.17	2.60
dag_fr_tw3	20	ARIEL	0.0180	0.0147	0.00	9.46	9.46	4.17	–
dag_fr_tw3	20	CVE_{cm}	0.1853	0.1833	0.65	23.43	24.08	4.17	2.60
dag_fr_tw3	20	IBP	0.0115	0.0099	0.66	0.07	0.73	4.17	2.60
dag_fr_tw3	30	CDVE	0.0092	0.0122	1.03	36.88	37.90	10.25	2.80
dag_fr_tw3	30	ARIEL	0.0118	0.0022	0.00	13.20	13.20	10.25	–
dag_fr_tw3	30	CVE_{cm}	0.1779	0.1827	0.95	284.01	284.96	10.25	2.75
dag_fr_tw3	30	IBP	0.0085	0.0100	0.94	0.09	1.03	10.25	2.80
dag_fr_tw3	40	CDVE	0.0148	0.0189	1.43	110.19	111.62	14.22	3.00
dag_fr_tw3	40	ARIEL	0.0046	0.0312	0.00	32.43	32.43	14.22	–
dag_fr_tw3	40	CVE_{cm}	0.1591	0.2025	1.09	1135.19	1136.29	14.22	3.00
dag_fr_tw3	40	IBP	0.0081	0.0073	1.35	0.17	1.51	14.22	3.00
dag_fr_tw3	50	CDVE	0.0173	0.0148	1.65	238.85	240.50	23.21	3.00
dag_fr_tw3	50	ARIEL	0.0122	0.0063	0.00	24.86	24.86	23.21	–
dag_fr_tw3	50	CVE_{cm}	0.1807	0.1878	1.54	194.40	195.94	23.21	3.00
dag_fr_tw3	50	IBP	0.0124	0.0080	1.67	0.20	1.87	23.21	3.00
dag_fr_tw3	100	CDVE	0.0167	0.0142	3.00	3278.99	3281.98	80.21	3.00
dag_fr_tw3	100	ARIEL	0.0096	0.0083	0.00	56.21	56.21	80.21	–
dag_fr_tw3	100	IBP	0.0062	0.0064	3.34	0.40	3.75	80.21	3.00

I.5 REALISTIC LCNS

The two tables below give the complete per-instance results for the ten Bayesian-network-derived LCNs and the sixteen Supreme Court chain-graph LCNs, including the nine dense `supreme_court` instances omitted from Table 1. On those nine, only ARIEL and the CredalJT reference complete: with n=10 atoms but 345–1009 sentences, the size of the constraint system rather than the treewidth is the binding limit, and IVBP, CDVE and all CVE variants exhaust the budget. One instance in the suite, `benchmark_scaling_directions_n24_2`, is a repaired variant of the original benchmark: in the chain graph the four reviewers of a stage are mutually independent, so the probability of their conjunction is forced to the product of their marginals, which the original's `component` sentences contradicted; the three incompatible sentences were dropped and the repaired instance is consistent, verified by a junction-tree feasibility check.

Table 23: Mean error metrics vs. CJT reference

Instance	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
alarm_consistent.lcn	CDVE	0.1008	0.0146	0.10	0.59	0.69	0.72	2.00
alarm_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.92	3.92	0.72	–
alarm_consistent.lcn	CVE_{cm}	0.0866	0.0670	0.10	0.14	0.24	0.72	2.00
alarm_consistent.lcn	IBP	0.1008	0.0149	0.10	0.03	0.13	0.72	2.00
asia_consistent.lcn	CDVE	0.0033	0.0073	0.08	0.24	0.32	0.45	2.00
asia_consistent.lcn	ARIEL	0.0000	0.0000	0.00	2.86	2.86	0.45	–
asia_consistent.lcn	CVE_{cm}	0.0323	0.0291	0.08	0.08	0.16	0.45	2.00
asia_consistent.lcn	IBP	0.0035	0.0063	0.08	0.01	0.10	0.45	2.00
cancer_consistent.lcn	CDVE	0.0653	0.0272	0.06	0.06	0.12	0.37	2.00
cancer_consistent.lcn	ARIEL	0.0013	0.0139	0.00	2.10	2.10	0.37	–
cancer_consistent.lcn	CVE	0.0653	0.0272	0.06	42.46	42.52	0.37	2.00
cancer_consistent.lcn	CVE_{cm}	0.0491	0.0267	0.06	0.03	0.09	0.37	2.00
cancer_consistent.lcn	IBP	0.0639	0.0392	0.06	0.01	0.07	0.37	2.00
credit_consistent.lcn	CDVE	0.2570	0.0000	0.11	0.58	0.69	1.15	3.00
credit_consistent.lcn	ARIEL	0.0496	0.0000	0.00	3.18	3.18	1.15	–
credit_consistent.lcn	CVE_{cm}	0.1079	0.1650	0.11	1.62	1.73	1.15	3.00
credit_consistent.lcn	IBP	0.2570	0.0000	0.11	0.13	0.24	1.15	3.00
earthquake_consistent.lcn	CDVE	0.0041	0.0000	0.05	0.05	0.10	0.53	2.00
earthquake_consistent.lcn	ARIEL	0.0002	0.0000	0.00	1.88	1.88	0.53	–
earthquake_consistent.lcn	CVE	0.0041	0.0000	0.05	7.63	7.68	0.53	2.00
earthquake_consistent.lcn	CVE_{cm}	0.0019	0.0036	0.05	0.03	0.08	0.53	2.00
earthquake_consistent.lcn	IBP	0.0041	0.0000	0.05	0.01	0.06	0.53	2.00
engine_consistent.lcn	CDVE	0.0591	0.0104	0.10	0.43	0.53	0.53	2.00
engine_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.25	3.25	0.53	–
engine_consistent.lcn	CVE_{cm}	0.1234	0.0839	0.10	0.32	0.42	0.53	2.00
engine_consistent.lcn	IBP	0.0591	0.0104	0.10	0.01	0.12	0.53	2.00
hepatitis_consistent.lcn	CDVE	0.0703	0.0851	0.24	0.90	1.15	0.56	3.00
hepatitis_consistent.lcn	ARIEL	0.0084	0.0075	0.00	3.91	3.91	0.56	–
hepatitis_consistent.lcn	CVE_{cm}	0.1137	0.1838	0.24	16.29	16.53	0.56	3.00
hepatitis_consistent.lcn	IBP	0.0686	0.0797	0.24	0.50	0.75	0.56	3.00
suicide_consistent.lcn	CDVE	0.0849	0.0302	0.10	0.52	0.62	0.87	2.00
suicide_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.74	3.74	0.87	–
suicide_consistent.lcn	CVE_{cm}	0.0970	0.0513	0.10	0.13	0.23	0.87	2.00
suicide_consistent.lcn	IBP	0.0849	0.0302	0.10	0.02	0.12	0.87	2.00
tank_consistent.lcn	CDVE	0.1149	0.0751	0.30	0.87	1.18	0.77	3.00
tank_consistent.lcn	ARIEL	0.0048	0.0034	0.00	5.02	5.02	0.77	–
tank_consistent.lcn	CVE_{cm}	0.0653	0.0802	0.31	0.19	0.50	0.77	3.00
tank_consistent.lcn	IBP	0.1163	0.0739	0.30	0.02	0.32	0.77	3.00
toy_consistent.lcn	CDVE	0.0891	0.0745	0.09	0.03	0.12	0.16	2.00
toy_consistent.lcn	ARIEL	0.0000	0.0000	0.00	1.60	1.60	0.16	–
toy_consistent.lcn	CVE_{cm}	0.0742	0.0294	0.09	0.01	0.10	0.16	2.00
toy_consistent.lcn	IBP	0.0891	0.0745	0.09	0.01	0.09	0.16	2.00

Table 24: Standard deviation of error metrics vs. CJT reference

Instance	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
alarm_consistent.lcn	CDVE	0.1476	0.0203	0.10	0.59	0.69	0.72	2.00
alarm_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.92	3.92	0.72	–
alarm_consistent.lcn	CVE_{cm}	0.1279	0.0854	0.10	0.14	0.24	0.72	2.00
alarm_consistent.lcn	IBP	0.1476	0.0208	0.10	0.03	0.13	0.72	2.00
asia_consistent.lcn	CDVE	0.0043	0.0172	0.08	0.24	0.32	0.45	2.00
asia_consistent.lcn	ARIEL	0.0000	0.0000	0.00	2.86	2.86	0.45	–
asia_consistent.lcn	CVE_{cm}	0.0308	0.0359	0.08	0.08	0.16	0.45	2.00
asia_consistent.lcn	IBP	0.0047	0.0144	0.08	0.01	0.10	0.45	2.00
cancer_consistent.lcn	CDVE	0.0374	0.0320	0.06	0.06	0.12	0.37	2.00
cancer_consistent.lcn	ARIEL	0.0026	0.0278	0.00	2.10	2.10	0.37	–
cancer_consistent.lcn	CVE	0.0374	0.0320	0.06	42.46	42.52	0.37	2.00
cancer_consistent.lcn	CVE_{cm}	0.0397	0.0233	0.06	0.03	0.09	0.37	2.00
cancer_consistent.lcn	IBP	0.0369	0.0323	0.06	0.01	0.07	0.37	2.00
credit_consistent.lcn	CDVE	0.2149	0.0000	0.11	0.58	0.69	1.15	3.00
credit_consistent.lcn	ARIEL	0.0414	0.0000	0.00	3.18	3.18	1.15	–
credit_consistent.lcn	CVE_{cm}	0.1401	0.1170	0.11	1.62	1.73	1.15	3.00
credit_consistent.lcn	IBP	0.2149	0.0000	0.11	0.13	0.24	1.15	3.00
earthquake_consistent.lcn	CDVE	0.0035	0.0000	0.05	0.05	0.10	0.53	2.00
earthquake_consistent.lcn	ARIEL	0.0003	0.0000	0.00	1.88	1.88	0.53	–
earthquake_consistent.lcn	CVE	0.0035	0.0000	0.05	7.63	7.68	0.53	2.00
earthquake_consistent.lcn	CVE_{cm}	0.0029	0.0043	0.05	0.03	0.08	0.53	2.00
earthquake_consistent.lcn	IBP	0.0035	0.0000	0.05	0.01	0.06	0.53	2.00
engine_consistent.lcn	CDVE	0.1131	0.0131	0.10	0.43	0.53	0.53	2.00
engine_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.25	3.25	0.53	–
engine_consistent.lcn	CVE_{cm}	0.1318	0.0677	0.10	0.32	0.42	0.53	2.00
engine_consistent.lcn	IBP	0.1131	0.0131	0.10	0.01	0.12	0.53	2.00
hepatitis_consistent.lcn	CDVE	0.1232	0.1429	0.24	0.90	1.15	0.56	3.00
hepatitis_consistent.lcn	ARIEL	0.0252	0.0167	0.00	3.91	3.91	0.56	–
hepatitis_consistent.lcn	CVE_{cm}	0.1580	0.1630	0.24	16.29	16.53	0.56	3.00
hepatitis_consistent.lcn	IBP	0.1236	0.1445	0.24	0.50	0.75	0.56	3.00
suicide_consistent.lcn	CDVE	0.1160	0.0430	0.10	0.52	0.62	0.87	2.00
suicide_consistent.lcn	ARIEL	0.0000	0.0000	0.00	3.74	3.74	0.87	–
suicide_consistent.lcn	CVE_{cm}	0.1147	0.0637	0.10	0.13	0.23	0.87	2.00
suicide_consistent.lcn	IBP	0.1160	0.0430	0.10	0.02	0.12	0.87	2.00
tank_consistent.lcn	CDVE	0.1337	0.1096	0.30	0.87	1.18	0.77	3.00
tank_consistent.lcn	ARIEL	0.0144	0.0102	0.00	5.02	5.02	0.77	–
tank_consistent.lcn	CVE_{cm}	0.0659	0.0555	0.31	0.19	0.50	0.77	3.00
tank_consistent.lcn	IBP	0.1351	0.1089	0.30	0.02	0.32	0.77	3.00
toy_consistent.lcn	CDVE	0.1363	0.0751	0.09	0.03	0.12	0.16	2.00
toy_consistent.lcn	ARIEL	0.0000	0.0000	0.00	1.60	1.60	0.16	–
toy_consistent.lcn	CVE_{cm}	0.1190	0.0345	0.09	0.01	0.10	0.16	2.00
toy_consistent.lcn	IBP	0.1363	0.0751	0.09	0.01	0.09	0.16	2.00

Table 25: Mean error metrics vs. CJT reference

Instance	Algorithm	MAE_{lb}	MAE_{ub}	Build	Run	Total	Ref	IW
benchmark_scaling_directions_n24_2.lcn	CDVE	0.1215	0.4370	1.78	22.80	24.57	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	ARIEL	0.0000	0.0000	0.00	13.42	13.42	1258.24	–
benchmark_scaling_directions_n24_2.lcn	CVE_{cm}	0.1123	0.1749	1.86	2206.67	2208.54	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	IBP	0.1215	0.4370	1.91	685.00	686.91	1258.24	4.00
benchmark_scaling_directions_n32_1.lcn	CDVE	0.1148	0.2247	0.57	19.98	20.55	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	ARIEL	0.0000	0.0000	0.00	19.80	19.80	2.42	–
benchmark_scaling_directions_n32_1.lcn	CVE_{cm}	0.0643	0.0716	0.59	8.59	9.18	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	IBP	0.1148	0.2247	0.64	0.05	0.69	2.42	1.00
maintenance_scaling_n12_3.lcn	CDVE	0.1287	0.5013	0.29	0.94	1.23	0.66	2.00
maintenance_scaling_n12_3.lcn	ARIEL	0.0284	0.0000	0.00	8.13	8.13	0.66	–
maintenance_scaling_n12_3.lcn	CVE_{cm}	0.0793	0.2270	0.31	0.25	0.57	0.66	2.00
maintenance_scaling_n12_3.lcn	IBP	0.1286	0.4926	0.31	0.04	0.35	0.66	2.00
maintenance_scaling_n6_1.lcn	CDVE	0.1029	0.3972	0.17	0.10	0.27	0.28	2.00
maintenance_scaling_n6_1.lcn	ARIEL	0.0000	0.0000	0.00	4.16	4.16	0.28	–
maintenance_scaling_n6_1.lcn	CVE	0.1029	0.4017	0.14	789.77	789.91	0.28	2.00
maintenance_scaling_n6_1.lcn	CVE_{cm}	0.1021	0.2685	0.14	0.05	0.19	0.28	2.00
maintenance_scaling_n6_1.lcn	IBP	0.1029	0.4017	0.15	0.01	0.16	0.28	2.00
maintenance_scaling_n9_2.lcn	CDVE	0.1447	0.4748	0.21	0.48	0.69	0.44	2.00
maintenance_scaling_n9_2.lcn	ARIEL	0.0398	0.0000	0.00	6.09	6.09	0.44	–
maintenance_scaling_n9_2.lcn	CVE_{cm}	0.1095	0.2437	0.20	0.12	0.32	0.44	2.00
maintenance_scaling_n9_2.lcn	IBP	0.1447	0.4662	0.23	0.02	0.25	0.44	2.00
paper_scaling_directions_n35_2.lcn	CDVE	0.1506	0.4467	7.59	190.48	198.07	1342.14	5.00
paper_scaling_directions_n35_2.lcn	ARIEL	0.0064	0.0047	0.00	36.58	36.58	1342.14	–
paper_scaling_directions_n40_1.lcn	CDVE	0.1288	0.2631	0.70	50.55	51.25	3.27	1.00
paper_scaling_directions_n40_1.lcn	ARIEL	0.0000	0.0000	0.00	34.72	34.72	3.27	–
paper_scaling_directions_n40_1.lcn	CVE_{cm}	0.0696	0.0566	0.74	658.10	658.84	3.27	1.00
paper_scaling_directions_n40_1.lcn	IBP	0.1288	0.2631	0.82	0.09	0.91	3.27	1.00
supreme_court_judicial_power_credal_source_comparison_n10_1.lcn	ARIEL	0.0532	0.1234	0.00	115.72	115.72	348.07	–
supreme_court_judicial_power_credal_source_comparison_n10_2.lcn	ARIEL	0.0321	0.1107	0.00	76.72	76.72	337.23	–
supreme_court_judicial_power_credal_source_comparison_n10_3.lcn	ARIEL	0.0259	0.0972	0.00	527.92	527.92	426.93	–
supreme_court_judicial_power_dual_source_credal_n10_1.lcn	ARIEL	0.0259	0.0972	0.00	523.64	523.64	435.71	–
supreme_court_judicial_power_release_comparison_n10_1.lcn	ARIEL	0.0317	0.0860	0.00	68.03	68.03	346.18	–
supreme_court_judicial_power_release_comparison_n10_2.lcn	ARIEL	0.0532	0.1234	0.00	117.31	117.31	337.95	–
supreme_court_judicial_power_release_comparison_n10_3.lcn	ARIEL	0.0259	0.0972	0.00	562.73	562.73	426.77	–
supreme_court_judicial_power_source_variants_n10_1.lcn	ARIEL	0.0317	0.0860	0.00	71.64	71.64	338.65	–
supreme_court_judicial_power_source_variants_n10_2.lcn	ARIEL	0.0176	0.0913	0.00	70.58	70.58	357.82	–

Table 26: Standard deviation of error metrics vs. CJT reference

Instance	Algorithm	SD_{lb}	SD_{ub}	Build	Run	Total	Ref	IW
benchmark_scaling_directions_n24_2.lcn	CDVE	0.0648	0.1928	1.78	22.80	24.57	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	ARIEL	0.0000	0.0000	0.00	13.42	13.42	1258.24	–
benchmark_scaling_directions_n24_2.lcn	CVE_{cm}	0.0780	0.1686	1.86	2206.67	2208.54	1258.24	4.00
benchmark_scaling_directions_n24_2.lcn	IBP	0.0648	0.1928	1.91	685.00	686.91	1258.24	4.00
benchmark_scaling_directions_n32_1.lcn	CDVE	0.0657	0.1463	0.57	19.98	20.55	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	ARIEL	0.0000	0.0000	0.00	19.80	19.80	2.42	–
benchmark_scaling_directions_n32_1.lcn	CVE_{cm}	0.0430	0.0738	0.59	8.59	9.18	2.42	1.00
benchmark_scaling_directions_n32_1.lcn	IBP	0.0657	0.1463	0.64	0.05	0.69	2.42	1.00
maintenance_scaling_n12_3.lcn	CDVE	0.0795	0.2363	0.29	0.94	1.23	0.66	2.00
maintenance_scaling_n12_3.lcn	ARIEL	0.0514	0.0000	0.00	8.13	8.13	0.66	–
maintenance_scaling_n12_3.lcn	CVE_{cm}	0.0491	0.1572	0.31	0.25	0.57	0.66	2.00
maintenance_scaling_n12_3.lcn	IBP	0.0794	0.2313	0.31	0.04	0.35	0.66	2.00
maintenance_scaling_n6_1.lcn	CDVE	0.0729	0.2197	0.17	0.10	0.27	0.28	2.00
maintenance_scaling_n6_1.lcn	ARIEL	0.0000	0.0000	0.00	4.16	4.16	0.28	–
maintenance_scaling_n6_1.lcn	CVE	0.0729	0.2232	0.14	789.77	789.91	0.28	2.00
maintenance_scaling_n6_1.lcn	CVE_{cm}	0.0710	0.1599	0.14	0.05	0.19	0.28	2.00
maintenance_scaling_n6_1.lcn	IBP	0.0729	0.2232	0.15	0.01	0.16	0.28	2.00
maintenance_scaling_n9_2.lcn	CDVE	0.0953	0.2482	0.21	0.48	0.69	0.44	2.00
maintenance_scaling_n9_2.lcn	ARIEL	0.0496	0.0000	0.00	6.09	6.09	0.44	–
maintenance_scaling_n9_2.lcn	CVE_{cm}	0.0699	0.1858	0.20	0.12	0.32	0.44	2.00
maintenance_scaling_n9_2.lcn	IBP	0.0952	0.2428	0.23	0.02	0.25	0.44	2.00
paper_scaling_directions_n35_2.lcn	CDVE	0.0682	0.1408	7.59	190.48	198.07	1342.14	5.00
paper_scaling_directions_n35_2.lcn	ARIEL	0.0250	0.0276	0.00	36.58	36.58	1342.14	–
paper_scaling_directions_n40_1.lcn	CDVE	0.0656	0.1334	0.70	50.55	51.25	3.27	1.00
paper_scaling_directions_n40_1.lcn	ARIEL	0.0000	0.0000	0.00	34.72	34.72	3.27	–
paper_scaling_directions_n40_1.lcn	CVE_{cm}	0.0672	0.0365	0.74	658.10	658.84	3.27	1.00
paper_scaling_directions_n40_1.lcn	IBP	0.0656	0.1334	0.82	0.09	0.91	3.27	1.00
supreme_court_judicial_power_credal_source_comparison_n10_1.lcn	ARIEL	0.0620	0.2451	0.00	115.72	115.72	348.07	–
supreme_court_judicial_power_credal_source_comparison_n10_2.lcn	ARIEL	0.0407	0.2477	0.00	76.72	76.72	337.23	–
supreme_court_judicial_power_credal_source_comparison_n10_3.lcn	ARIEL	0.0414	0.2514	0.00	527.92	527.92	426.93	–
supreme_court_judicial_power_dual_source_credal_n10_1.lcn	ARIEL	0.0414	0.2514	0.00	523.64	523.64	435.71	–
supreme_court_judicial_power_release_comparison_n10_1.lcn	ARIEL	0.0624	0.2550	0.00	68.03	68.03	346.18	–
supreme_court_judicial_power_release_comparison_n10_2.lcn	ARIEL	0.0620	0.2451	0.00	117.31	117.31	337.95	–
supreme_court_judicial_power_release_comparison_n10_3.lcn	ARIEL	0.0414	0.2514	0.00	562.73	562.73	426.77	–
supreme_court_judicial_power_source_variants_n10_1.lcn	ARIEL	0.0624	0.2550	0.00	71.64	71.64	338.65	–
supreme_court_judicial_power_source_variants_n10_2.lcn	ARIEL	0.0442	0.2536	0.00	70.58	70.58	357.82	–